ABES Engineering College, Ghaziabad

Department of ASH

Estd. 2000

Web Designing

Year/ Sem. : 1st year/ I sem.

Subject Name : WD Workshop-I

Subject Code : 25VA151

Faculty Name : Ms. Deepti Deshmukh , Mr. Jitendra Kumar Chauhan, Mr. Sanjeev Soni , Ms. Pranshi Verma , Ms. Monika , Mr. Rohit Pratap Singh

Approved by AICTE, New Delhi & Affiliated to Dr. APJ Abdul Kalam Technical University, UP, Lucknow

NBA Accredited Courses (CSE, IT, ECE, EN)

19th KM Stone, NH-24, Ghaziabad-201009, UP

ABES Engineering College, Ghaziabad

VISION

To take ABES Engineering College to such a level that, it is at par with the leading institutions of the world in providing leadership to the international education system and be amongst the top-rated institutions of the world by providing a transformative education to create leaders and innovators embedded in traditional Indian values.

MISSION

- To create an ambiance for healthy teaching learning process.
- To nurture the students and infuse in them:
  - A passion to excel professionally
  - A spirit to be of utmost use to the industry, corporate sector and the society at large
  - An intense desire to take challenging responsibilities and leadership roles
  - A craving to be wholesome good human beings
- To develop an environment for creating new knowledge through research and by thriving to explore innovative ideas.

QUALITY POLICY

To continuously thrive to provide a congenial and wholesome academic environment and a healthy culture for faculty, staff and students which would motivate teacher's full participation with passion and develop an intense desire in the students to acquire comprehensive education and hence become a useful and confident human resource for the industry and academia.

VALUES

Respect – We treat people the way we want to be treated

Integrity – We believe in doing things right when no one is watching

Accountability – We take full responsibility of our actions and are accountable for them

Transparency –We communicate openly amongst ourselves and all our stakeholders to avoid any misconception.

Excellence – We endeavor to think out-of-the-box and deliver maximum value to all our Stakeholders.

ABES Engineering College, Ghaziabad
Department of ASH

VISION

The department of ASH has a distinct vision to nurture and inculcate knowledge and understanding of engineering sciences amongst the budding technocrats and groom the students with intellectual and professional strength. ASH is an anvil to chisel future professionals with the skills of engineering, imbibed with social, ethical and moral values, so that the pupils can expedite their efforts to build technologies from knowledge and be a part of the workforce, instrumental in the development of the nation's technological and economic progress.

MISSION

.

M1: To play a pivotal role in laying the foundations of engineering education by building the professional strengths of the students in basic sciences and engineering disciplines and strengthening the foundation of aspiring engineers.

M2: To provide a unique learning environment for the students, preparing the students at par with the global technical workforce, building confidence, character and human values.

M3: To upgrade the quality of technical education and produce dynamic engineers and entrepreneurs for the country, with a sole aim to spot ABESEC amongst the best educational institutions of global standards

PROGRAMME OUTCOMES (POs)

PO 1. Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO 2. Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO 3. Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO 4. Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO 5. Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO 6. The engineer and society: Apply reasoning informed by contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to professional engineering practice.

PO 7. Environment and sustainability: Understand the impact of professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO 8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of engineering practice.

PO 9. Individual and teamwork: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO 10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO 11. Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO 12. Life-long learning: Recognize the need for and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

ABES Engineering College, Ghaziabad

B.Tech. First Year, Semester-I Common to all branches)

Course Code	25VA151	Course Name	Web Designing	L	T	P	C
				3	0	0	0

Course Objective

- Introduce students to the fundamentals of web technologies and the full-stack development environment. *
Familiarize students with modern tools like VS Code and Git for development and version control practices.
- Provide hands-on experience with HTML, CSS, and GitHub for collaborative development.
- Develop practical skills in creating structured, styled, and interactive web pages.

Syllabus

Unit-1	**Introduction to Web Technology**	
Introduction to Web Technology, Full Stack Development, Web Designing, Web Development, Client Server Architecture, Introduction to Text Editors, Installation & Set up of VS Code Editor, Introduction of Git & GitHub, Installation & Configuration of Git Account, Introduction to HTML & HTML Features, HTML Page Structure, Elements in HTML (Semantic and Non-Semantic), Types of Elements (Inline/Block), Text formatting: headings, paragraphs, HTML Lists, Unordered Lists, Ordered Lists, Description Lists, Nesting and Mixed Lists, Lists for Navigation Menus, HTML links, Images and attributes, Comment in HTML, HTML Formatting Elements / Special Character & Symbol, Text related Elements (Quotation and Citation Elements), Using Multimedia (Image/Video/Audio/YouTube) in HTML, Creating & Using Hyperlinks, Accessibility and Semantic Considerations, Practical		**10**
Unit-2	**Web Page Development using HTML**	
HTML Table: Creating and using Tables, Basic Table Structure, Table Headers and Captions, HTML Table: Rowspan and Colspan, Table Attributes, Responsive Tables, HTML Table: Nested Tables, Accessibility in Tables, Practical, HTML Form: The <form> Element, Input Elements, Label and Fieldset, HTML Form: Select and Option Tags, TextArea, Button Element, HTML Form: Form Attributes, Input Validation, Accessibility and Usability, What is Git and why use it?, Git setup and config (git config), Creating a local repository: git init, Tracking changes: add, commit, status, log, .gitignore basics, Practical		**9**
Unit-3	**Version Control with Git & GitHub and CSS Fundamentals**	
Creating a GitHub account, Connecting local repos to GitHub: remote add, push, Cloning and pulling repositories, GitHub repository structure and README.md, Basic collaboration terms: fork, pull request, branch (intro), Branching: git branch, git checkout, git switch, Merging branches, handling merge conflicts, GitHub collaboration workflow, Best practices for commits, branches, and messages, Practical with all steps of git, CSS selectors: element, class, ID, Color, fonts, text alignment, font size, Box model: padding, margin, border, Box model: padding, margin, border, Applying styles: inline, internal, external		**11**
Unit-4	**CSS for Advanced Styling and Responsive Layout**	
display: block, inline, inline-block, float and clear, Introduction to Flexbox: justify-content, align-items, flex-direction, Practical in CSS, Units: px, %, em, rem, vh, vw, Media queries and breakpoints, Media queries and breakpoints, Mobile-first design, Responsive navigation concepts, Practicals, Project planning and design, Version control in large projects, Deploy using GitHub Pages, Final portfolio polishing, Project using HTML & CSS		**10**
	Total No. of Lectures	**40**

Text Books
1. Jon Duckett, HTML and CSS: Design and Build Websites, Wiley.
2. Jennifer Robbins, Learning Web Design, O'Reilly.
Reference Books
1. Ethan Brown, Web Development with Node and Express, O'Reilly.
2. Paul McFedries, HTML, CSS & JavaScript Web Publishing in One Hour a Day, SAMS.

Internal Practical Examination Evaluation Process (Weightage 50%)

Components of Internal Marks (50) For Lab Courses			
Exam	**Lab ST1 (20)**	**Lab ST2 (20)**	**Attendance**
Lab	Quiz: 5 Lab Records & Viva:10 Exp. Conduction: 5	Quiz: 5 Lab Records & Viva: 0 Exp. Conduction: 5	**10**
Duration	During the Regular Time Table of the Lab	During the Regular Time Table of the Lab	**Criteria decided By EC**
Syllabus	After First 5 Experiments	After First 10 Experiments	

External Practical Examination Evaluation Process (Weightage 50%)

- Duration: 2 Hours
- Marks: 50 (Viva Voce: 25 + Practical Exam: 25)
- Syllabus: All Experiments
- Project Evaluation: (Project Execution/ Viva Voce/Report

Table of Index

1.0 Introduction to Web Technology Introduction to Web Technology, Full Stack Development, Web Designing

1.0.0 What is the web?

The web, also known as the World Wide Web (WWW).
The Web (World Wide Web) is a system of websites and webpages that can be accessed through the Internet using browsers.

1.0.1 Evolution of Web

Web 1.0: Static, read-only websites.
Web 2.0: Interactive, user-driven content (social media, blogs).
Web 3.0: AI-driven, decentralized, blockchain-based, semantic web.

Some web components

Web–Page

A document which can be displayed in a web browser such as Firefox, Google Chrome, Opera, Microsoft Internet Explorer or Edge, or Apple's Safari.

Web-Site

Collection of web pages which are grouped together and usually connected together in various ways.

Web Server

A web server is a computer that runs websites.

A web server is a software that Serves web content through the HTTP protocol

1.0.2 Web Technology

"Web Technology is the technology used to make and run websites on the Internet."

"Web Technology refers to the collection of tools, languages, and protocols that allow computers and devices to communicate through the World Wide Web (WWW)."

These technologies allow users to browse websites, interact with online content, and share information over the internet.

Components of Web Technology

1. HTML (Hypertext Markup Language)

- HTML is the foundational markup language of the web.
- It defines the structure of web pages by specifying elements such as headings, paragraphs, lists, images, and links.
- HTML acts as the starting point for any website or web application.

2. CSS (Cascading Style Sheets):

- While HTML defines the structure, CSS is responsible for the appearance.
- It controls the design, including colors, fonts, layout, and responsiveness, ensuring that websites look good across various devices and screen sizes.

3. JavaScript:

- JavaScript is a programming language that adds interactivity to websites.
- It enables dynamic content, such as clickable buttons, animations, and real-time form validation.
- JavaScript is often used alongside HTML and CSS to create seamless user experiences.

4. Web Browsers:

- Browsers like Google Chrome, Mozilla Firefox, and Safari interpret HTML, CSS, and JavaScript and render websites on users' devices.
- They play a crucial role in ensuring a smooth user experience.

5. Web Servers:

- Web servers host websites and serve content over the internet.
- They store files and data that make up a website and deliver them to users when requested.
- Web servers use protocols like **HTTP** (Hypertext Transfer Protocol) and **HTTPS** (HTTP Secure) to communicate with browsers.

6. Web Protocols:

- Protocols are the rules that govern communication between web servers and browsers.
- **HTTP** is the most common protocol, but **HTTPS,** which encrypts data for security, is becoming more prevalent.

- Newer protocols like **HTTP/2** and **HTTP/3** offer faster and more efficient communication.

7. APIs (Application Programming Interfaces):

- APIs are essential for modern web applications.
- They allow different software systems to communicate with each other.
- **Example**, a weather website may use an API to fetch real-time data from a third-party weather service.

8. Databases:

- Databases store and manage data for websites and applications.
- They allow for efficient storage, retrieval, and updating of data.
- Examples of databases used in web technology include **MySQL, MongoDB**, and **PostgreSQL**.

1.03 Types of Web Technologies

1. Frontend Technologies

Frontend technologies are used to build the user-facing part of a website or web application. They include:

- **HTML, CSS, and JavaScript:** The trio of core languages for designing and developing user interfaces.
- **Frontend Frameworks:** Tools like **React, Vue.js**, and **Angular** help developers quickly build complex, interactive user interfaces and manage state.

2. Backend Technologies

Backend technologies are used to build and manage the server-side of web applications. These include:

- **Server-side Languages:** Python, Ruby, PHP, and **Node.js** are popular backend programming languages used to handle business logic, manage databases, and process user requests.
- **Database Management Systems: MySQL, MongoDB, and PostgreSQL** are commonly used to store and retrieve data on the server-side.
- **Web Servers: Apache, Nginx,** and cloud services like **AWS** and **Google Cloud** host and manage websites.

3. Full-Stack Technologies

Full-stack development involves both frontend and backend development. Frameworks like **Django** (Python), **Ruby on Rails** (Ruby), and **Node.js** (JavaScript) allow developers to handle both client-side and server-side development.

4. Web APIs

Web APIs enable communication between different software components. Some popular API types include:

- **RESTful APIs:** Representational State Transfer (REST) APIs are widely used for client-server communication.
- **GraphQL:** A modern alternative to REST APIs, allowing clients to request only the data they need.
- **WebSockets:** WebSockets allow for real-time, two-way communication between the client and server.

1.04 Web Designing & Development

1.04A Web Design

It involves the color scheme, layout, information flow, and all aspects of UI/UX (user interface and user experience).

Web design covers everything related to a website's visual aesthetics and usability

Adobe Creative Cloud. Software like Photoshop and Illustrator help designers create design elements for web pages, such as graphics and logos.

Graphic design. Graphic design involves crafting visuals and layouts to make the website visually appealing and enhance the overall web experience.

UI/UX design. UI/UX design focuses on creating intuitive user interfaces and seamless user experiences by prioritizing ease of use and navigation.

Figma. This browser-based design tool helps designers do collaborative design work and prototyping, as well as create high-fidelity mock-ups.

1.04B Web Development

Front-end developers

Front-end developers focus on what users interact with on the website. They use HTML, CSS, and JavaScript to bring web designs to life. Key tools and technologies include:

1.04.1.1 CSS preprocessors like LESS or Sass for efficient style management.

1.04.1.2 JavaScript frameworks such as AngularJS, React, Ember.js, and Vue.js have revolutionized how developers build dynamic and responsive web applications.

1.04.1.3 Libraries like jQuery.

1.04.1.4 Git and GitHub for version control.

They also use on-site search engine optimization (SEO) to make sure the website ranks high in search engine results.

Back-end developers

Back-end developers handle the data and server-side functions of web applications. Essential tools and skills include:

1.04.1.5 Programming languages like PHP, Python, Java, and C#.

1.04.1.6 Frameworks such as Ruby on Rails, Symfony, and .NET that streamline building robust server-side applications.

1.04.1.7 Database management systems, including MySQL, MongoDB, and PostgreSQL.

1.04.1.8 Security and authentication technologies such as OAuth and Passport for protecting data integrity and user privacy.

Front-end developer	Back-end developer
It is Part of website User Can see Code and Interact it.	It is brain of website that Code never visible to End User
Run on the client side browser	Run on the server side browser
HTML, CSS, JavaScript (and frameworks like Bootstrap, React).	PHP, Python, Java, Node.js and Databases like MySQL, MongoDB.
Designs the layout, look and feel of the website.	Handles the logic, database, and server operations.
Limited Security	Advanced Security

1.1 Client–Server Architecture & Text Editors

1.1.0 Definition & Concept

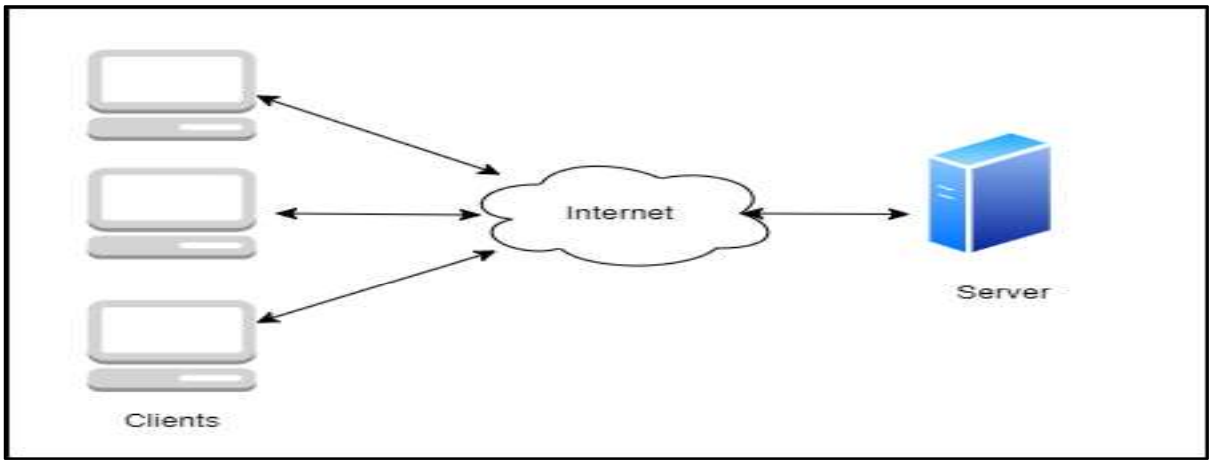
Client-Server Architecture is a model where tasks are divided between two entities:

(i)Client = Request

(ii)Server = Response

A computing model where the client requests services and the server processes and responds.

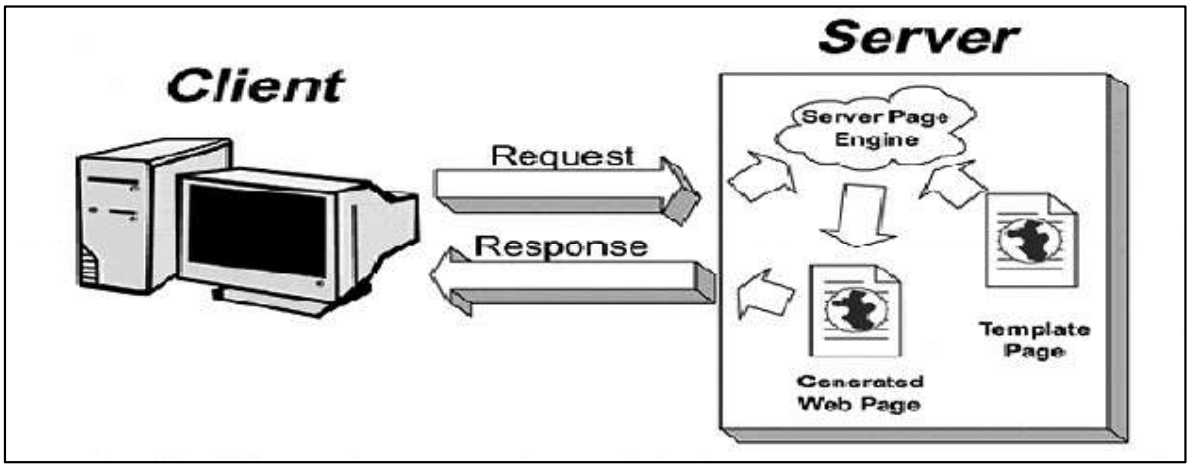
Client-Server Model is a distributed architecture where clients request services and servers provide them



a) **Client:** An application that requests services from the server, such as data retrieval, storage, calculations, or other functions.

b) **Server**: An application that processes client requests, sends responses, or performs specified actions. The server and client may reside on the same machine or different devices across the network. Effective Communication occurs via predefined protocols like HTTP, FTP, or SMTP.

How client requests and server responds



1.1.1 Types of Client-Server Architecture

1. 2-tier Architecture

Client directly communicates with Server.

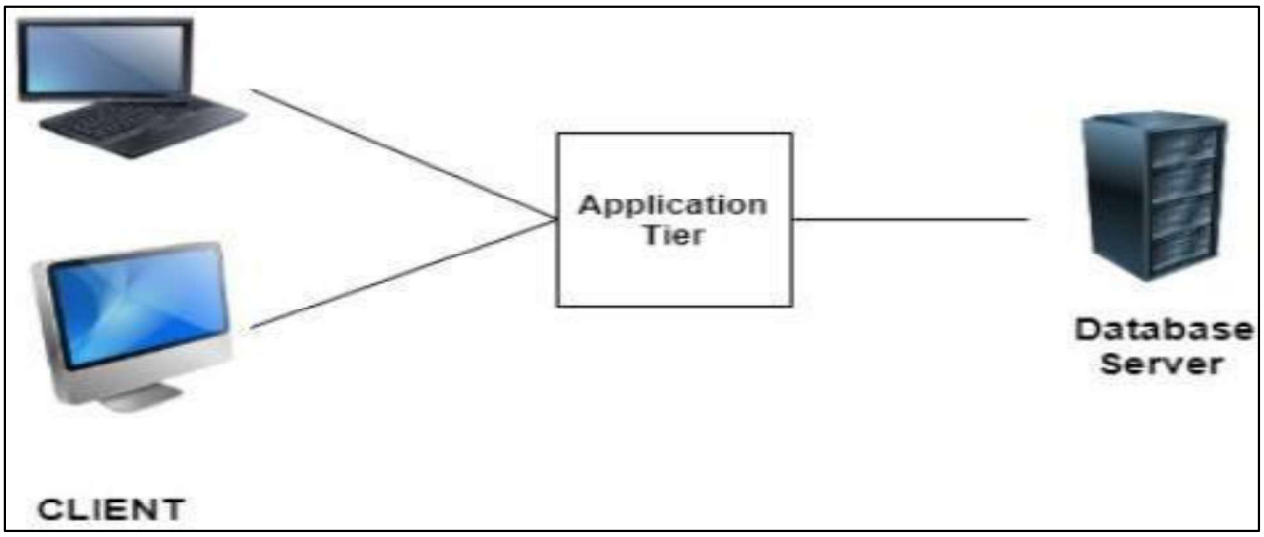
Two tier architecture primarily has two parts, a client tier and a server tier. The client tier sends a request to the server tier and the server tier responds with the desired information.

2. 3-tierArchitecture

Three tier architecture has three layers namely client, application and data layer.

Client → Application Server → Database.

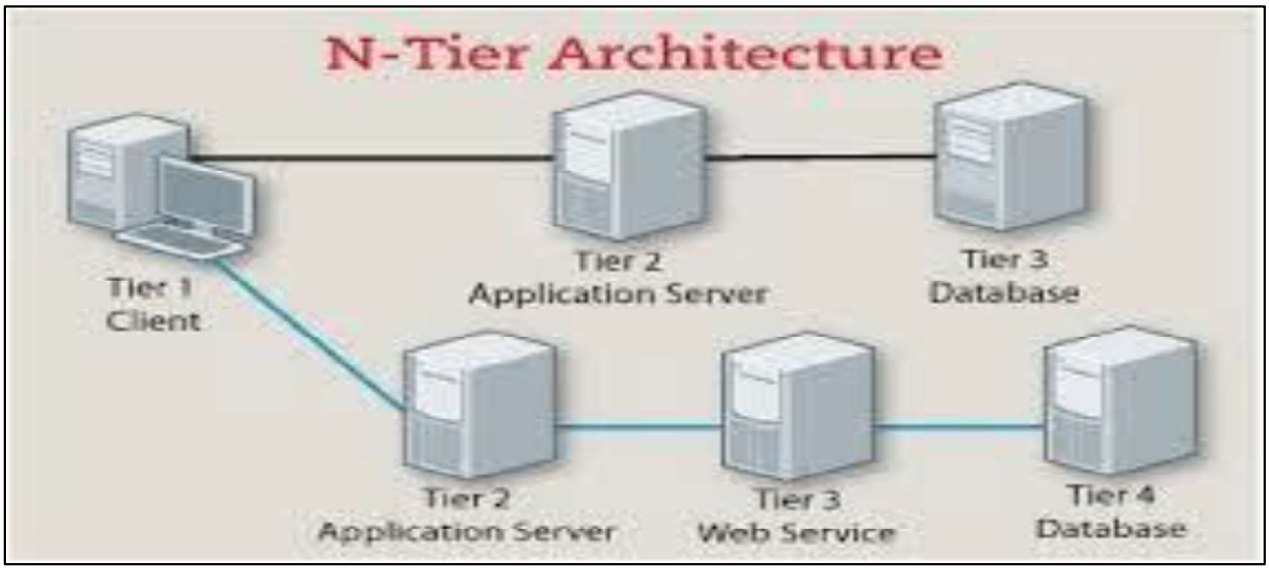
client layer is the one that requests the information. In this case it could be the GUI, web interface etc. The application layer acts as an interface between the client and data layer. It helps in communication and also provides security. The data layer is the one that actually contains the required data.



3. N-tier Architecture

N-tier architecture is also called Distributed Architecture or Multi-tier Architecture. Multiple layers of servers for scalability.

It is similar to three-tier architecture but the number of the application server is increased and represented in individual tiers



1.1.2 Introduction to Text Editors

A Text Editor is software used to write and edit code.

Text editor is a program that lets the user edit text. Whenever you're able to type some text in the input, you are editing the text, and that input is somehow a text editor. By this definition, we can call the HTML input element a basic text editor.

1.12A Basic text editors

The simplest form of text editors is the basic one. In a basic text editor, you can create and edit plain text without any styling or formatting.

- TextEdit (Mac)
- Notepad (Windows)
- Nano (Linux)
- HTML textarea (Web)

1.12B Integrated Development Environments

IDEs are some sort of special text editors that are designed for the programming and software development purposes. Syntax highlighting, intelligence and code completion, version controlling and debugging are the benefits of integrated development environments.

- Visual Studio
- Eclipse
- Jet Brains Products like PyCharm and WebStorm

1.12C Difference between Text Editor & IDE

Text Editor: Lightweight, mainly for writing code (Notepad, Sublime, Atom).

- **IDE:** Advanced, includes debugging, compilation, version control (VS Code, IntelliJ).

Feature	**Text Editor**	**IDE (Integrated Development Environment)**
Definition	A simple tool to write and edit plain text or code.	A complete software suite that provides tools for writing, testing, and debugging code.
Main Purpose	Only for writing and editing text/code.	For software development (coding, compiling, debugging, project management).
Complexity	Lightweight and simple.	Heavy, feature-rich, and advanced.
Examples	Notepad, Notepad++, Sublime Text, VS Code (when used just for editing).	Eclipse, PyCharm, IntelliJ IDEA, Visual Studio, Android Studio.
Features	Syntax highlighting, basic editing.	Code editor + Debugger + Compiler/Interpreter + Build automation + Version control integration.
Usage	Best for small coding tasks or quick edits.	Best for large projects and full application development.
Resource Requirement	Low system resources (RAM, CPU).	Requires more system resources.

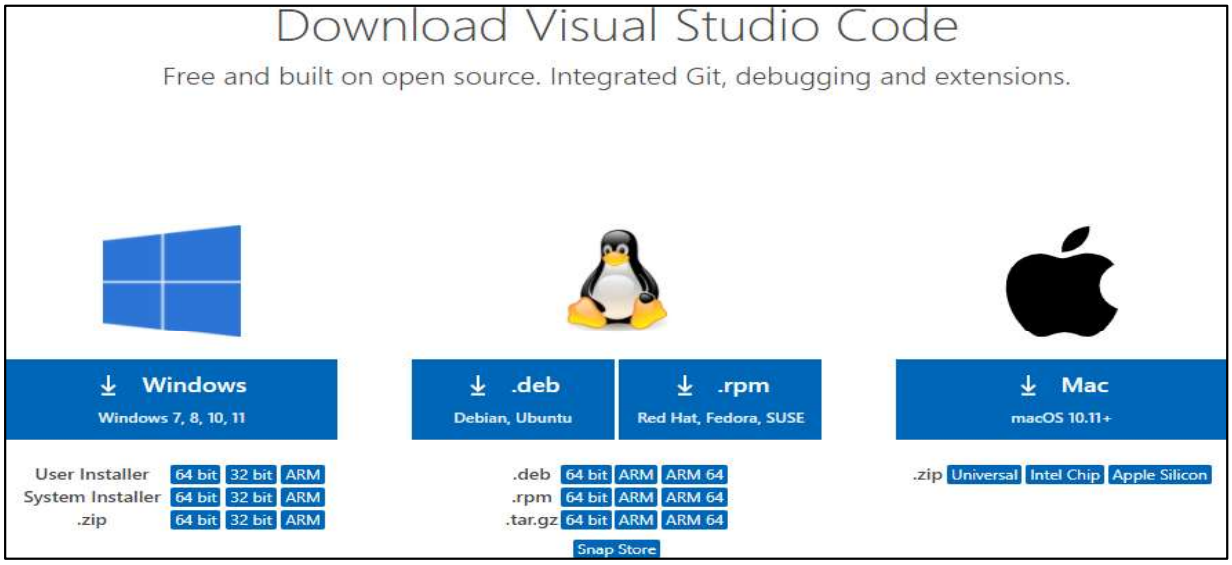
1.2 Installation & Setup of VS Code Editor Installation & Set up of VS Code Editor, Introduction of Git & Git Hub, Installation & Configuration of Git Account

1.2.0 Installation & Setup

Step 1: Downloading VS Code (from official site)

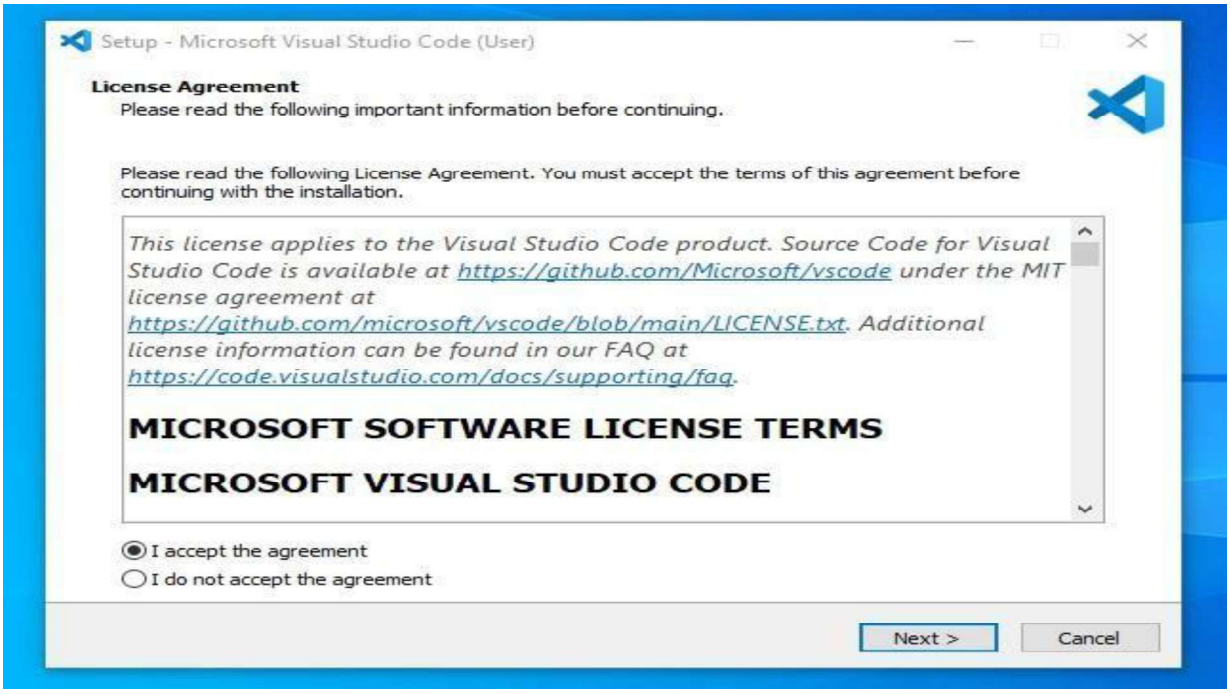
Visit https://code.visualstudio.com/ and download the installer.

Press the "**Download for Windows**" button on the website to start the download of the Visual Studio Code Application.



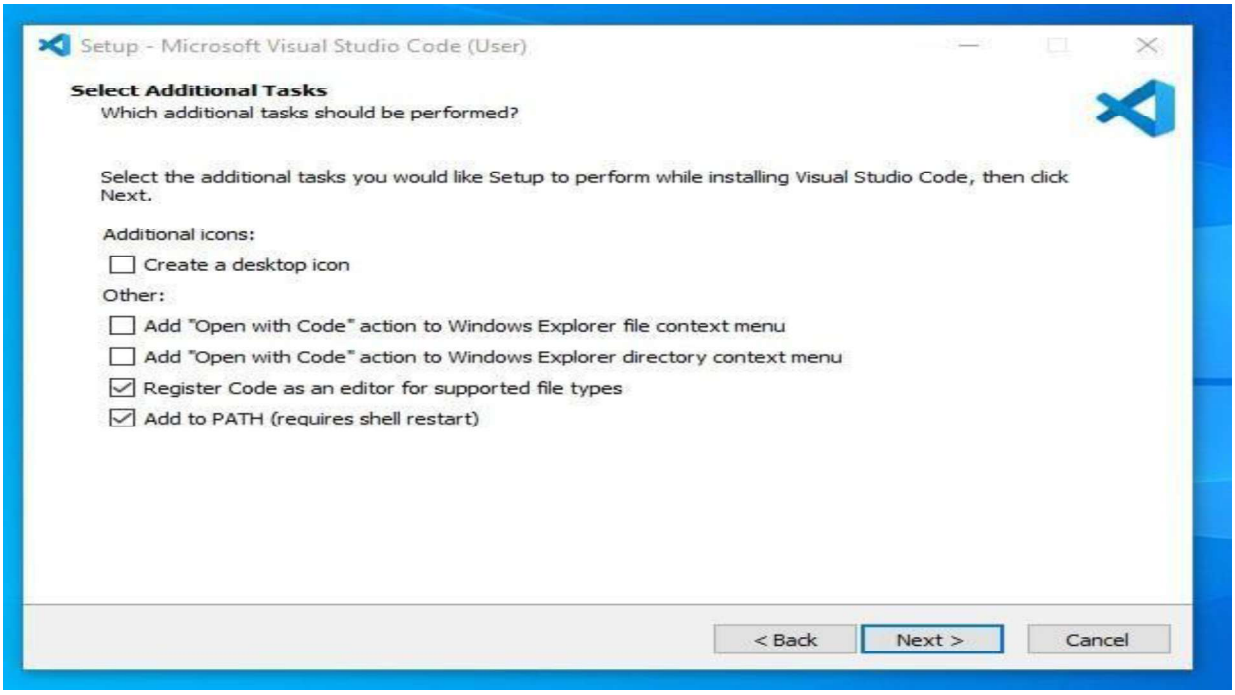
Step 2: Installation process (Windows/Linux/Mac)
Run the installer → Accept License → Choose location → Install.

After the Installer opens, it will ask you to accept the terms and conditions of the Visual Studio Code. Click on **I accept the agreement** and then click the **Next** button.



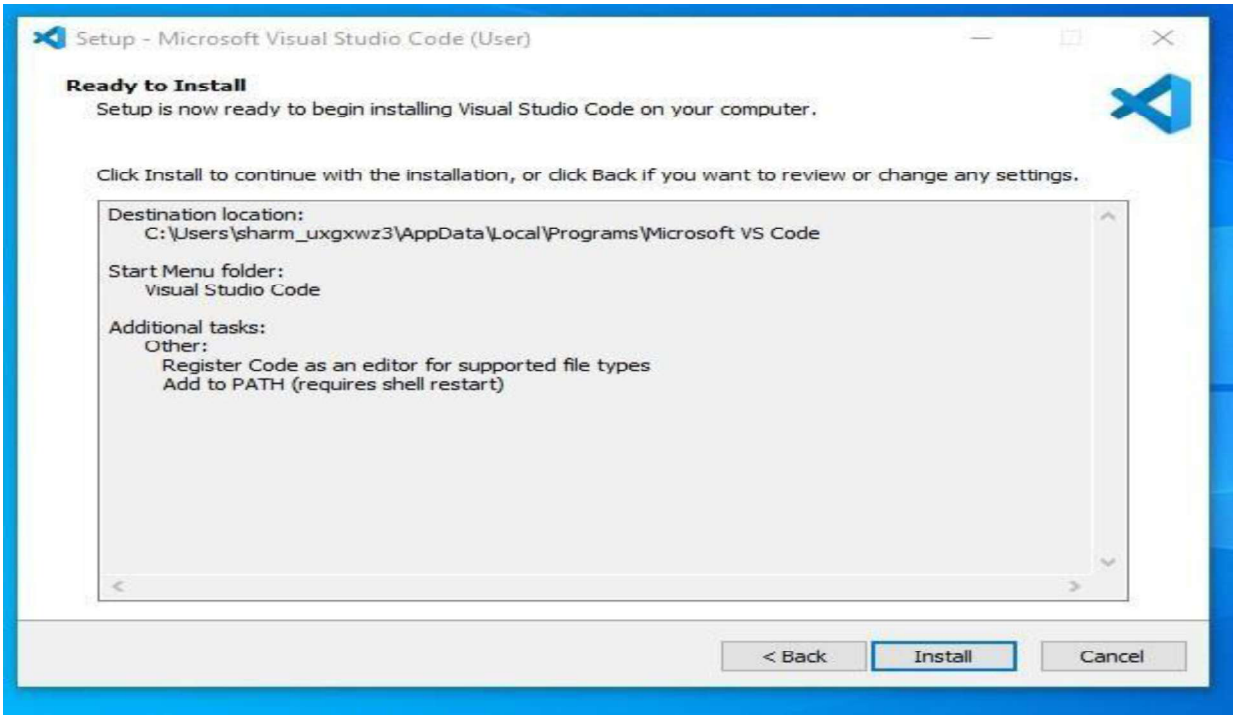
Step 3: Choose the location data for running the Visual Studio Code

Choose the location data for running the Visual Studio Code. It will then ask you to browse the location. Then click on the **Next** button.



Step 4: installation setup.

Then it will ask to begin the installation setup. Click on the **Install** button.

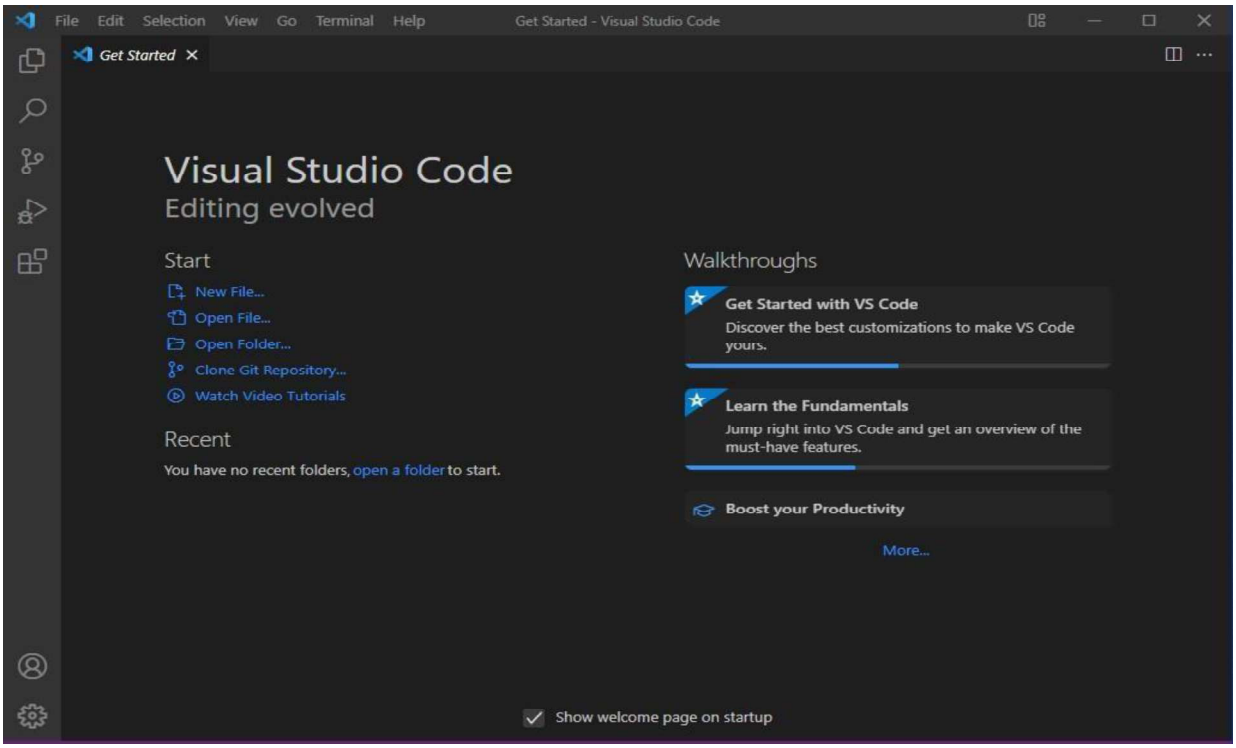


Step 5: Launch Visual Studio Code

After the Installation setup for Visual Studio Code is finished, it will show a window like this below. Tick the "**Launch Visual Studio Code**" checkbox and then click **Next**.

Step 6: Start Your Coding in Visual Studio

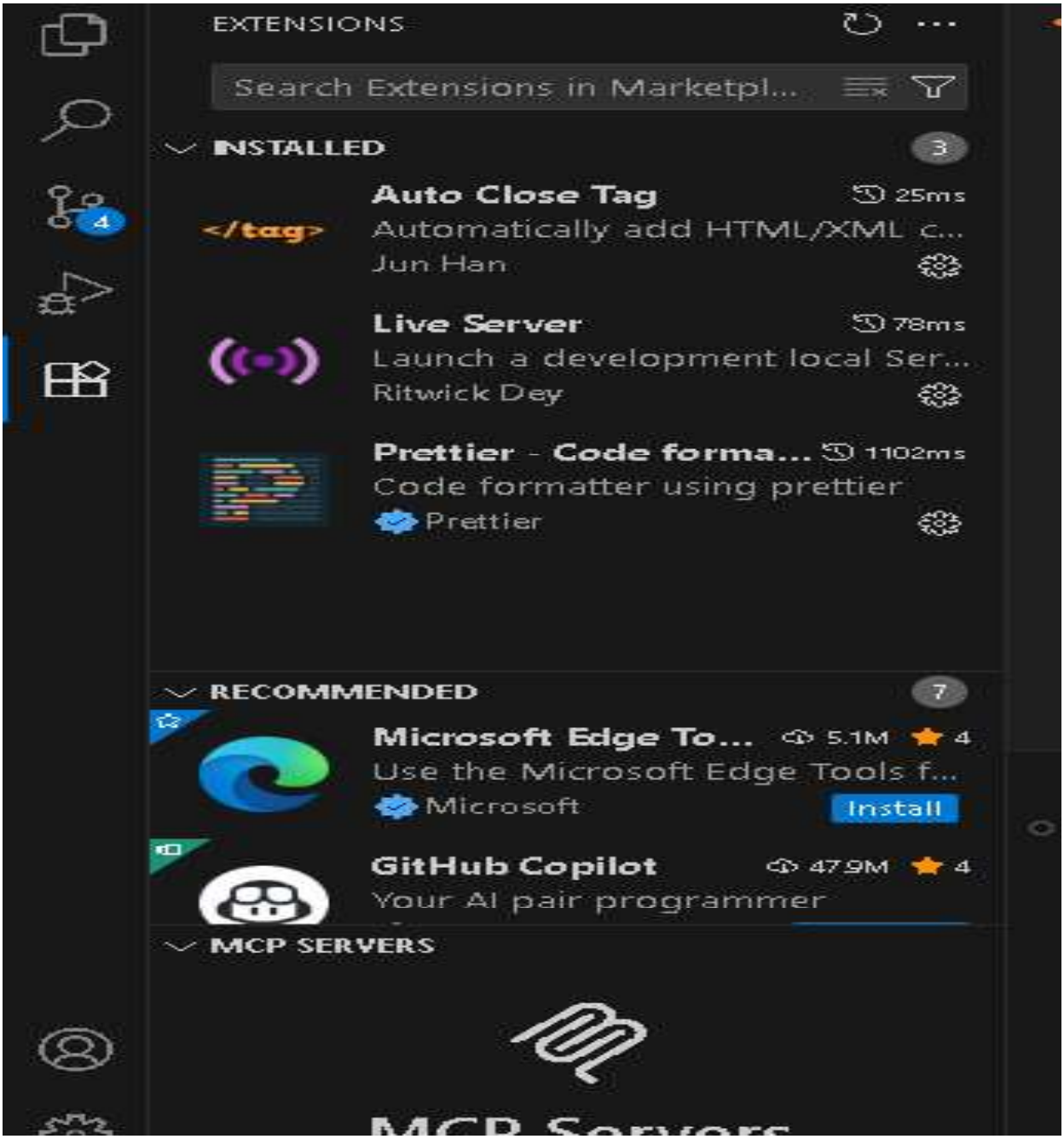
After the previous step, the Visual Studio Code window opens successfully. Now you can create a new file in the Visual Studio Code window and choose a language of yours to begin your programming journey!



1.2.1 How to Add Extensions in VS Code

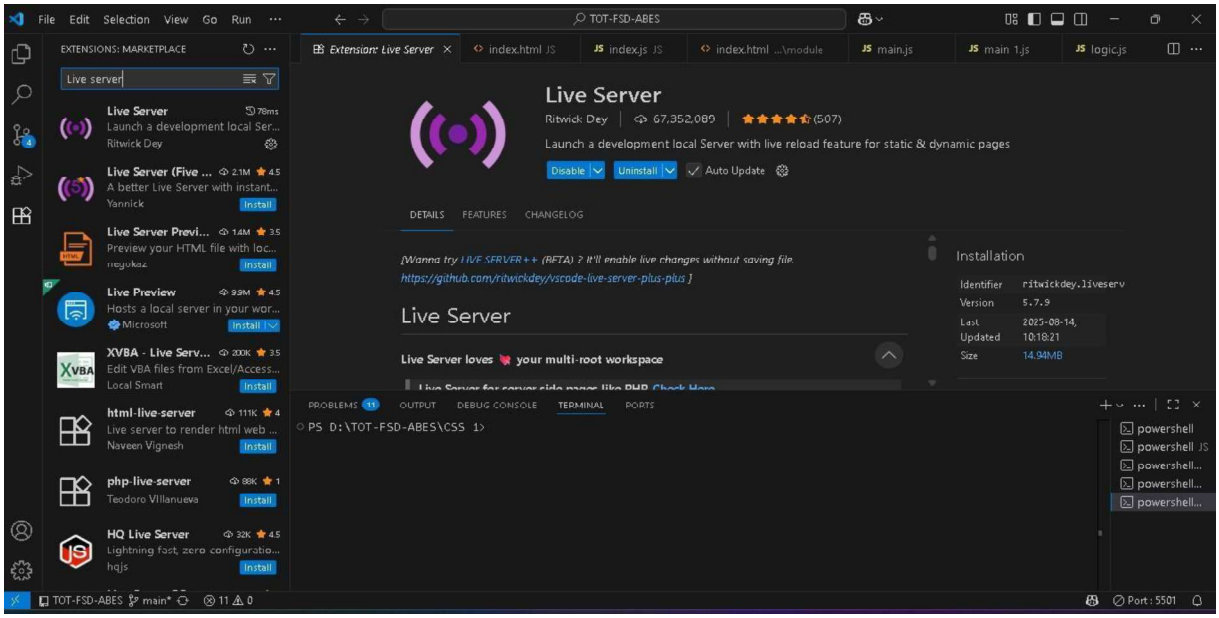
Step 1: Open Extensions Panel

- Open Visual Studio Code.
- On the left sidebar, click the Extensions icon (looks like 4 small squares).
- Shortcut: Ctrl + Shift + X



Step 2: Search for an Extension

- At the top of the Extensions panel, type the name of the extension you want.
- Example: Python, C/C++, Prettier - Code Formatter, Live Server



Step 3: Install the Extension

- From the search results, click the Install button next to the extension.
- It will download and install automatically.

Step 4: Reload if Needed

- Some extensions require a reload of VS Code.
- If prompted, click Reload.

Step 5: Manage Extensions

- Go to the Installed tab in the Extensions panel.
- From here you can:
    - Enable / Disable
    - Update
    - Uninstall

First sample HTML Program in VS Code:

- Open **Visual Studio Code.**
- Create a **new folder** (for example: MyFirstHTML).
- Inside the folder, create a new file and save it as: **index.html**

```
<!DOCTYPE html>
<html>
<head>
  <title>My First Web Page</title>
</head>
<body>
  <h1>Hello, world!</h1>
</body>
</html>
```

Output

Output: Displays 'Hello, Web Technology!' in browser.

Second HTML Program in VS Code

- Open **Visual Studio Code**.
- Create a **new folder** (for example: MySecondHTML).
- Inside the folder, create a new file and save it as:
  New.html

```
<!DOCTYPE html>
<html>
<head>
  <title>College Info</title>
</head>
<body>
  <h1> ABES Engineering College </h1>
  <h2>Ghaziabad </h2>
  <h3>ASH- Department</h3>
</body>
</html>
```

Output:

ASSIGNMENT -1

1. **Define Web Technologies. Explain different types of Web Technologies with suitable examples.**
2. **Difference between Front-End Developer and Back-End Developer.**
3. **Explain Client-Server Architecture with diagram**
4. **Create a simple HTML webpage in VS Code with the following details:**

linstructions:

1. **Create a new folder named Web-Assignment on your computer.**
2. **Inside the folder, create a file named index.html.**
3. **The webpage must include the following:**
    - **A title in the browser tab: *"My first HTML Page"*.**
    - **A main heading (H1) containing your Name, Father's Name, and Mother's Name.**
    - **A subheading (H2) mentioning your Branch and Section.**
    - **A paragraph (P) that contains your address in 3–4 lines.**
4. **Save the file and run it in a web browser to check the output.**

Solution:

1.Define Web Technologies. Explain different types of Web Technologies with suitable examples.

Web Technology is the technology used to make and run websites on the Internet.

Web Technology refers to the collection of tools, languages, and protocols that allow computers and devices to communicate through the World Wide Web (WWW)."

These technologies allow users to browse websites, interact with online content, and share information over the internet.

Types of Web Technologies

Frontend Technologies

Frontend technologies are used to build the user-facing part of a website or web application. They include:

- **HTML, CSS, and JavaScript:** The trio of core languages for designing and developing user interfaces.
- **Frontend Frameworks:** Tools like **React, Vue.js**, and **Angular** help developers quickly build complex, interactive user interfaces and manage state.

Backend Technologies

Backend technologies are used to build and manage the server-side of web applications. These include:

- **Server-side Languages:** Python, Ruby, PHP, and **Node.js** are popular backend programming languages used to handle business logic, manage databases, and process user requests.
- **Database Management Systems: MySQL, MongoDB, and PostgreSQL** are commonly used to store and retrieve data on the server-side.
- **Web Servers: Apache, Nginx,** and cloud services like **AWS** and **Google Cloud** host and manage websites.

Full-Stack Technologies

Full-stack development involves both frontend and backend development. Frameworks like **Django** (Python), **Ruby on Rails** (Ruby), and **Node.js** (JavaScript) allow developers to handle both client-side and server-side development.

Web APIs

Web APIs enable communication between different software components. Some popular API types include:

- **RESTful APIs:** Representational State Transfer (REST) APIs are widely used for client-server communication.
- **GraphQL:** A modern alternative to REST APIs, allowing clients to request only the data they need.
- **WebSockets:** WebSockets allow for real-time, two-way communication between the client and server.

2.Difference between Front-End Developer and Back-End Developer.

Front-end developer	**Back-end developer**
It is Part of website User Can see Code and Interact it.	It is brain of website that Code never visible to End User
Run on the client side browser	Run on the server side browser
HTML, CSS, JavaScript (and frameworks like Bootstrap, React).	PHP, Python, Java, Node.js and Databases like MySQL, MongoDB.
Designs the layout, look and feel of the website.	Handles the logic, database, and server operations.
Limited Security	Advanced Security

3.Explain Client-Server Architecture with diagram

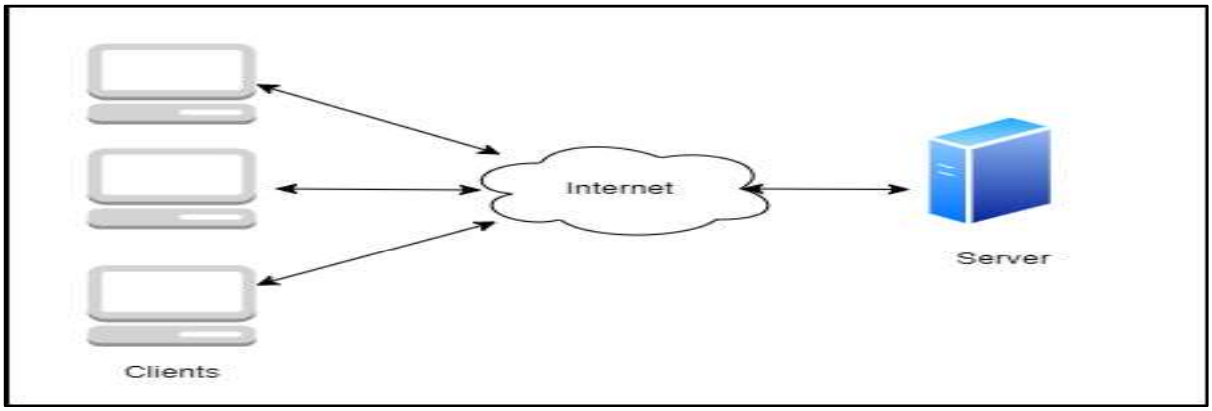
Client-Server Architecture is a model where tasks are divided between two entities:

(i)Client = Request

(ii)Server = Response

A computing model where the client requests services and the server processes and responds.

Client-Server Model is a distributed architecture where clients request services and servers provide them



Client: An application that requests services from the server, such as data retrieval, storage, calculations, or other functions.

Server: An application that processes client requests, sends responses, or performs specified actions. The server and client may reside on the same machine or different devices across the network. Effective Communication occurs via predefined protocols like HTTP, FTP, or SMTP.

4.Create a simple HTML webpage in VS Code with the following details

Instructions:

a. Create a new folder named **Web_Assignment** on your computer.

b. Inside the folder, create a file named **index.html**.

c. The webpage must include the following:

i. A **title** in the browser tab: *"My first HTML Page"*.

ii. A **main heading (H1)** containing your **Name, Father's Name, and Mother's Name**.

iii. A **subheading (H2)** mentioning your **Branch and Section**

iv. A **paragraph (P)** that contains your **address** in 3–4 lines.

d. Save the file and **run it in a web browser** to check the output

```
<!DOCTYPE html>
<html lang="en">
<head>
<title>My first HTML Page</title>
</head>
<h1>
Name: ABES Engineering College<br >
Founded: 1 July 2000<br >
 Address: Ghaziabad, Uttar Pradesh 201009
 </h1>
<h2>
Branches: B.Tech , M.Tech ,
MCA , Business School
</h2>
<p>
Vision :
To take ABES Engineering College to such a level that, it is at par with the leading institutions of the world in providing leadership to the international education system and be amongst the top rated institutions of the world by providing a transformative education to create leaders and innovators embedded in traditional Indian values.
</p>
<p>
Mission :
<ul>
  <li>To create an ambiance for healthy teaching-learning process.</li>
  <li>To nurture the students and infuse in them
    <ul>
      <li>A passion to excel professionally.</li>
      <li>A spirit to be of utmost use to the industry, corporate sector and the society at large.</li>
      <li>An intense desire to take challenging responsibilities and leadership roles.</li>
      <li>A craving to be wholesome good human beings.</li>
    </ul>
  </li>
  <li>To develop an environment for creating new knowledge through research and by thriving to explore innovative ideas.</li>
</ul>
</p>
```

Output:

Name: ABES Engineering College
Founded: 1 July 2000
Address: Ghaziabad, Uttar Pradesh 201009

Branches: B.Tech , M.Tech , MCA , Business School

Vision To take ABES Engineering College to such a level that, it is at par with the leading institutions of the world in providing leadership to the international education system and be amongst the top rated institutions of the world by providing a transformative education to create leaders and innovators embedded in traditional Indian values.

Mission

- To create an ambiance for healthy teaching-learning process.
- To nurture the students and infuse in them
  - A passion to excel professionally.
  - A spirit to be of utmost use to the industry, corporate sector and the society at large.
  - An intense desire to take challenging responsibilities and leadership roles.
  - A craving to be wholesome good human beings.
- To develop an environment for creating new knowledge through research and by thriving to explore innovative ideas.

1.2.2 Introduction of Git & GitHub

What is a Version Control System (VCS)?

Imagine you are writing your **Project Report**.

- Day 1: You save it as report.docx.
- Day 2: You add more content and save as report-final.docx.
- Day 3: You make changes and save as report-final2.docx.

Soon, your folder looks like this:

report.docx
report-final.docx
report-final2.docx
report-final2-new.docx

Confusing, right? Which one is the latest? Which one has the correct data?
What if you want to **go back** to Day 1's version?

Version Control System (like Git) solves this problem!

- Git keeps track of every change (like a time machine).
- You can go back to *any previous version* easily.
- You don't need to create multiple final-final.docx files anymore!

Definition:

A **Version Control System (VCS)** is a tool that **records changes to files over time** so that you can recall specific versions later.

◈ What is Git?

Think of **Git as a CCTV Camera for your code.**

- It watches every change (who did it, when, what was changed).
- It stores snapshots (commits) of your project.
- You can rewind and see the history.
- Multiple developers can work together and Git will track "who changed what".

◈ In technical words:
Git is a **distributed version control system**. It runs on your **local machine** and manages code versions, branches, and merging.

- **Git** = Version Control System (VCS).
- Tracks project changes → allows rollback.
- Enables collaboration among developers.

◈ What is GitHub?

If **Git is your CCTV camera at home**, then **GitHub is your cloud storage (like Google Drive) where you upload those CCTV recordings**.

- GitHub is an **online platform** where you can store your Git repositories.
- A **cloud-based platform** hosting Git repositories.
- It helps you **share code with teammates**, manage collaboration, and contribute to open-source.
- Provides features like:
    - **Pull requests** (suggesting changes)
    - **Issues** (tracking bugs/tasks)
    - **Forks** (copy of someone else's repo)
    - **Branches** (parallel development)

- Think of Git as **your notebook** and GitHub as **Google Drive for your notebook**.
- **Git vs GitHub – Relation & Difference**

Git	GitHub
Tool to track changes (VCS).	Platform to host Git repositories.
Works on your **local system**.	Works **online (cloud)**.
Can be used alone (without GitHub).	Cannot work without Git.
Example: Track project locally.	Example: Share project with your team worldwide.

- Git = **Notebook** (where you write your code & track changes).
- GitHub = **Library** (where you store your notebook so that others can read, borrow, or contribute).

Why Use Both Git & GitHub Together?

- **Git alone** → You can track changes locally, but **can't collaborate easily**.
- **GitHub alone** → It's just a website, can't track versions without Git.
- **Together** → You get **version control + cloud hosting + collaboration tools**.

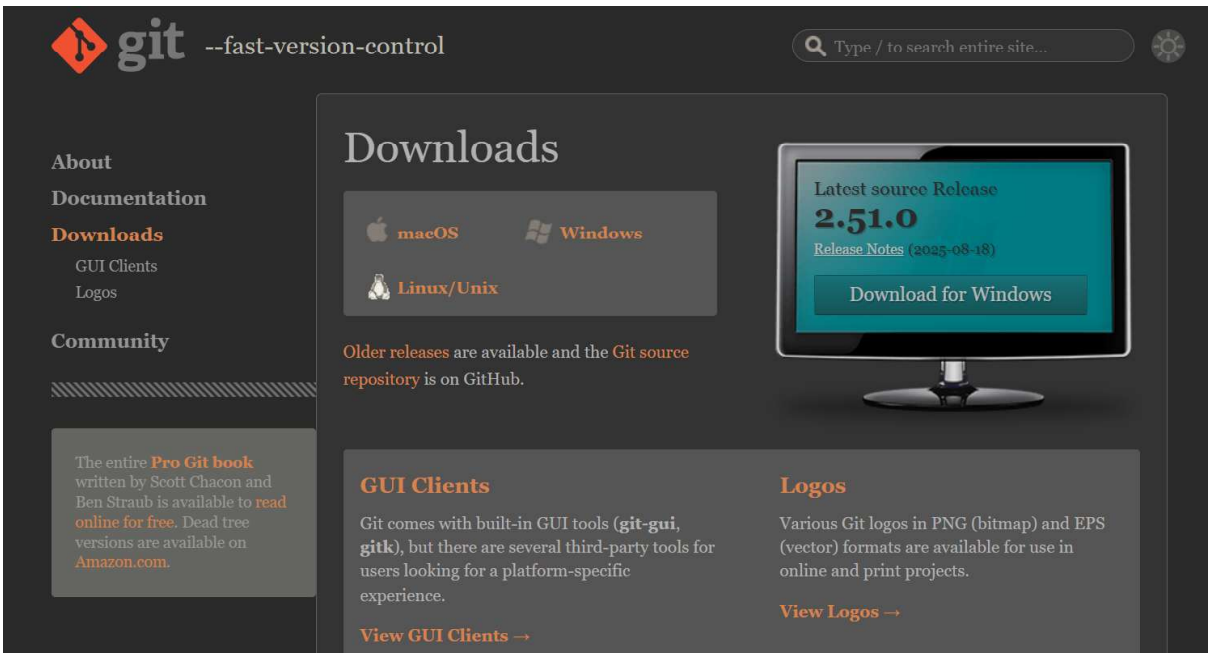
Example:
Imagine you and your friends are building a **mini web project**:

- On your laptop, Git tracks your changes (like saving checkpoints).
- You push it to GitHub → your friends can see and pull updates.
- They add new features → push again → you pull back.
- GitHub acts as a **shared code hub**, Git acts as the **history manager**.

1.2.3 Installation & Configuration of Git Account

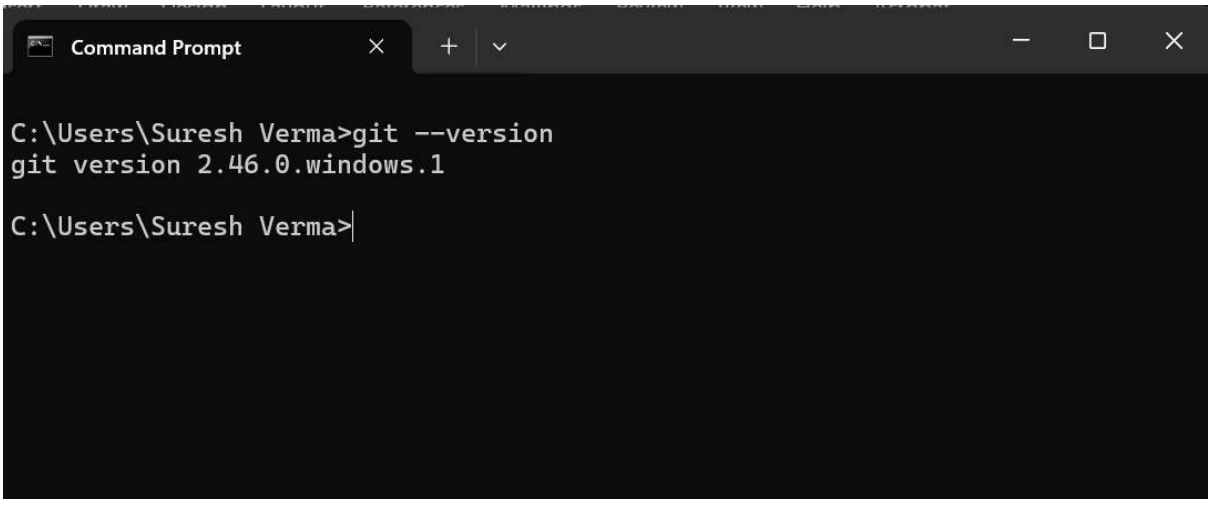
Steps:

1. Download: https://git-scm.com/downloads



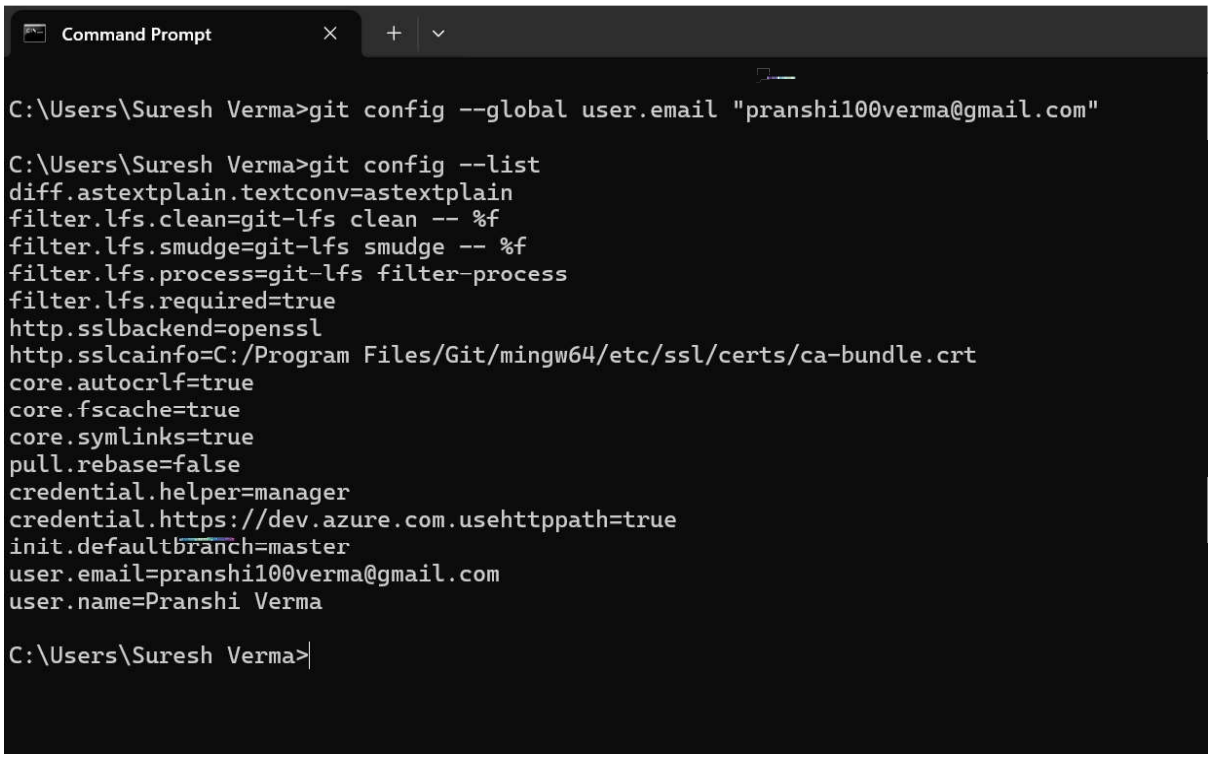
2. Install with default settings.
3. Open terminal (CMD/VS Code terminal) → check version:

   git –version



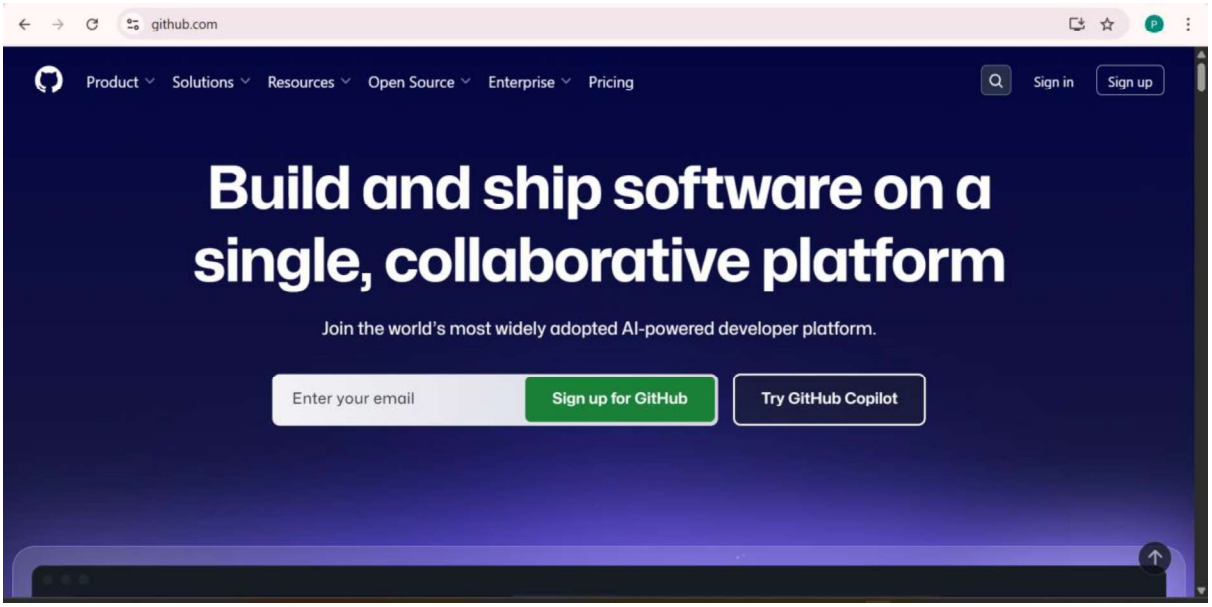
Configure Git (first-time only):

git config --global user.name "Your Name"
git config --global user.email "your_email@example.com"
git config --list # check configuration

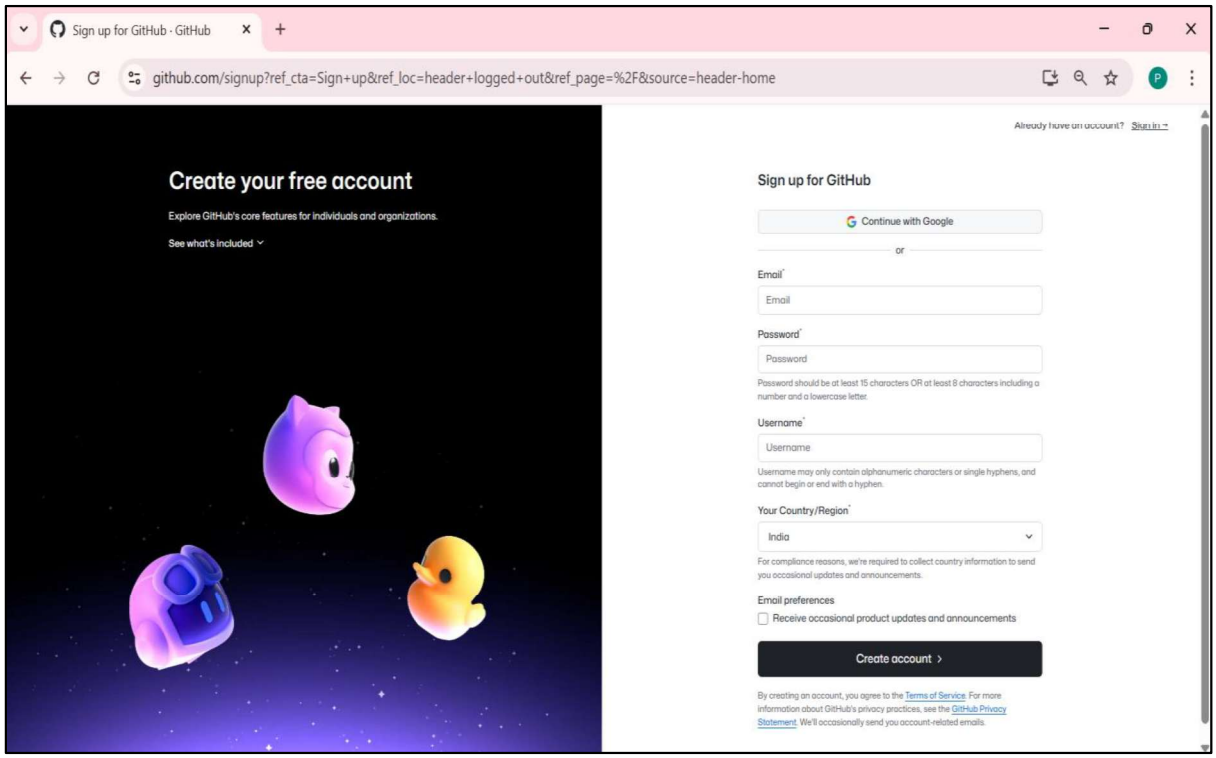


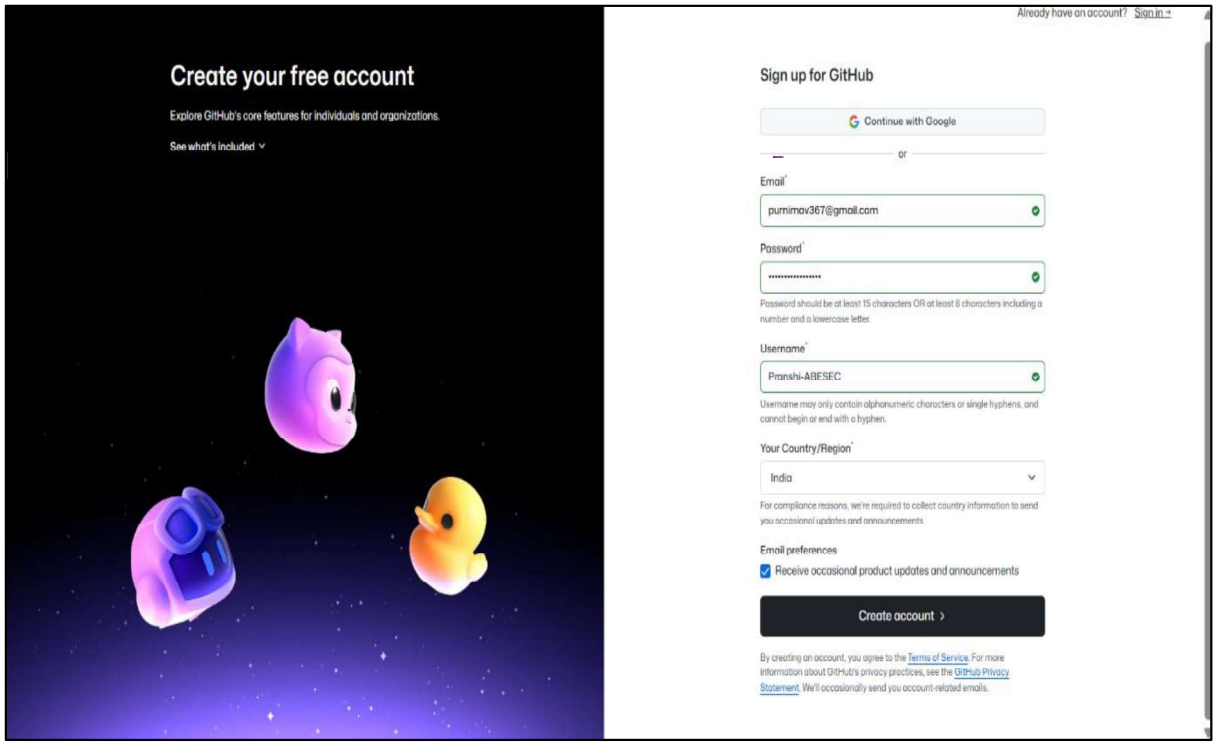
4. Creating a GitHub Account

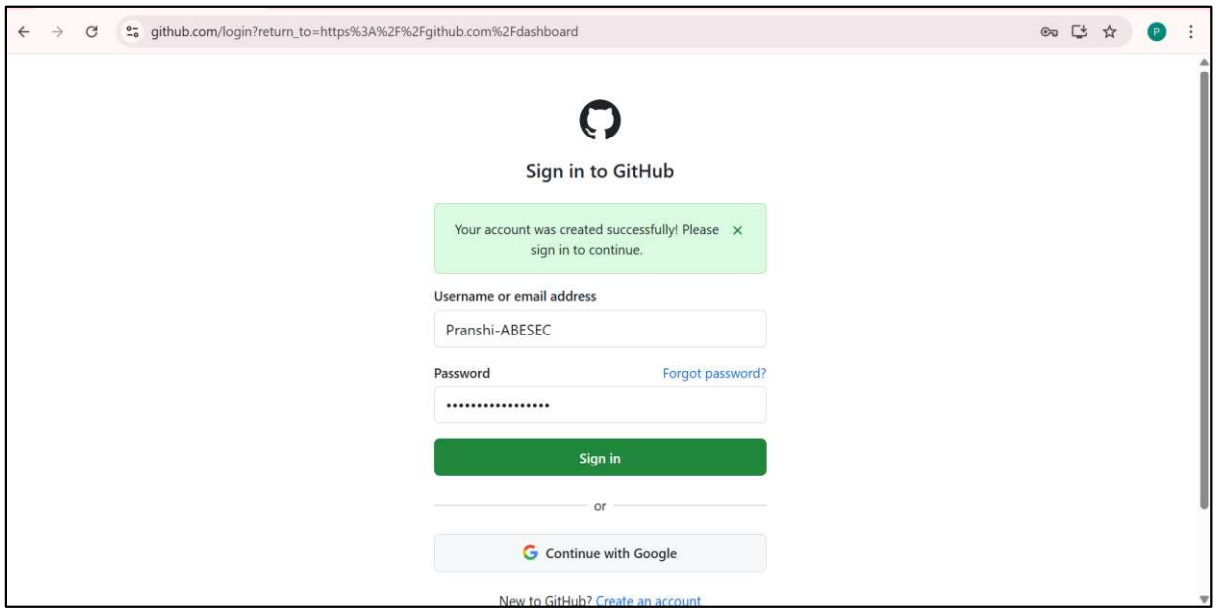
1. Go to https://github.com/



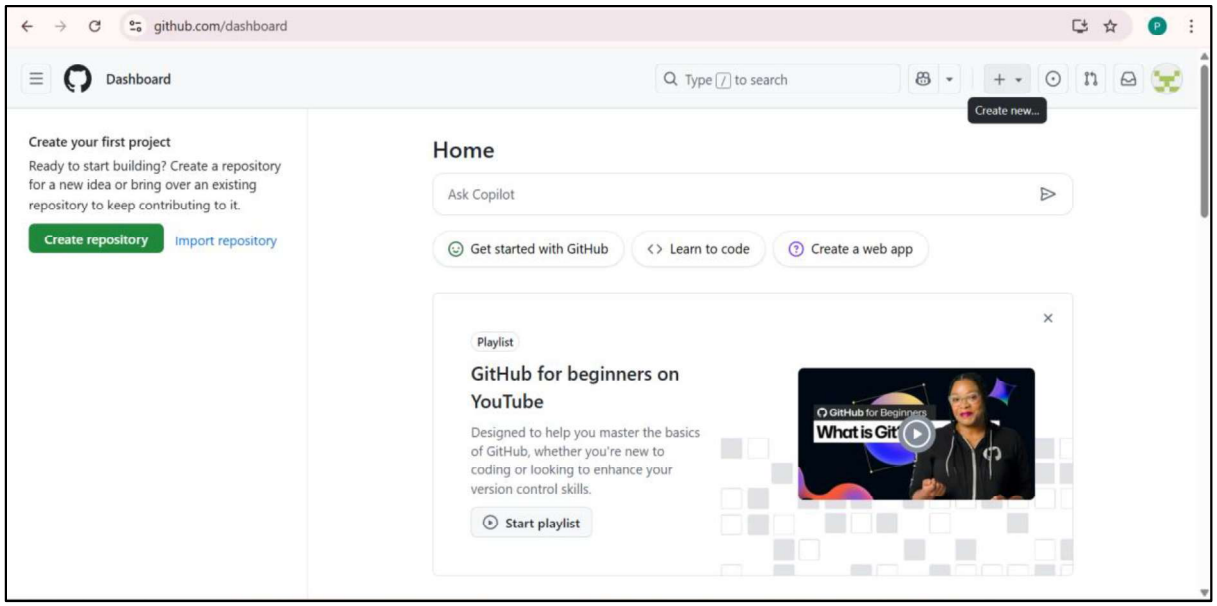
2. Sign up with email → verify → choose username.

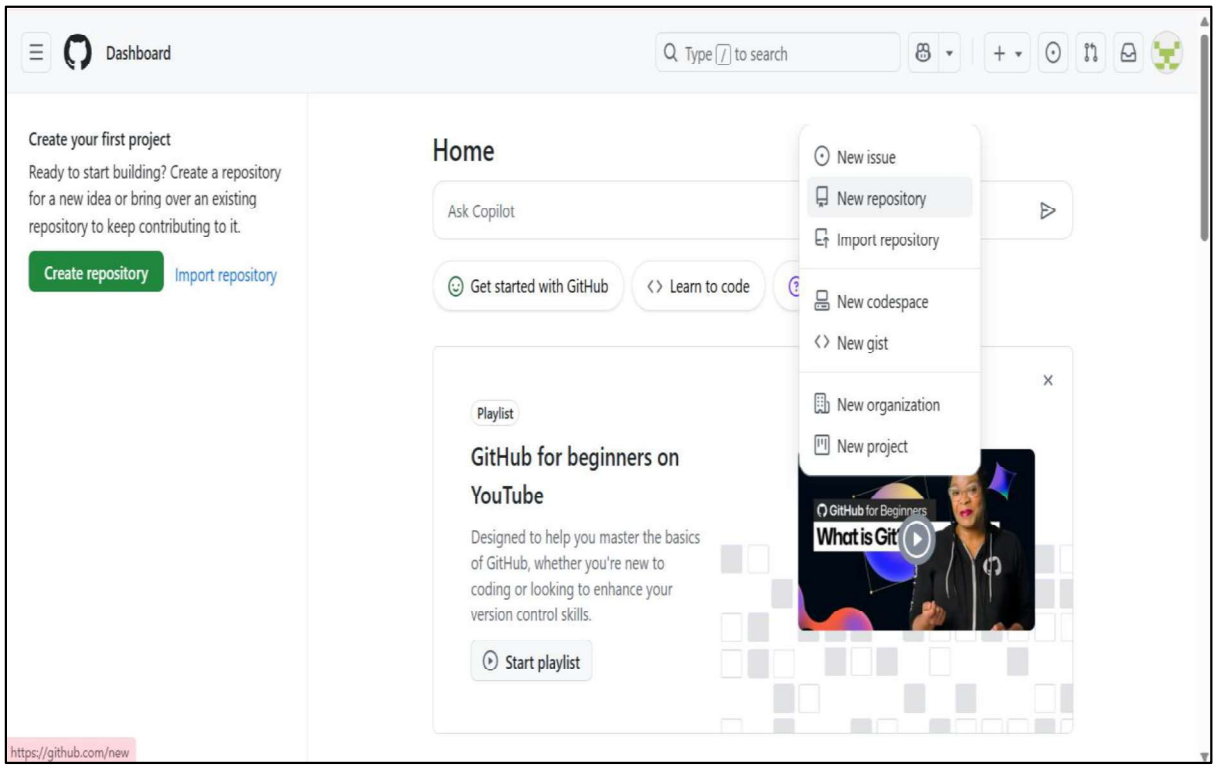




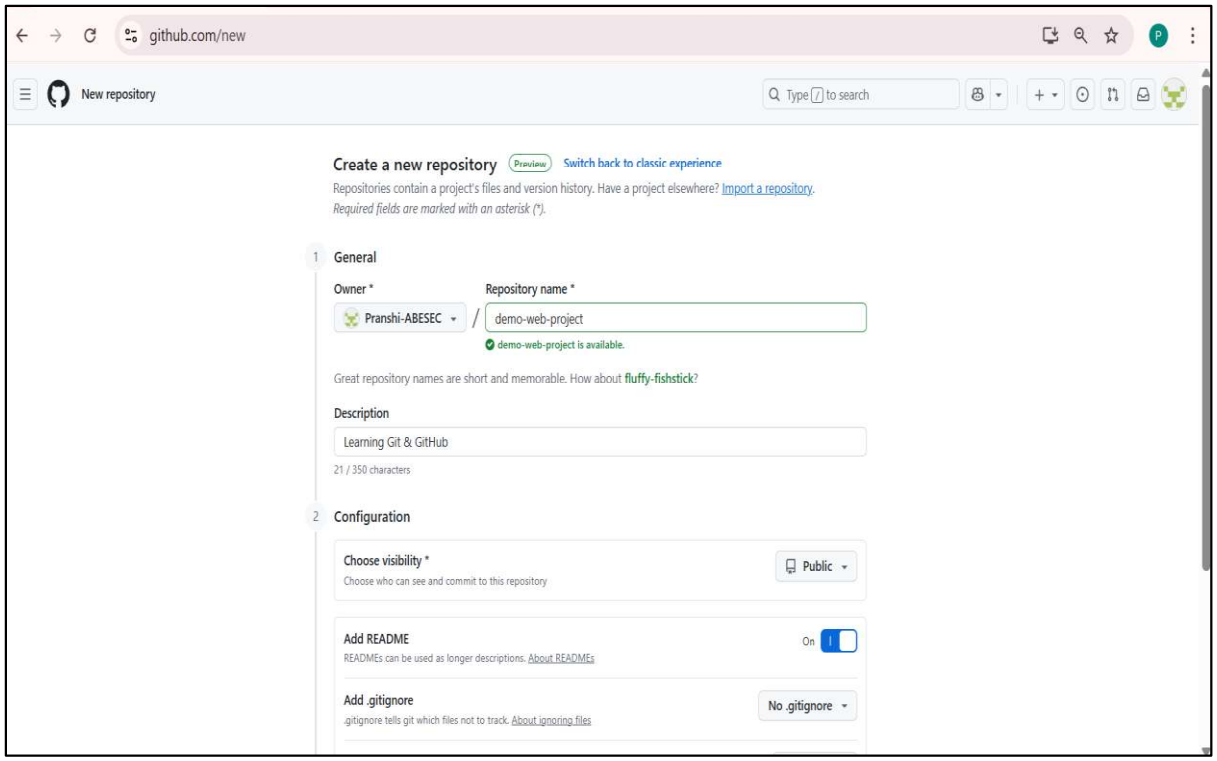


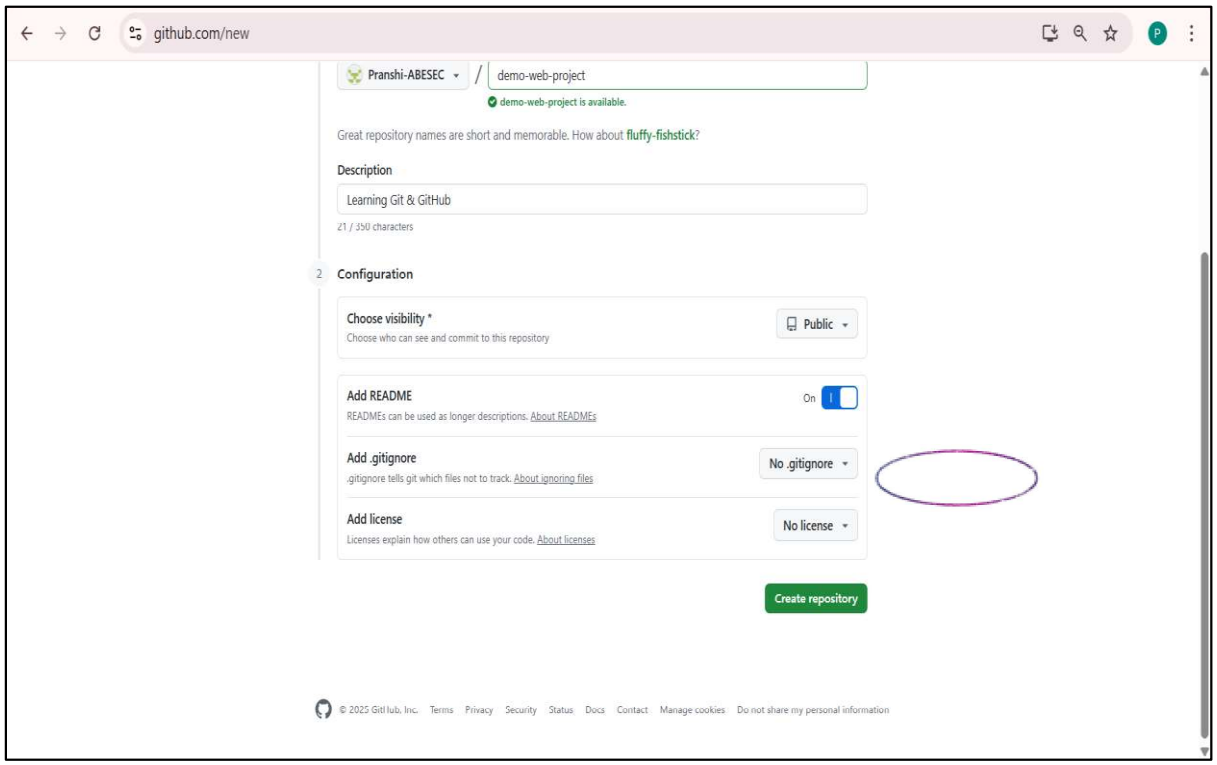
3. Create a new repository:





- Repository Name: demo-web-project
- Description: "Learning Git & GitHub"
- Public → Add README.md → Create Repository.





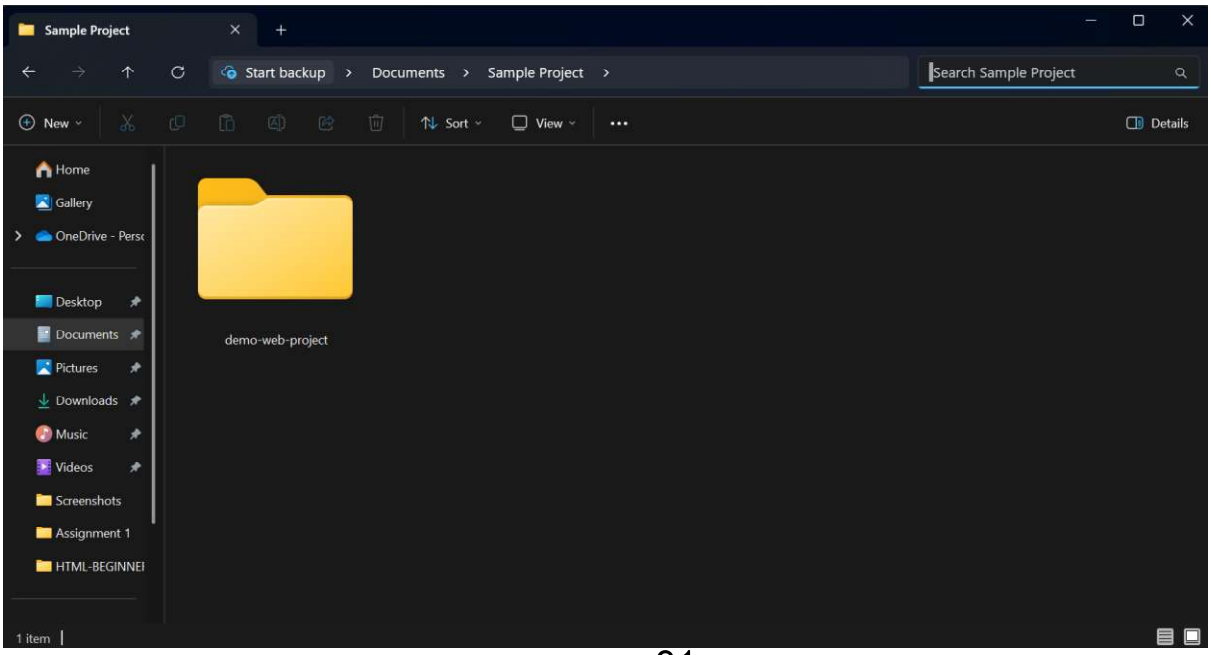
Creating a Local Repository

A **local repository** is a project folder on your computer that Git will track. This is the **first step** before pushing anything to GitHub.

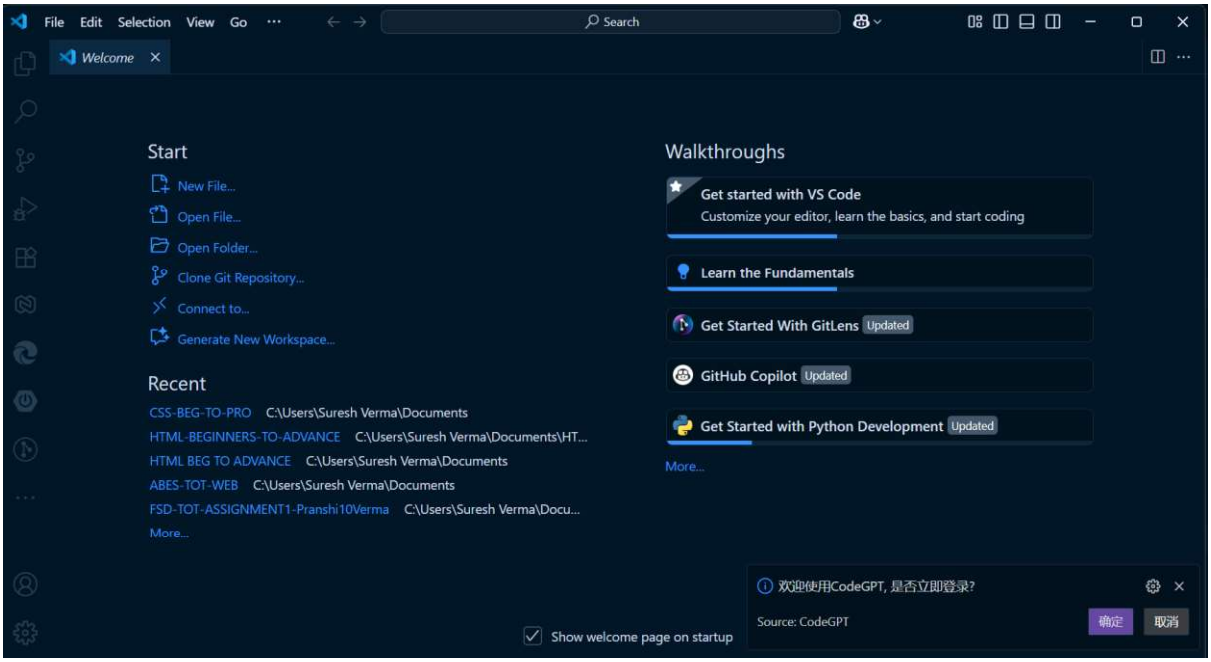
You can create it in two ways:

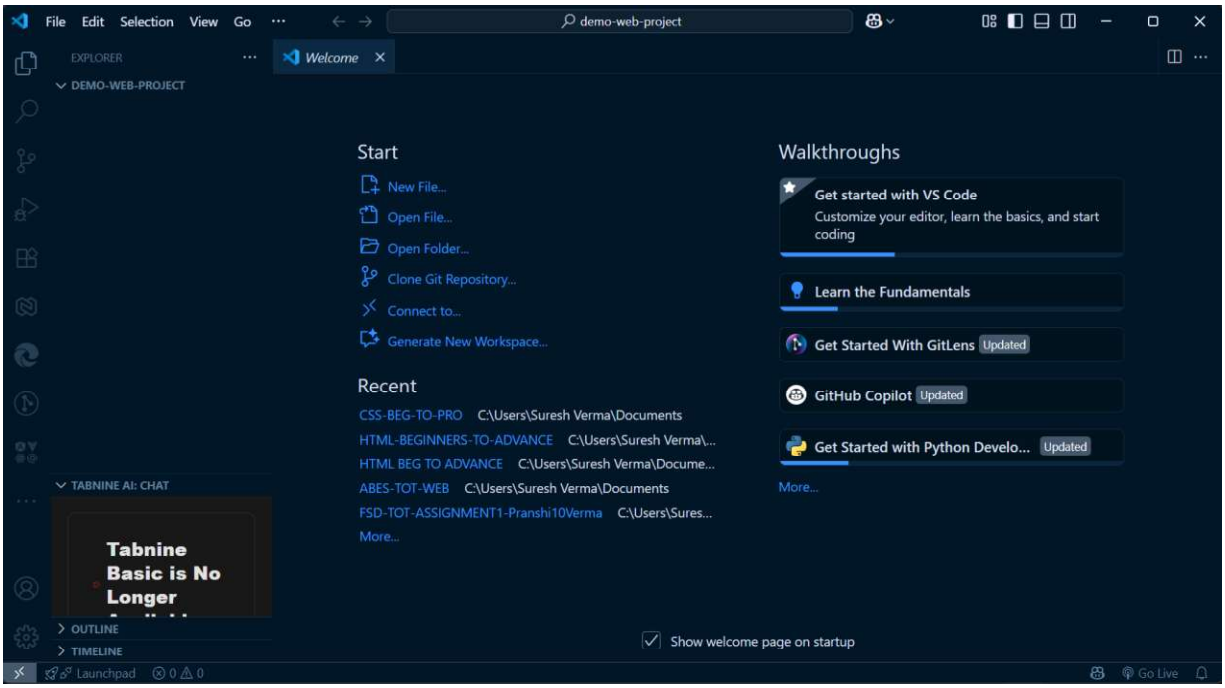
Method A – Using VS Code (Beginner-Friendly UI)

1. **Create a project folder**
    - Example: demo-web-project



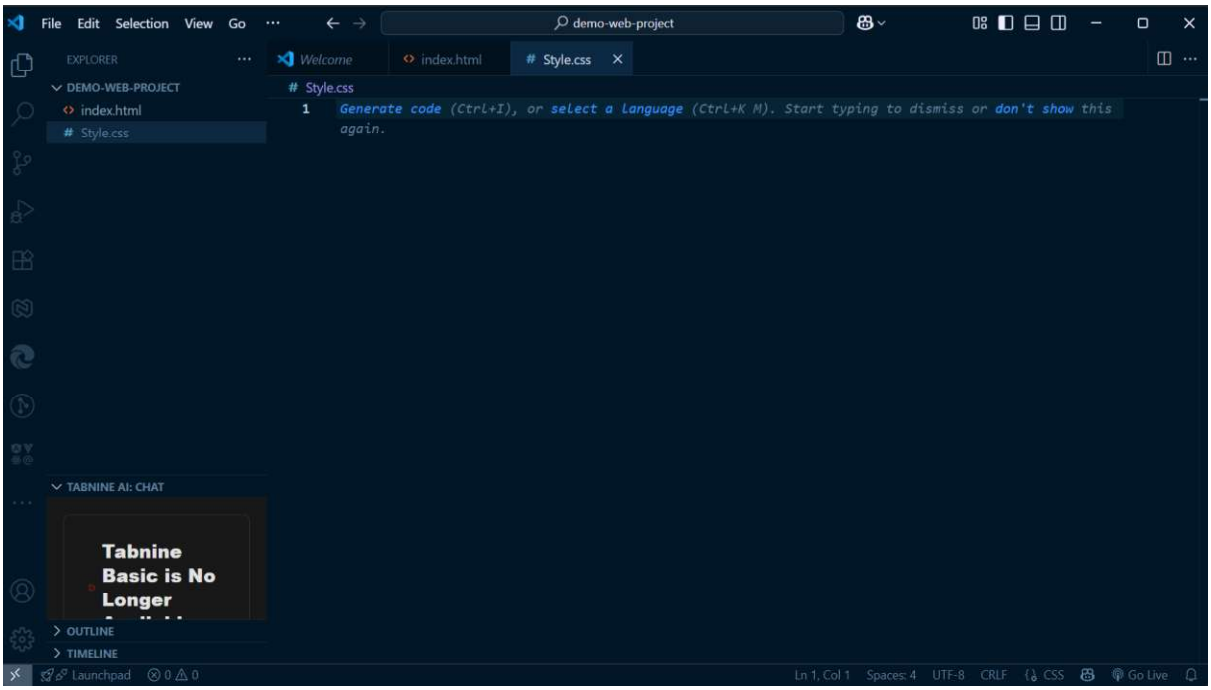
- Open it in VS Code (**File → Open Folder**).





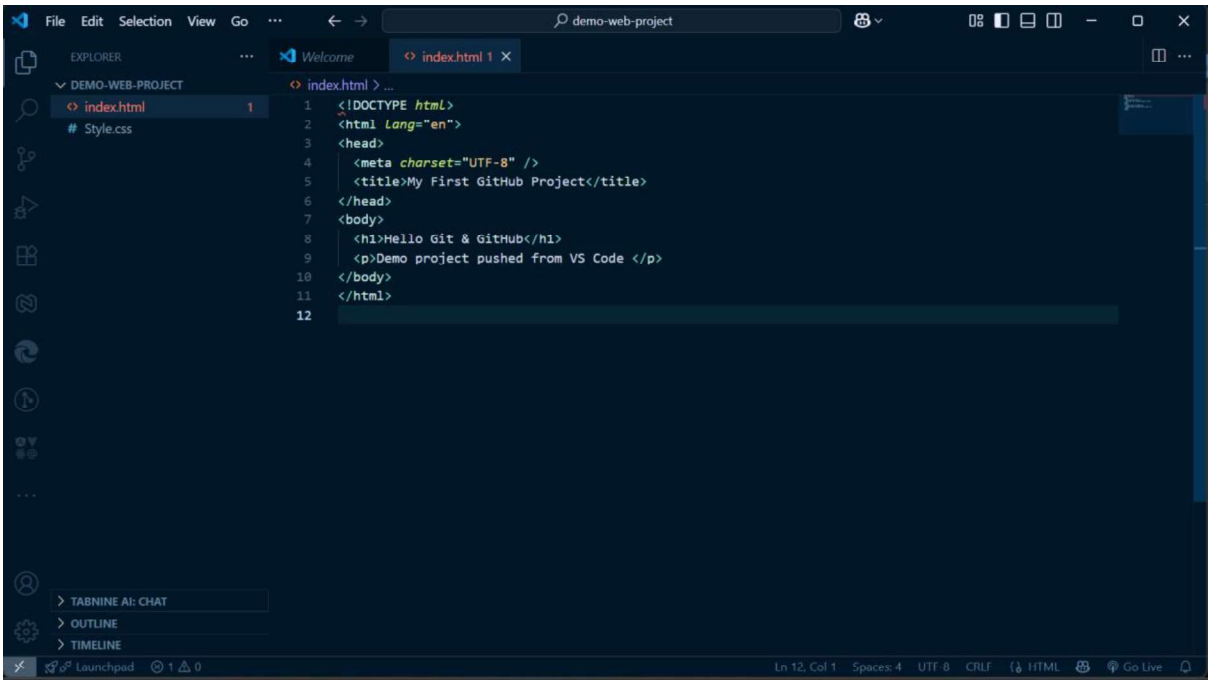
2. **Create project files**
   Example:

   - index.html
   - style.css



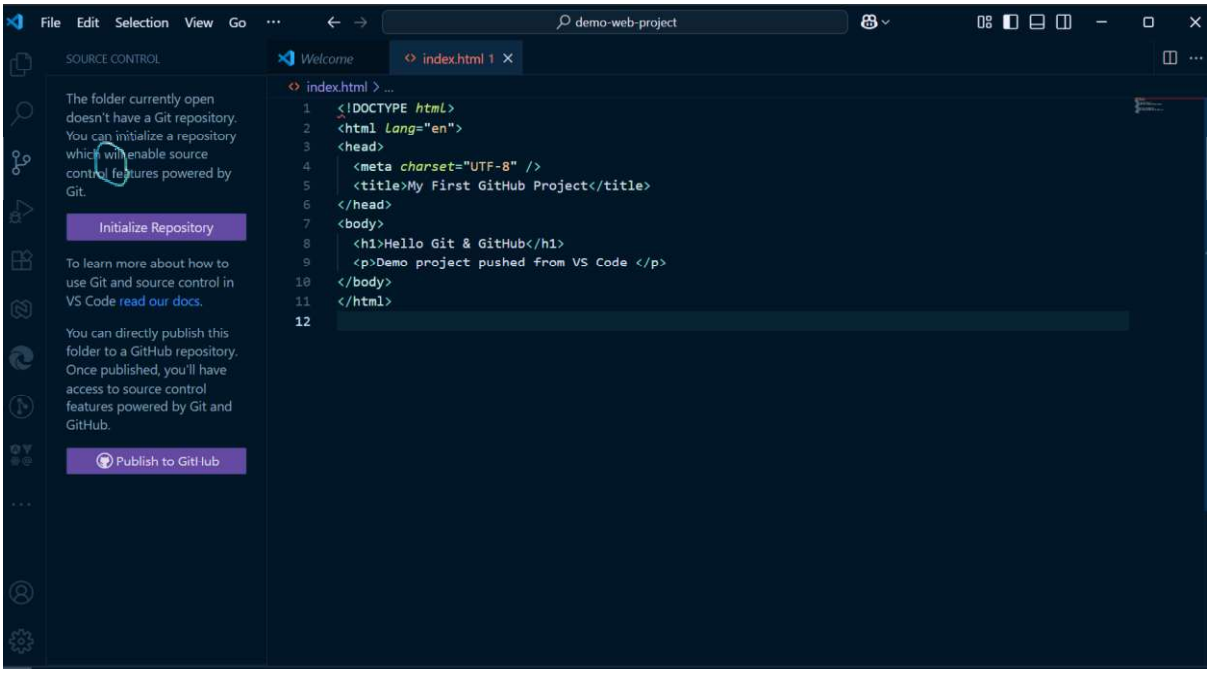
index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8" />
 <title>My First GitHub Project</title>
</head>
<body>
 <h1>Hello Git & GitHub</h1>
 <p>Demo project pushed from VS Code </p>
</body>
</html>
```



3. Initialize Git (create local repository)

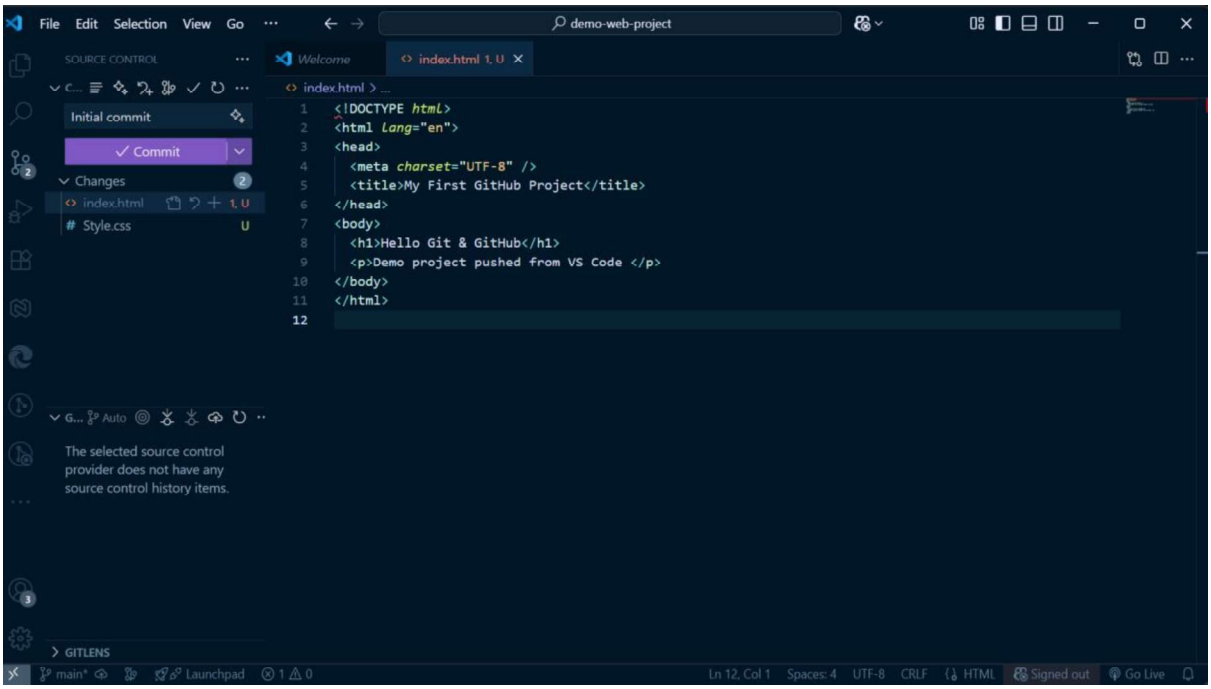
- Go to the **Source Control panel** in VS Code (branch icon on sidebar).



- Click **Initialize Repository**.
- This creates a **local Git repository** inside the folder.

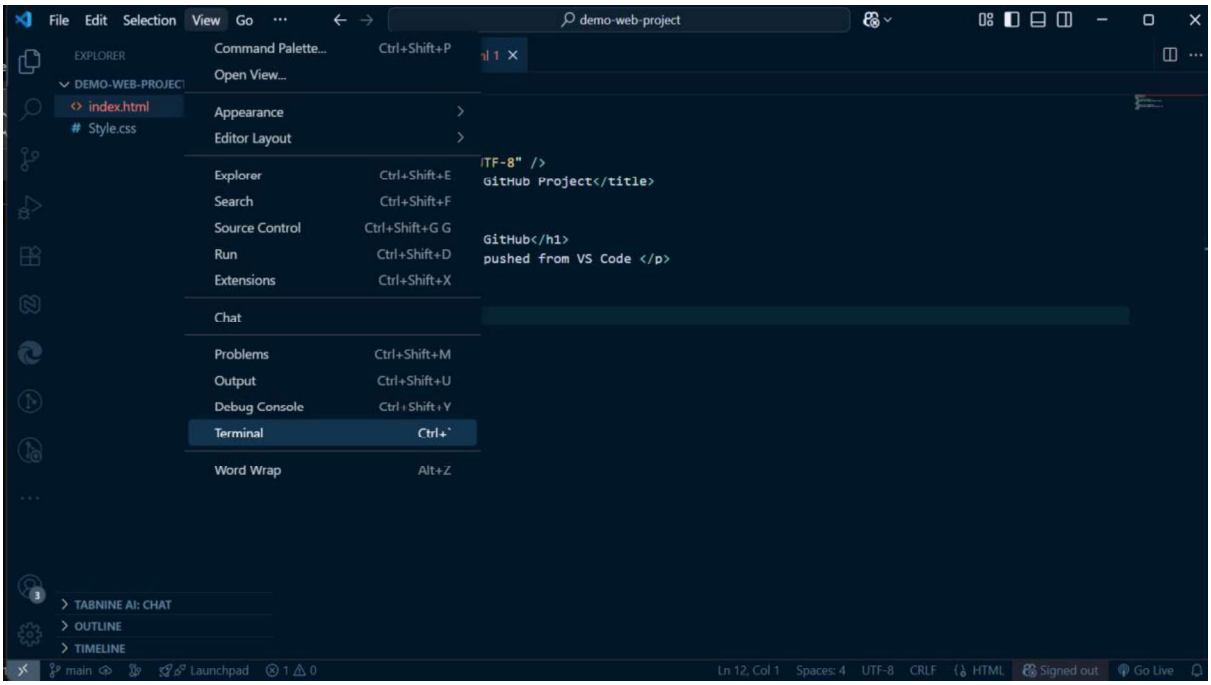
4. Make your first commit

- Stage changes (+ button).
- Enter a commit message: "Initial commit".
- Click **Commit**.

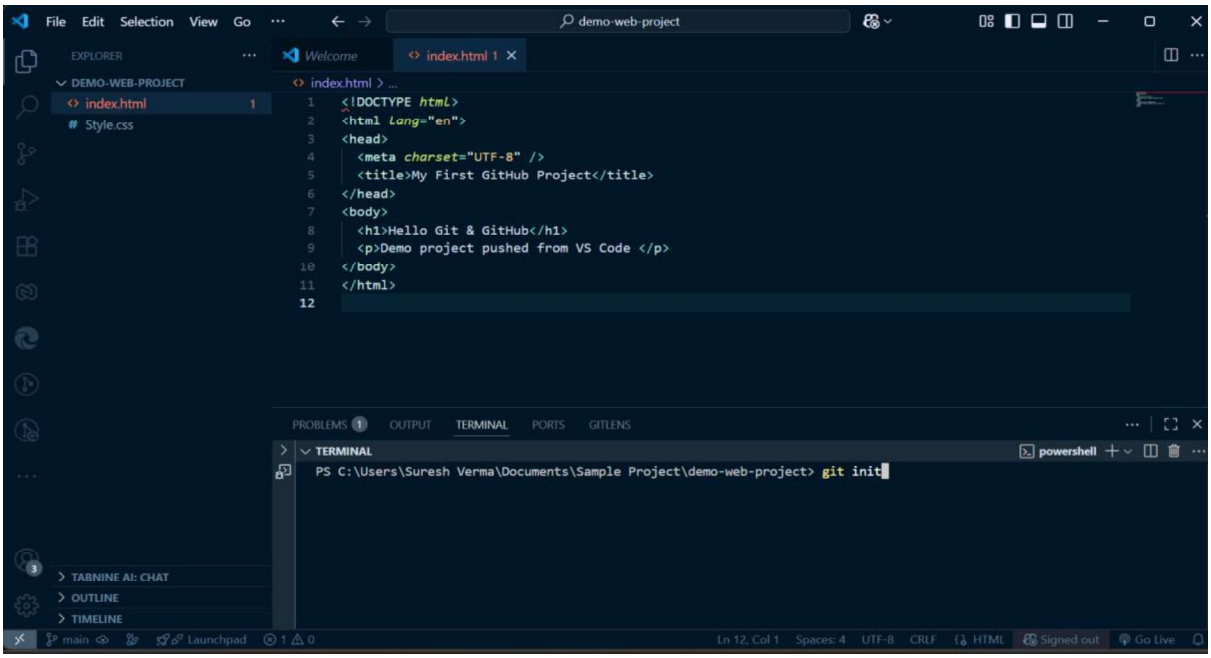


Method B – Using Terminal

1. Open **VS Code Terminal** (Ctrl + ~) or CMD.
2. Navigate to your project folder: *cd path/to/demo-web-project*



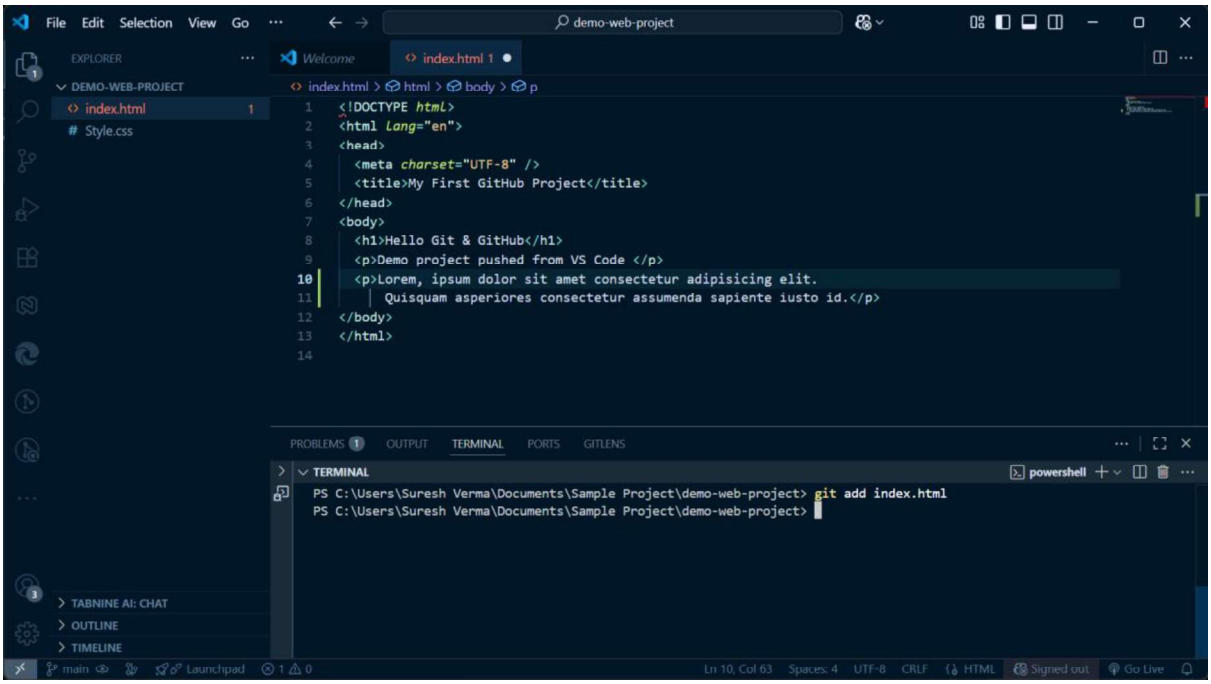
Initialize the repository: git init

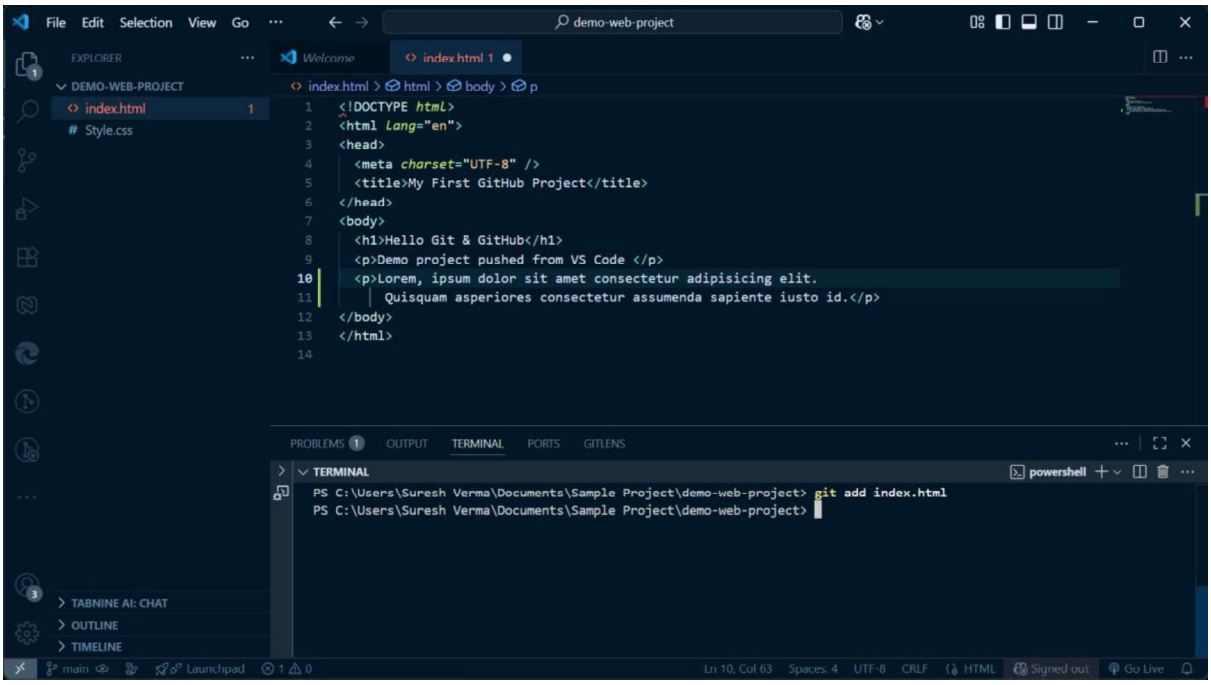


- This command creates a **hidden `.git` folder**, which means Git is now tracking your project.
- Add and commit files:

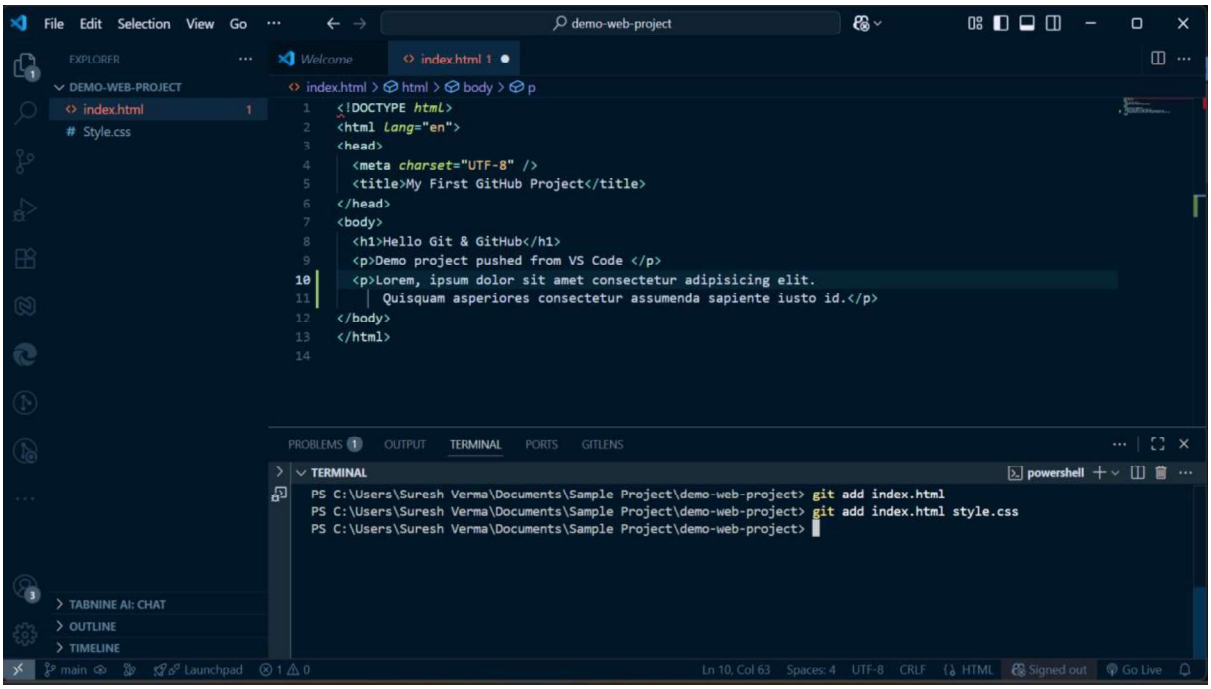
Add a Single File
If you only want to add **one file** to the staging area (for example, index.html): *git add index.html*

Add Multiple Specific Files
You can also add more than one file at a time by naming them: *Git add index.html*





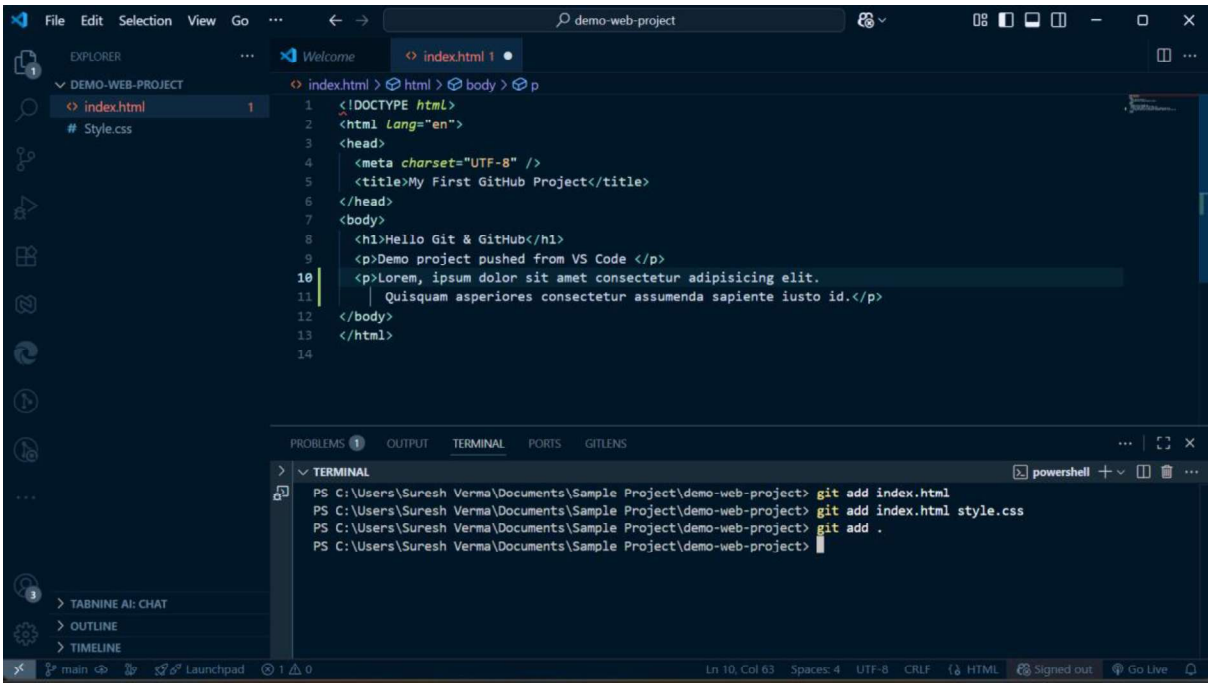
git add index.html style.css



Add All Files (Recommended when you want everything)

If you want to add **all files in the project folder**:

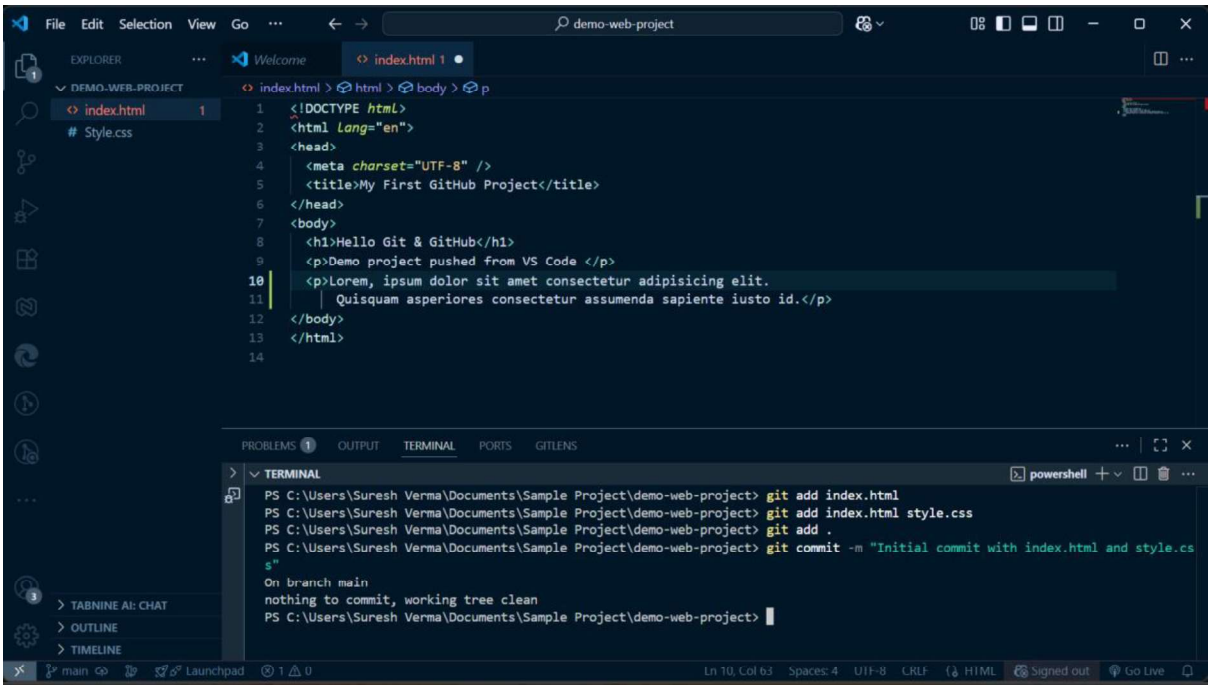
git add .



Commit the Changes

After adding files, you need to **commit** them with a message:

git commit -m "Initial commit with index.html and style.css"



A **commit** is like saving a snapshot of your project at that point in time.

Now we will link our local repository with the GitHub repository (remote).

Now your project is on GitHub!

1.3 Introduction to HTML & HTML Features, HTML Page Structure, Elements in HTML (Semantic and Non Semantic)

1.3.0 What is HTML?

- HTML stands for Hyper Text Markup Language
- HTML is the standard markup language for creating Web pages
- HTML describes the structure of a Web page
- HTML consists of a series of elements
- HTML elements tell the browser how to display the content
- HTML elements label pieces of content such as "this is a heading", "this is a paragraph", "this is a link", etc.

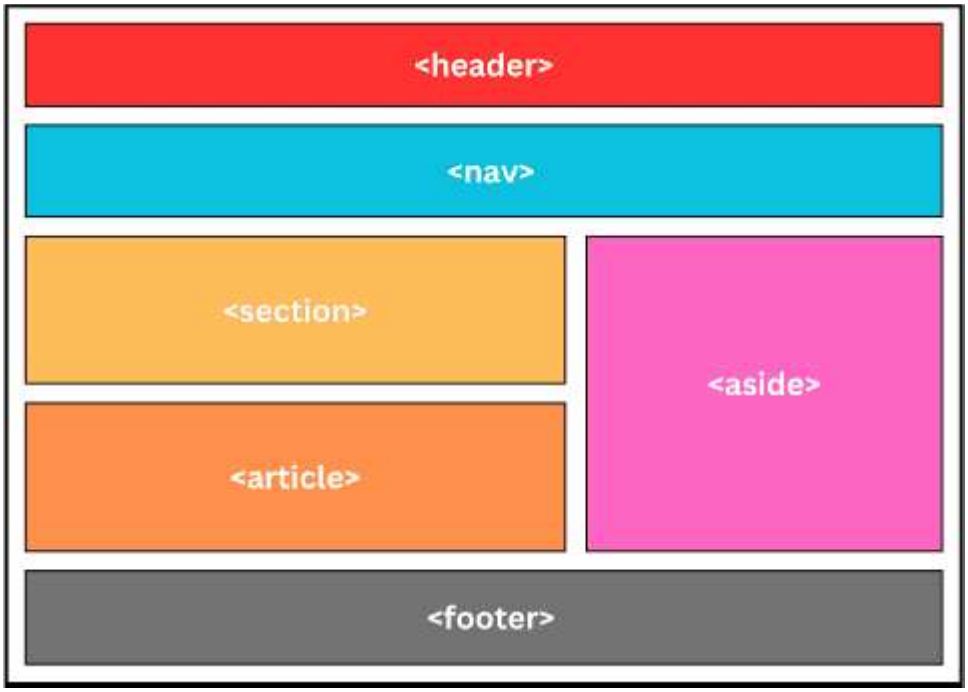
1.3.1 HTML 5 Features

HTML5 was introduced having some new features following are

- Error tolerance
- Save as html or .htm
- Pass data in "……." or '..... '
- Name Casing <HTML> or <html>
- Semantic Elements
- Audio and Video Support
- Canvas Elements
- Geolocation API
- Local Storage
- Responsive Images
- Web Workers
- Drag and Drop API
- Form Enhancements
- Web Sockets
- Micro Data

1.3.2 HTML Page Structure

An HTML page is structured using a hierarchical arrangement of elements, providing a logical organization for the content. The fundamental structure includes:



<!DOCTYPE html> Declaration:

This declaration, placed at the very beginning of an HTML document, informs the browser that the document is an HTML5 document. It is not an HTML tag but a directive.

<html> Element:

This is the root element of an HTML page, encapsulating all other HTML elements. It signifies the beginning and end of the HTML document.

<head> Element:

Located within the <html> element, the <head> contains metadata about the HTML document. This information is not directly displayed on the web page but is crucial for browser rendering, search engine optimization, and other functionalities. Common elements within the <head> include:

<title>: Defines the title of the document, which appears in the browser tab or window title bar.

<meta>: Provides various metadata, such as character set (charset), viewport settings, and descriptions for search engines.

<link>: Links external resources like stylesheets (.css files) or favicons.

<style>: Embeds internal CSS styles directly within the HTML document.

<script>: Embeds or links external JavaScript code for interactive functionality.

<body> Element:

Also located within the <html> element, the <body> contains all the visible content of the web page. This is where the actual content that users interact with is placed. Examples of elements found within the <body> include:

Headings (<h1> to <h6>): Define titles and subtitles for sections of content.

Paragraphs (<p>): Enclose blocks of text.

Links (<a>): Create hyperlinks to other web pages or resources.

Images (<img>): Embed images into the page.

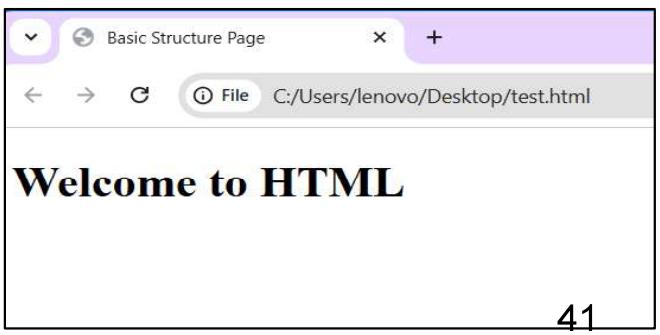
Lists (<ul>, <ol>, <li>): Create unordered or ordered lists of items.

Divisions ( ): Generic container for grouping other elements for styling or scripting purposes. Provide more meaningful structure to the content, improving accessibility and SEO.

This nested structure, starting from the <!DOCTYPE html> and encompassing the <html>, <head>, and
<body> elements, forms the fundamental framework of every HTML page.

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Basic Structure Page</title>
</head>
<body>
   <h1>Welcome to HTML</h1>
</body>
</html>
```

Output:



1.3.3 Elements in HTML (Semantic and Non Semantic)

A tag is a keyword that defines how a web browser should format and display a web page. HTML tags are the basic building blocks of any website. In html, codes(normal text) enclosed in brackets written in usually paired.

See below example:

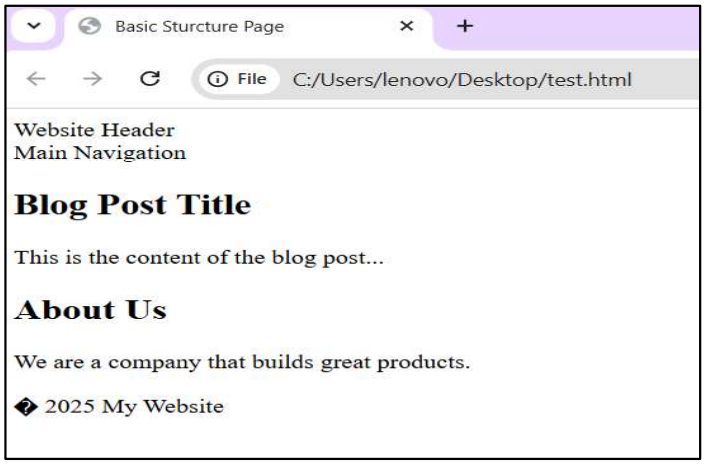
<TITLE>My Web Page</TITLE>

Note: - HTML is not case sensitive. <TITLE> = <title> =

<TItLE> In HTML,

```
<!DOCTYPE html>
<html lang="en">
 <head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Basic Sturcture Page</title>
 </head>
 <body>
  <header>Website Header</header>
  <nav>Main Navigation</nav>
  <main>
   <article>
    <h2>Blog Post Title</h2>
    <p>This is the content of the blog post...</p>
   </article>
   <section>
    <h2>About Us</h2>
    <p>We are a company that builds great products.</p>
   </section>
  </main>
  <footer>© 2025 My Website</footer>
 </body>
```

Output:



Semantic tags define the meaning of the content they contain, while non-semantic tags only old content and do not indicate its role on the page.

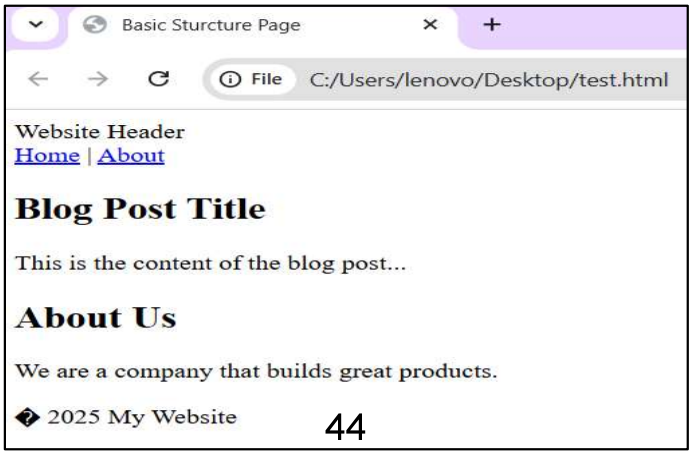
Semantic tags are both human- and machine-readable, and they can help users and machines understand the context and importance of the content.

Non-semantic tags, on the other hand, are used for styling purposes or as containers for other elements.

For Example: the tag does not suggest the content it will carry, however using the <p> tag suggests it can be used to hold paragraph information.

```
<!DOCTYPE html>
<html lang="en">
 <head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Basic Sturcture Page</title>
 </head>
 <body>
  <div id="header">Website Header</div>
  <div id="menu"><a href="#">Home</a> | <a href="#">About</a></div>
  <div id="content">
   <div class="post">
    <h2>Blog Post Title</h2>
    <p>This is the content of the blog post...</p>
   </div>
   <div class="info">
    <h2>About Us</h2>
    <p>We are a company that builds great products.</p>
   </div>
  </div>
  <div id="footer">© 2025 My Website</div>
 </body>
```

Output:



There are many reasons why you should write Semantic tags instead of normal HTML tags, to leverage SEO, accessibility, and browser compatibility.

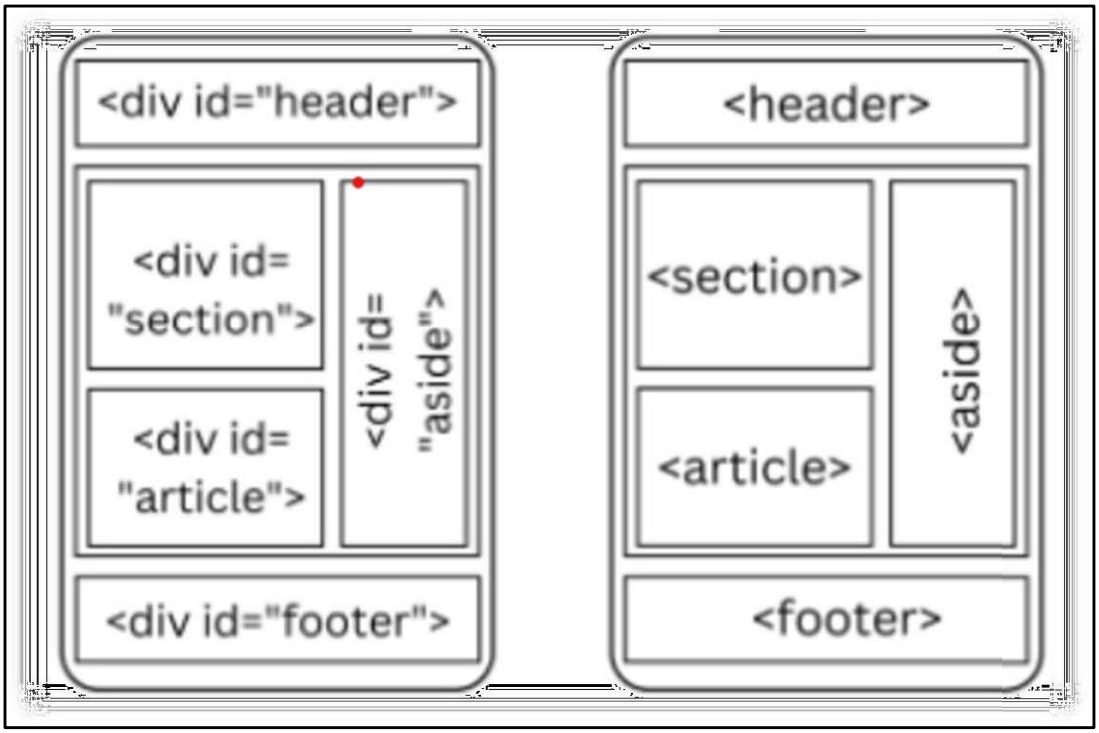


Fig: Non-Semantic and Semantic Tags

1.4 Types of Elements (Inline/Block), Text formatting- headings, paragraphs

1.4.0 Inline and Block Elements

HTML elements can be categorized in several ways, primarily based on their display behavior and their purpose.

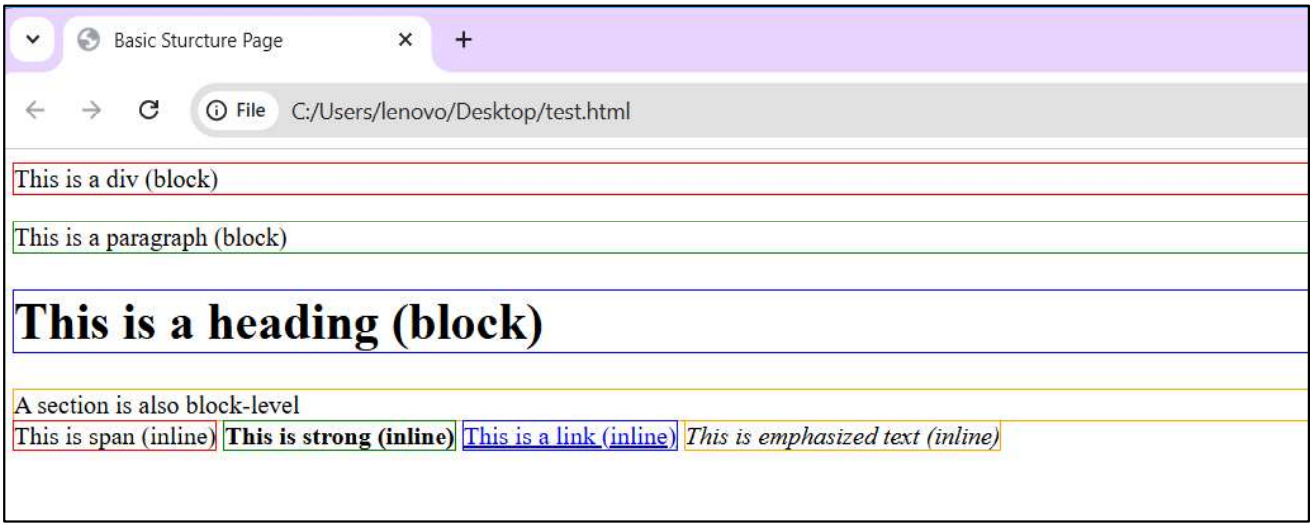
1. Block-level Elements:
These elements typically start on a new line and take up the full available width of their parent container.
They are used to structure the main sections of a web page.
Examples include:
<address>,<article>,<aside>,<blockquote>,<canvas>,<dd>, ,<dl>,<dt>,<fieldset>, <figcaption>,<figure>,<footer>,<form>,<h1>…<h6>,<header>,<hr>,<li>,<main>,<nav>,<noscript>, <ol>,<p>,<pre>,<section>,<table>,<tfoot>,<ul>,<video>.

2. Inline Elements:
These elements do not start on a new line and only take up as much width as their content requires. They are used to style or structure content within a block-level element.
Examples include:
<a>,<abbr>,<acronym>,<b>,<bdo>,<big>,
,<button>,<cite>,<code>,<dfn>,,<i>,<img>,<input>,

<kbd>,<label>,<map>,<object>,<output>,<q>,<samp>,<script>,<select>,<small>,,,<sub>,<sup>,<textarea>,<time>,<tt>,<var>.

```
<!DOCTYPE html>
<html lang="en">
 <head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Basic Sturcture Page</title>
 </head>
 <body>
  <!--block level elements-->
  <div style="border: 1px solid red">This is a div (block)</div>

  <p style="border: 1px solid green">This is a paragraph (block)</p>

  <h1 style="border: 1px solid blue">This is a heading (block)</h1>

  <section style="border: 1px solid
   orange"> A section is also block-level
  </section>
  <!--inline elements-->
  <span style="border: 1px solid red">This is span (inline)</span>
  <strong style="border: 1px solid green">This is strong (inline)</strong>
  <a href="#" style="border: 1px solid blue">This is a link (inline)</a>
  <em style="border: 1px solid orange">This is emphasized text (inline)</em>
 </body>
```

Output:



Tag	Description
<html> ... </html>	Declares the Web page to be written in HTML
<head> ... </head>	Delimits the page's head
<title> ... </title>	Defines the title (not displayed on the page)
<body> ... </body>	Delimits the page's body
<h *n*> ... </h*n*>	Delimits a level *n* heading
...	Set ... in boldface
...	Set ... in italics
...	Center ... on the page horizontally
...	Brackets an unordered (bulleted) list
...	Brackets a numbered list
...	Brackets an item in an ordered or numbered list
 	Forces a line break here
<p>	Starts a paragraph
<hr>	Inserts a horizontal rule
<img src="...">	Displays an image here
 ... 	Defines a hyperlink

1.4.1 Text formatting- headings, paragraphs

HTML Headings

- Inside the BODY element, heading elements H1 through H6 are generally used for major divisions of the document. Headings are permitted to appear in any order, but you will obtain the best results when your documents are displayed in a browser if you follow these guidelines:

- H1: should be used as the highest level of heading, H2 as the next highest, and so forth.

- You should not skip heading levels: e.g., an H3 should not appear after an H1, unless there is an H2 between them.

```
<HTML>

<HEAD>

<TITLE> Example Page</TITLE>

</HEAD>

<BODY>

<H1> Heading 1 </H1>

<H2> Heading 2 </H2>

<H3> Heading 3 </H3>

<H4> Heading 4 </H4>

<H5> Heading 5 </H5>

<H6> Heading 6 </H6>

</BODY>

</HTML>
```

HTML Paragraph

Paragraphs allow you to add text to a document in such a way that it will automatically adjust the end of line to suite the window size of the browser in which it is being displayed. Each line of text will stretch the entire length of the window.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>My First Page!!!!</title>
  </head>
  <body bgcolor="rgb(255,242,255)" text="yellow">
    <!--html elements-->
    <!--paragraph element-->
    <p>
      Lorem ipsum dolor sit amet consectetur adipisicing elit. Odio, nisi! Dicta
      nemo porro, aperiam ex, vero necessitatibus nostrum debitis obcaecati
      officiis nam numquam minima nisi tempora itaque expedita amet voluptatum!
    </p>
  </body>
</html>
```

1.5 HTML Lists: Unordered Lists, Ordered Lists, Description Lists, Nesting and Mixed Lists, Lists for Navigation Menus

An HTML list helps present data on web pages in a structured manner, either in an ordered or unordered format, making the content easier to read and visually attractive. Lists play a key role in creating organized and accessible content in web development.

Types of HTML Lists

1.5.0 Unordered Lists (<ul>): These lists are used for items that do not need to be in any specific order. The list items are typically marked with bullets.

1.5.1 Ordered Lists (<ol>): These lists are used when the order of the items is important. Each item in an ordered list is typically marked with numbers or letters.

Attributes:

type → numbering style (1, a, A, i, I)

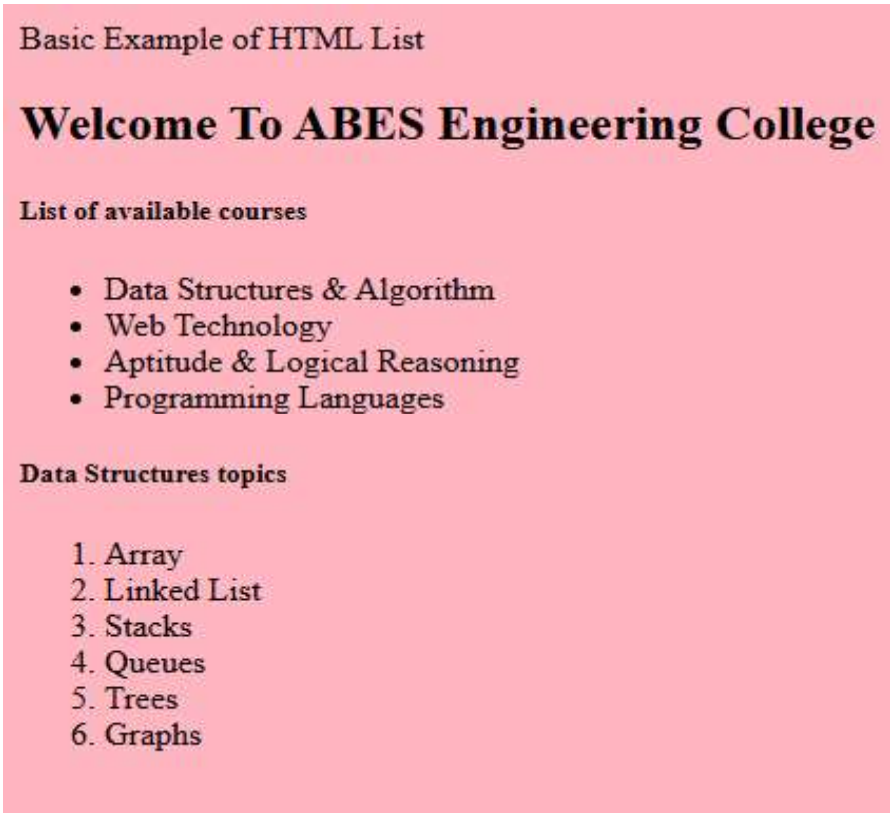
start → starting number/letter

reversed → reverses order

Example:

```
    <html>   </head>  <body>
Basic Example of HTML List
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport"
        content="width=device-width, initial-scale=1.0">
   <title>HTML</title>
</head>    <body>
   <h2>Welcome To ABES Engineering College </h2>
   <h5>List of available courses</h5>
   <ul>
        <li>Data Structures & Algorithm</li>
          <li>Web Technology</li>
          <li>Aptitude & Logical Reasoning</li>
         <li>Programming Languages</li>
 </ul>    <h5>Data Structures topics</h5>
   <ol>
         <li>Array</li>
           <li>Linked List</li>
           <li>Stacks</li>
           <li>Queues</li>
           <li>Trees</li>
           <li>Graphs</li>
       </ol>   </body>   </html>
```

Output:



1.5.2 Description Lists (<dl>): These lists are used to contain terms and their corresponding descriptions.

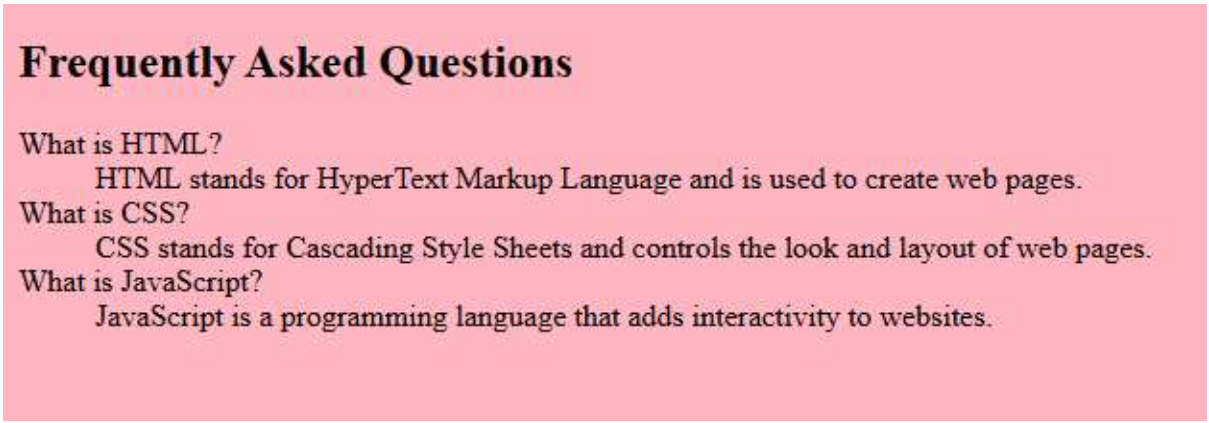
Contains:

1. <dt> → **Definition Term**
2. <dd> → **Definition Description**

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport"
    content="width=device-width, initial-scale=1.0">
    <title>HTML</title>
</head>
<body style="background-color: #FFB6C1";>
   <h2>Frequently Asked Questions</h2>
<dl>
  <dt>What is HTML?  </dt>
  <dd>HTML stands for HyperText Markup Language and is used to create web pages. </dd>
  <dt>What is CSS? </dt>
  <dd>CSS stands for Cascading Style Sheets and controls the look and layout of web pages.
</dd>
  <dt>What is JavaScript? </dt>
  <dd>JavaScript is a programming language that adds interactivity to websites. </dd>
</dl>
  </body>   </html>   </body>
</html>
```

Output:



1.5.4 Nesting lists

Nesting lists means placing one list **inside an item of another list**. The nested list is usually wrapped inside an <li> element of the parent list.

The purpose of Nesting List:

✓ To represent **hierarchical data** (e.g., categories and subcategories).
✓ To create **multi-level navigation menus**.
✓ To organize **outlines, directories, or structured content**.

Key Rules for Nesting Lists:

The nested list **must be inside an <li> element** of the parent list.

Nesting can be:

- **Unordered inside unordered** (<ul> in <ul>).
- **Ordered inside unordered** (<ol> in <ul>), and vice versa.

✓ Indentation (visually) helps users understand the hierarchy, but this is **done via CSS**, not by adding spaces in HTML.

Types of Nesting

1. **Unordered → Unordered** (Common in multi-level bullet points)
2. **Ordered → Ordered** (Common in outlines or steps with sub-steps)
3. **Mixed Nesting** (Combination of <ul> and <ol>)
4. **Nesting inside Description List** (less common, but possible inside <dd>)

Example of Nesting Lists:

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport"
        content="width=device-width, initial-scale=1.0">
   <title>HTML</title>
</head>
<body style="background-color: #FFB6C1";>
<h3>Shopping List</h3>
<ul>
 <li>Fruits
   <ul>
     <li>Apple</li>
     <li>Mango</li>
     <li>Orange</li>
   </ul>
 </li>
 <li>Vegetables
   <ul>
     <li>Carrot</li>
     <li>Potato</li>
     <li>Tomato</li>
   </ul>
 </li>
</ul>
 </body>
</html>
```

Output:

1.5.5 Mixed lists

Mixed lists in HTML refer to the combination of **ordered lists (<ol>) and unordered lists (<ul>) within each other**. This means that an ordered list can contain an unordered list as a nested list or vice versa. The purpose of using mixed lists is to represent complex hierarchical information where some items need to be displayed in a specific order, while others do not.

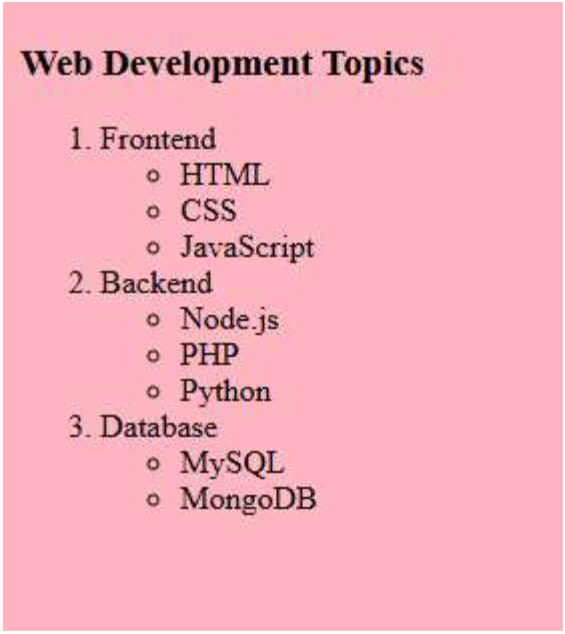
Mixed Lists uses:

- **To organize information with different importance levels:** For example, main steps (ordered) and sub-options (unordered).
- **To create structured outlines:** Where the top level is numbered, and the sub-points are bulleted.
- **To maintain clarity in instructions, menus, and structured content.**

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>HTML</title>
</head>
<body style="background-color: #FFB6C1;">
    <h3>Web Development Topics</h3>
    <ol>    <li>Frontend
            <ul>
                <li>HTML</li>
                <li>CSS</li>
                <li>JavaScript</li>
            </ul>   </li>
        <li>Backend
            <ul>
                <li>Node.js</li>
                <li>PHP</li>
                <li>Python</li>
            </ul>
        </li>
        <li>Database
            <ul>
                <li>MySQL</li>
                <li>MongoDB</li>
            </ul></li>
    </ol>   </body>   </html>
```

Output:



1.5.7 Lists for Navigation Menus

Navigation menus are a crucial part of any website, allowing users to move between different pages or sections easily. In HTML, navigation menus are commonly created using unordered lists (<ul>) because the order of menu items is usually not important. These lists are typically placed inside a <nav> element, which provides semantic meaning, indicating that the section is for navigation purposes.

The best practice for creating navigation menus is to wrap each navigation link inside an <li> element, and then place all <li> elements within a <ul>. Each menu item is usually linked using the <a> (anchor) tag inside the list item. This structure ensures that the menu is both semantic and accessible, improving usability for screen readers and assistive technologies.

The uses of Navigation list:

Semantic Structure: Using <nav> with <ul> and <li> clearly defines the navigation section.

Accessibility: Screen readers can interpret lists properly, making navigation easier for visually impaired users.

Styling Flexibility: Lists provide an easy structure for applying CSS to create horizontal or vertical menus.

Consistency: Most browsers and frameworks support list-based menus, making it a standard practice.

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>HTML</title>
</head>
<body style="background-color: #FFB6C1;">
    <nav>
        <span><a href="index.html">Home</a></span>   
        <span><a href="about.html">About</a></span>   
        <span><a href="services.html">Services</a></span>   
        <span><a href="contact.html">Contact</a></span>
    </nav>

    <h1>Welcome to My Website</h1>
    <p>This is a simple HTML page with a navigation bar.</p>
</body>
</html>
```

Output:

1.6 HTML Links, Images and attributes, Comment in HTML, HTML Formatting Elements/ Special Character & Symbol

1.6.0 HTML Links

HTML links are hyperlinks.

You can click on a link and jump to another document.

When you move the mouse over a link, the mouse arrow will turn into a little hand.

HTML Links - Syntax
The HTML **<a>** tag defines a hyperlink. It has the following syntax:

link text
The most important attribute of the <a> element is the href attribute, which indicates the link's destination.

```
<!DOCTYPE html>
<html>
<body>

<h1>HTML Links</h1>

<p><a href="https://www.abes.ac.in/">Welcome ABES EC</a></p>

</body>
</html>
```

1.6.1 HTML Image

The HTML <img> tag is used to embed an image in a web page.

Images are not technically inserted into a web page; images are linked to web pages. The <img> tag creates a holding space for the referenced image.

The <img> tag is empty; it contains attributes only, and does not have a closing tag.

The <img> tag has two required attributes:

src - Specifies the path to the image
alt - Specifies an alternate text for the image

```
<!DOCTYPE html>

<html>

<body>



<h2>Alternative text</h2>



<p>The alt attribute should reflect the image content, so users who cannot see the image get an
understanding of what the image contains:</p>



<img src="img_logo.jpg" alt="ABES Logo" width="460" height="345">



</body>

</html>
```

1.6.2 HTML Comments

```
<!DOCTYPE html>

<html lang="en">

 <head>

  <meta charset="UTF-8" />

  <meta name="viewport" content="width=device-width, initial-scale=1.0" />

  <title>HTML Comments</title>

 </head>

 <body> <!—body start -->

<!—section is started from here! -->

<section>

<h2> HTML Comments</h2>
<p>The comment tag is used to insert comments in the source code. Comments are not displayed in the browsers.<br> You can use comments to explain your code, which can help you when you edit the source code at a later date. <br>This is especially useful if you have a lot of code..</p>

 </section>

</body> <!--body end--></html>
-------------------------
```

Comments can be inserted into the HTML code to make it more readable and understandable. Comments are ignored by the browser and are not displayed.

Output:

1.6.3 HTML Formatting Elements

In this chapter, you will learn how to enhance your page with Bold, Italics, and other character formatting options.

<FONT SIZE="+2"> Two sizes bigger</FONT>

The size attribute can be set as an absolute value from 1 to 7 or as a relative value using the "+" or "-" sign. Normal text size is 3 (from -2 to +4).

<FONT COLOR="#RRGGBB">this text has color</FONT>

Color = "#RRGGBB" The COLOR attribute of the FONT element.

<B> Bold </B>

<I> Italic </I>

<U> Underline </U>

<PRE> Preformatted </PRE>

Text enclosed by PRE tags is displayed in a mono-spaced font. Spaces and line breaks are supported without additional elements or special characters.

<EM> Emphasis </EM>

Browsers usually display this as italics.

<STRONG> STRONG </STRONG>

Browsers display this as bold.

<TT> TELETYPE </TT>

Text is displayed in a mono-spaced font. A typewriter text, e.g. fixed-width font.

<STRIKE> strike-through text</STRIKE>

DEL is used for STRIKE at the latest browsers

<BIG> places text in a big font</BIG>

<SMALL> places text in a small font</SMALL>

_{places text in subscript position}

^{places text in superscript style position}

<ins>showing new inserted text</ins>

<mark>used for highlighting</mark>

1.6.4 HTML ENTITIES or Characters & Symbols

Some characters are reserved in HTML.

If you use the less than (<) or greater than (>) signs in your HTML text, the browser might mix them with tags.

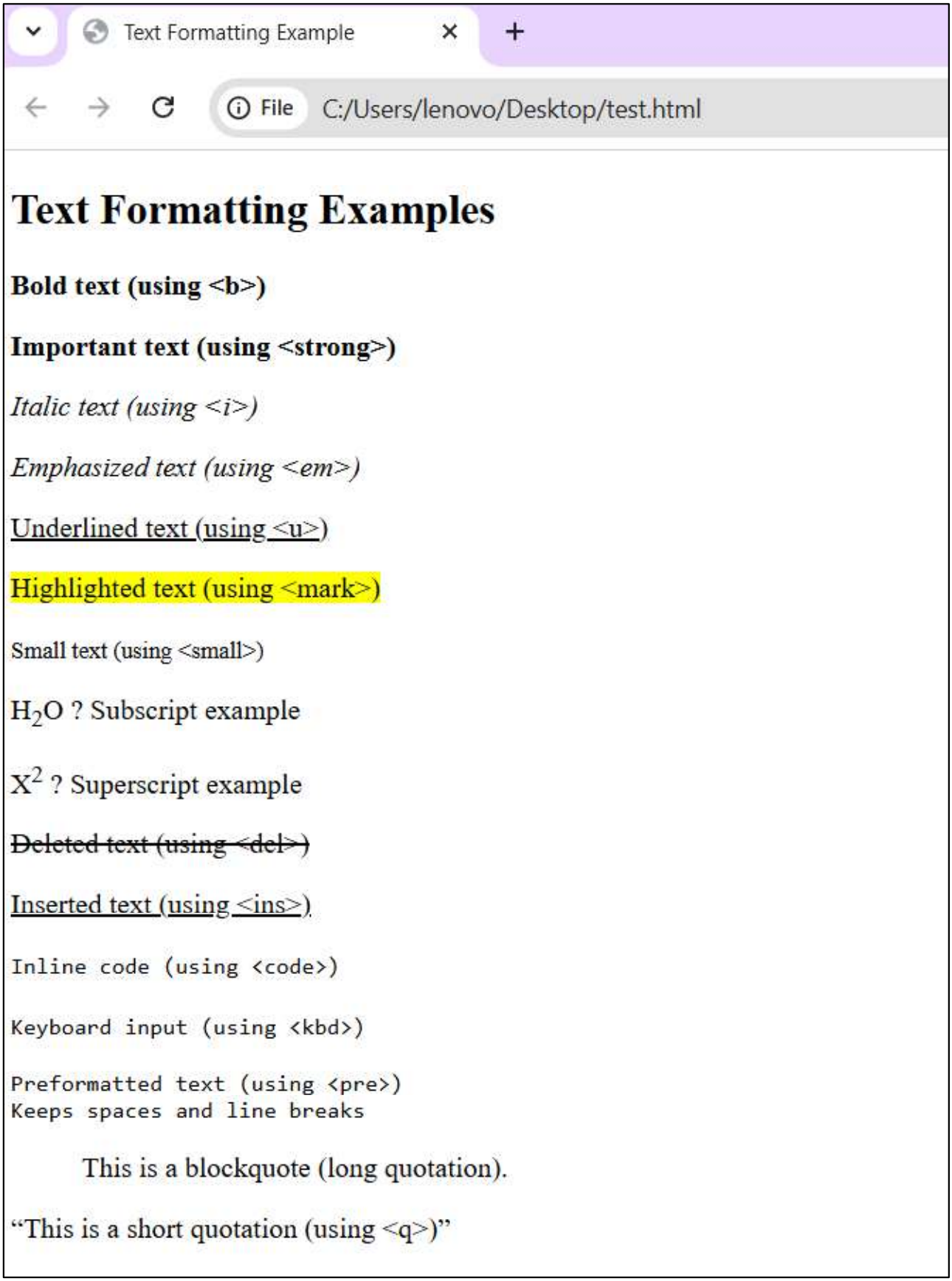
- Entity names or entity numbers can be used to display reserved HTML characters.
- Entity names look like this: &entity_name;
- Entity numbers look like this: &#entity_number;

These Characters are recognized in HTML as they begin with an ampersand and end with with a semi- colon e.g. &value; The value will either be an entity name or a standard ASCII character number. They are called escape sequences.

The next table represents some of the more commonly used special characters. For a comprehensive listing.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Text Formatting Example</title>
</head>
<body>
  <h2>Text Formatting Examples</h2> <p><b>Bold text (using
  &lt;b&gt;)</b></p> <p><strong>Important text (using
  &lt;strong&gt;)</strong></p> <p><i>Italic text (using
  &lt;i&gt;)</i></p> <p><em>Emphasized text (using
  &lt;em&gt;)</em></p> <p><u>Underlined text (using
  &lt;u&gt;)</u></p> <p><mark>Highlighted text (using
  &lt;mark&gt;)</mark></p> <p><small>Small text (using
  &lt;small&gt;)</small></p> <p>H<sub>2</sub>O → Subscript
  example</p> <p>X<sup>2</sup> → Superscript example</p>
  <p><del>Deleted text (using &lt;del&gt;)</del></p>

  <p><ins>Inserted text (using &lt;ins&gt;)</ins></p>
  <p><code>Inline code (using &lt;code&gt;)</code></p>

  <p><kbd>Keyboard input (using &lt;kbd&gt;)</kbd></p>
  <p><pre>

Preformatted text (using &lt;pre&gt;)
    Keeps spaces and line breaks
  </pre></p>
  <p><blockquote cite="https://www.example.com">
    This is a blockquote (long quotation).
  </blockquote></p>
<p><q>This is a short quotation (using &lt;q&gt;)</q></p>
</body></html>
```

Output:


Text Formatting Example
File
C:/Users/lenovo/Desktop/test.html
Text Formatting Examples
Bold text (using <b>)
Important text (using)
Italic text (using <i>)
Emphasized text (using)
Underlined text (using <u>)
Highlighted text (using <mark>)
Small text (using <small>)
H2O ? Subscript example
X2 ? Superscript example
Deleted text (using <del>)
Inserted text (using <ins>)
Inline code (using <code>)
Keyboard input (using <kbd>)
Preformatted text (using <pre>)
Keeps spaces and line breaks
This is a blockquote (long quotation).
“This is a short quotation (using <q>)”

Special Characters & Symbols OR ENTITIES

Special Character	Entity Name	Special Character	Entity Name
Ampersand	& &	Greater-than sign	> >
Asterisk	∗ ∗∗	Less-than sign	< <
Cent sign	¢ ¢	Non-breaking space	
Copyright	© ©	Quotation mark	" "
Fraction one qtr	¼ ¼	Registration mark	® ®
Fraction one half	½ ½	Trademark sign	™ ™

Additional escape sequences support-accented

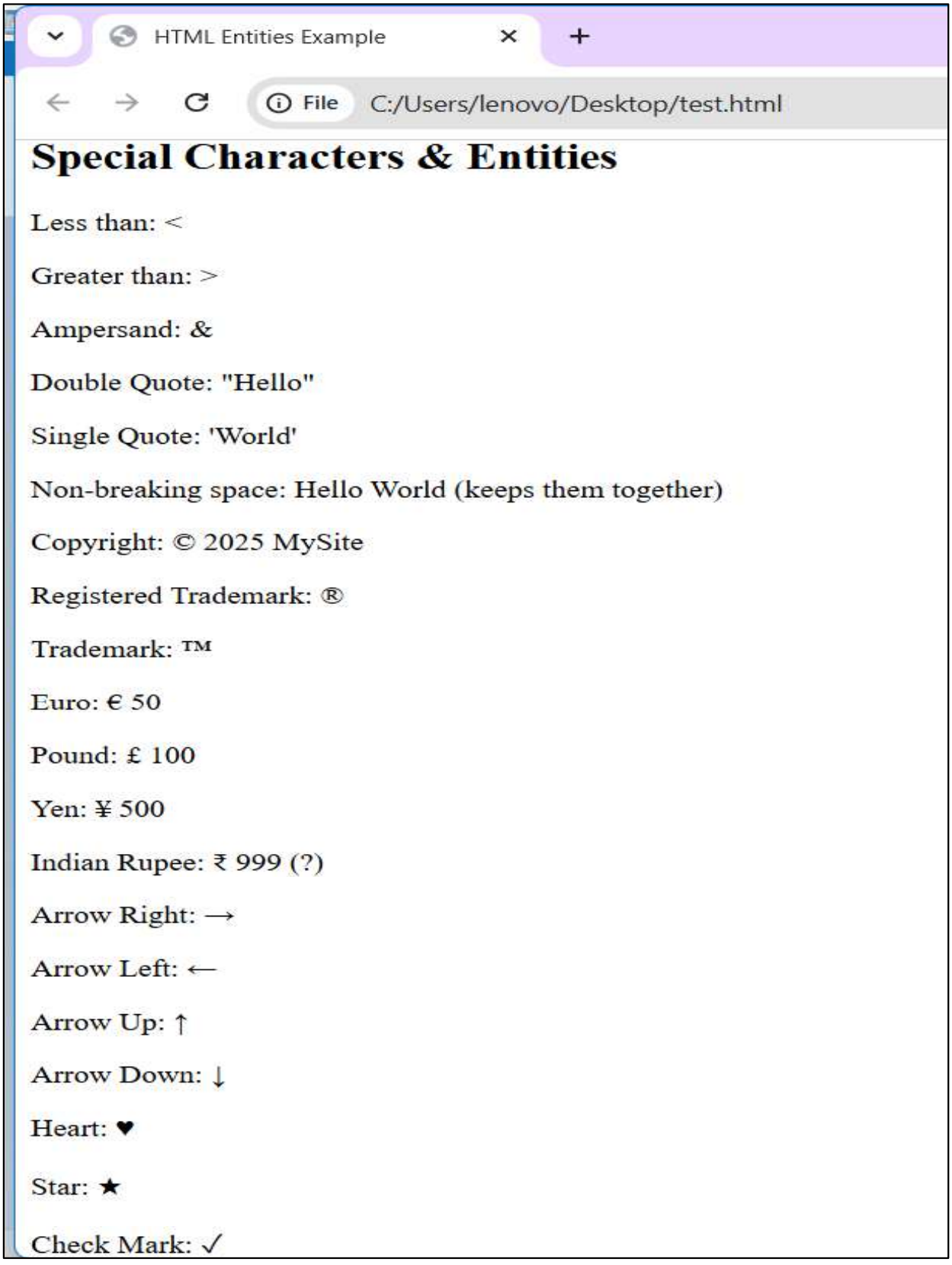
characters, such as:

- **ö** a lowercase o with an umlaut: ö
- **ñ** a lowercase n with a tilde: ñ
- **È** an uppercase E with a grave accent: È

NOTE: Unlike the rest of HTML, the escape sequences are case sensitive. You cannot, for instance, use < instead of <.

```
<html lang="en">
 <head>
  <meta charset="UTF-8" />
  <title>HTML Entities Example</title>
 </head>
 <body>
  <h2>Special Characters & Entities</h2>
  <p>Less than: &lt;</p>
  <p>Greater than: &gt;</p>
  <p>Ampersand: &amp;</p>
  <p>Double Quote: &quot;Hello&quot;</p>
  <p>Single Quote: &apos;World&apos;</p>
  <p>Non-breaking space: Hello World (keeps them together)</p>
  <p>Copyright: &copy; 2025 MySite</p>
  <p>Registered Trademark: &reg;</p>
  <p>Trademark: &trade;</p>
  <p>Euro: &euro; 50</p>
  <p>Pound: &pound; 100</p>
  <p>Yen: &yen; 500</p>
  <p>Indian Rupee: &#8377; 999 (₹)</p>
  <p>Arrow Right: &rarr;</p>
  <p>Arrow Left: &larr;</p>
  <p>Arrow Up: &uarr;</p>
  <p>Arrow Down: &darr;</p>
  <p>Heart: &hearts;</p>03;</p>
  </body>
  </html>
```

Output:


HTML Entities Example
File
C:/Users/lenovo/Desktop/test.html
Special Characters & Entities
Less than: <
Greater than: >
Ampersand: &
Double Quote: "Hello"
Single Quote: 'World'
Non-breaking space: Hello World (keeps them together)
Copyright: © 2025 MySite
Registered Trademark: ®
Trademark: ™
Euro: € 50
Pound: £ 100
Yen: ¥ 500
Indian Rupee: ₹ 999 (?)
Arrow Right: →
Arrow Left: ←
Arrow Up: ↑
Arrow Down: ↓
Heart: ♥
Star: ★
Check Mark: ✓

1.7 Text related Elements (Quotation and Citation Elements), Using Multimedia (Image/video/Audio/YouTube) in HTML, Creating & Using Hyperlinks

1.7.0 Text related Elements (Quotation and Citation Elements)

HTML provides specific elements for semantically marking up quoted text and citations, enhancing accessibility and structure.

- **<abbr>** Defines an abbreviation or acronym**<bdo>** Defines the text direction
- **<blockquote>** Defines a section that is quoted from another source
- **<cite>** Defines the title of a work
- **<q>** Defines a short inline quotation

- **<address>** Defines contact information for the author/owner of a document

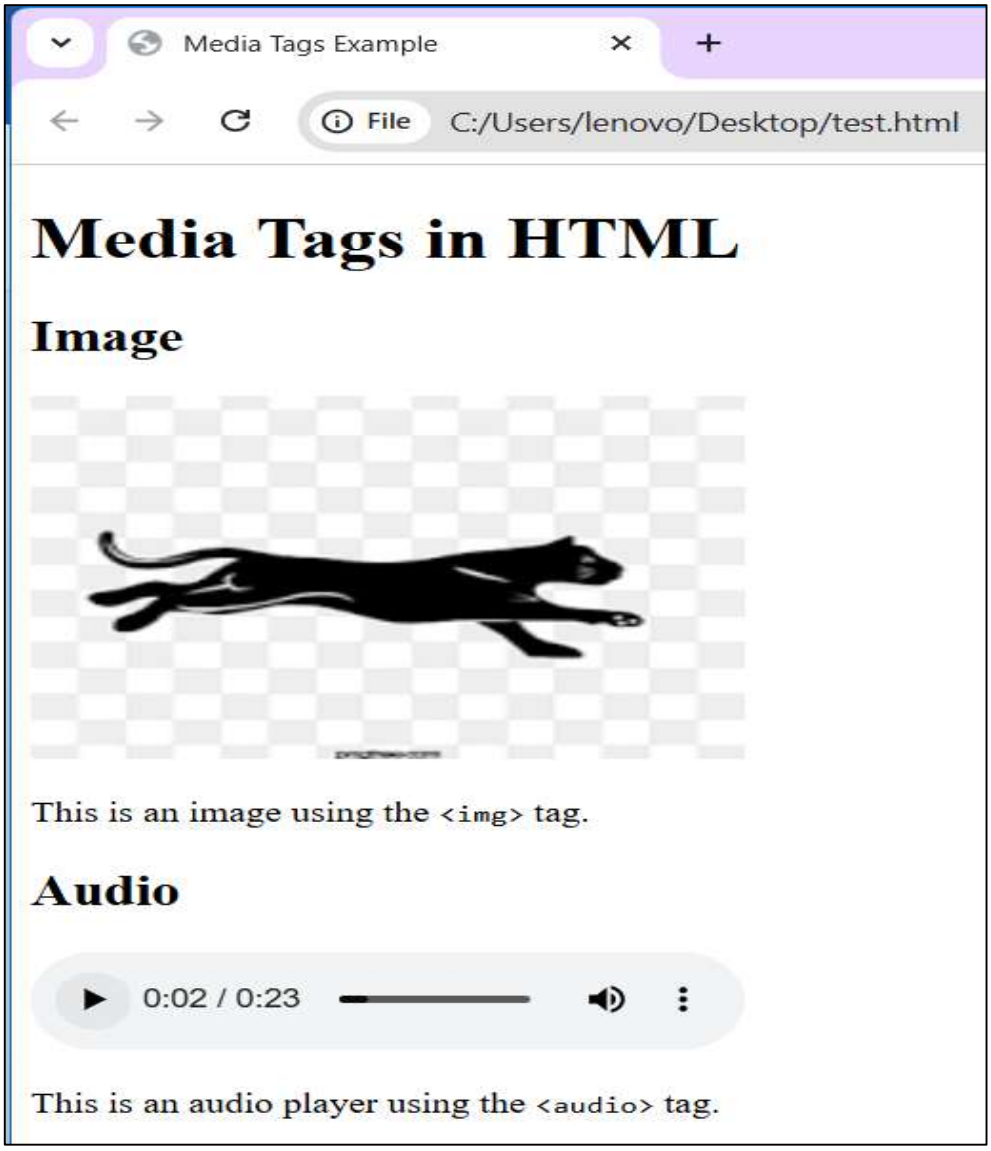
1.7.1 Using Multimedia (Image/video/Audio/you tube) in HTML

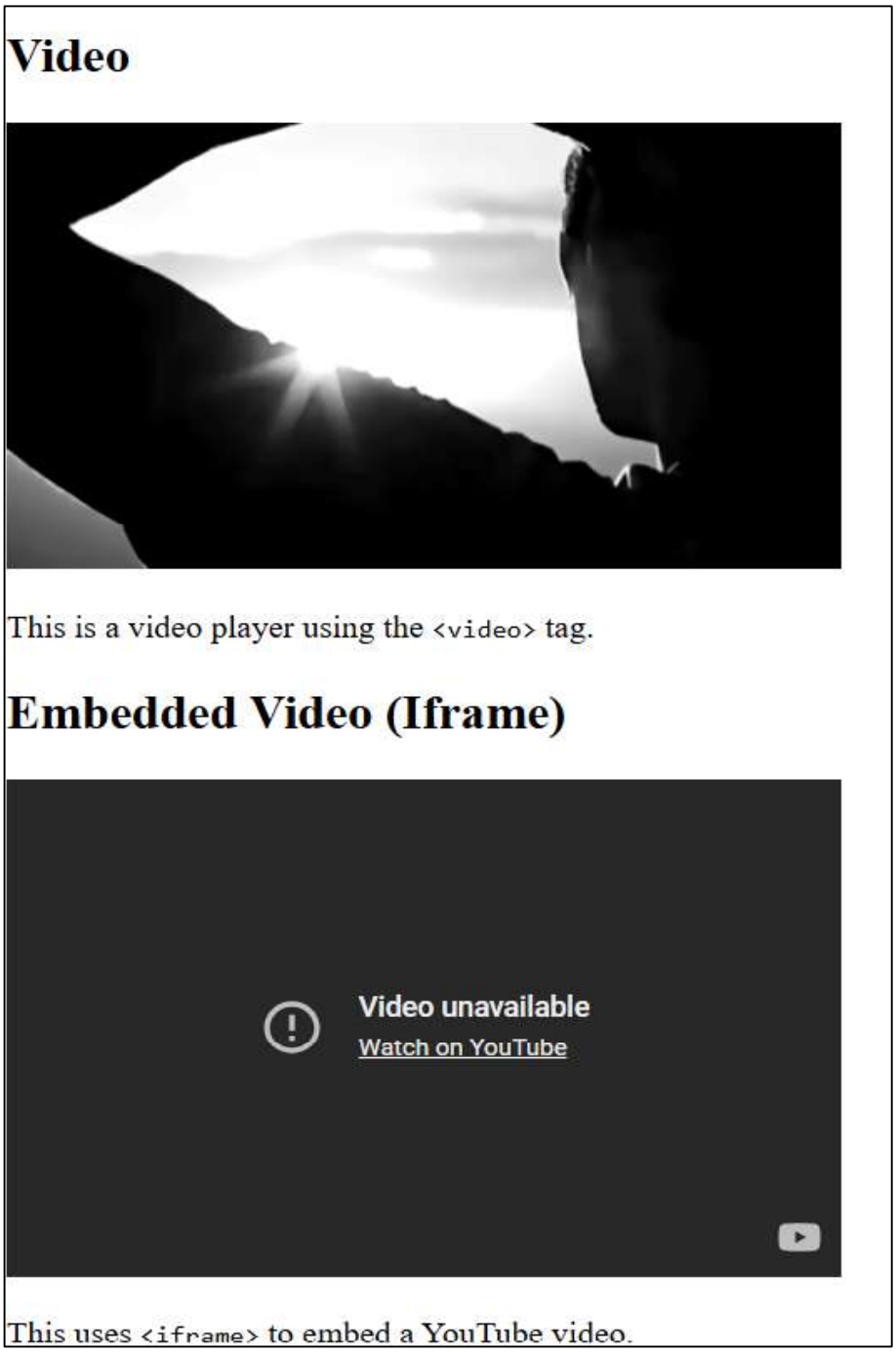
A variety of tags such as the <img> tag, <video> tag, and <audio> tag are available in HTML to include media on your web page. Multimedia combines different media, such as images, audio, and videos. Users will have a better experience when multimedia is embedded into HTML.

Media Tag	Description
<audio>	An inline element is used to embed sound files into a web page.
<video>	Used to embed video files into a web page.
<source>	Used to attach multimedia files like audio, video, and pictures.
<embed>	Used for embedding external applications, generally multimedia content like audio or video, into an HTML document.<
<track>	Specifies text tracks for media components, specifically for audio and video elements.

```
<!DOCTYPE html>
<html lang="en">
 <head>
  <meta charset="UTF-8" />
  <title>Media Tags Example</title>
 </head>
 <body>
  <h1>Media Tags in HTML</h1>
  <h2>Image</h2>   <!-- Image Example -->
  <img src=https://via.placeholder.com/300x200 alt="Sample Image" width="300" height="200" />
  <p>This is an image using the <code>&lt;img&gt;</code> tag.</p>

  <!-- Audio Example --><h2>Audio</h2>

  <audio controls>
   <source src="sample-audio.mp3" type="audio/mpeg" />
  <source src="sample-audio.ogg" type="audio/ogg" />
  Your browser does not support the audio element.
  </audio>

  <p>This is an audio player using the <code>&lt;audio&gt;</code>
  tag.</p> <h2>Video</h2> <!-- Video Example --> <video width="400"
  controls>

   <source src="sample-video.mp4" type="video/mp4" />
   <source src="sample-video.ogg" type="video/ogg" />
   Your browser does not support the video tag. </video>

  <p>This is a video player using the <code>&lt;video&gt;</code> tag.</p>
  <!-- Iframe Example -->
  <h2>Embedded Video (Iframe)</h2>
  <iframe width="400" height="250" src="https://www.youtube.com/embed/dQw4w9WgXcQ"
   title="YouTube Video" frameborder="0" allowfullscreen ></iframe>

  <p>This uses <code>&lt;iframe&gt;</code> to embed a YouTube video.</p>
</body> </html>
```

Output:


Media Tags Example
File
C:/Users/lenovo/Desktop/test.html
Media Tags in HTML
Image
This is an image using the <img> tag.
Audio
0:02 / 0:23
This is an audio player using the <audio> tag.



Advantage of Media tag:

- Plugins are no longer required
- fashion than imported third-party
- Native (built-in) controls are provided by the browser.
- Accessibilities (keyboard, mouse) are built-in automatically

1.7.3 Creating & Using Hyperlinks

The <a> HTML element (or anchor element), with its href attribute, creates a hyperlink to web pages, files, email addresses, locations in the same page, or anything else a URL can address.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>hyperlink in HTML</title>
  </head>
  <body>
    <h1>Hyperlink in HTML</h1>
    <!-- Hyperlink Example -->
<a href="http://www.facebook.com" target= " _blank" > This is a link</a>
  </body>
</html>
```

Output:

1.8 Accessibility & Semantic Considerations

Semantic elements = elements with a meaning.

Accessibility considerations focus on designing websites for users of all abilities, while semantic HTML provides the structural meaning and context that assistive technologies like

screen readers rely on to interpret content for visually impaired users and enable keyboard navigations.

When using lists in HTML, it is important to follow semantic and accessibility best practices to ensure that content is understandable and navigable for all users, including those using assistive technologies. **Semantic HTML means using the correct tags for their intended purpose:** for unordered lists, <ol> for ordered lists, <dl> for description lists, and <nav> for navigation menus. This not only improves readability for developers but also allows screen readers and other assistive tools to interpret the structure of the content accurately.

For navigation menus, wrapping menu items inside <li> elements within a <ul> and enclosing the whole menu in a <nav> element provides clear semantic meaning. Each link (<a> tag) should have descriptive text, avoiding generic phrases like "Click here," so users understand the purpose of each link. Proper use of semantic tags helps screen readers announce the type and number of items in a list, enabling users to navigate efficiently.

Other accessibility considerations include maintaining sufficient color contrast, providing visible focus indicators for keyboard navigation, and avoiding excessive list nesting that may confuse users. Following these semantic and accessibility principles ensures that HTML lists are both meaningful and user-friendly across different devices and abilities.

Accessibility & Semantic Features in this Example: semantic glossary using a description list (<dl>). Each term (<dt>) like "HTML" or "CSS" is paired with its description (<dd>), making the relationship clear. Semantic tags help screen readers announce terms and definitions correctly, improving accessibility. Optional id attributes allow linking directly to specific terms. CSS can style the list for readability without affecting its structure, ensuring the glossary is both user-friendly and accessible.

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
</head>
<body style="background-color: #ffb6c1;">
    <h2>Web Development Glossary</h2>
    <dl>
        <dt id="html-term">HTML</dt>
        <dd>
            HyperText Markup Language, the standard language for creating web pages.
        </dd>
        <dt id="css-term">CSS</dt>
        <dd>
            Cascading Style Sheets, used for styling and layout of web pages.
        </dd>
        <dt id="js-term">JavaScript</dt>
        <dd>
            A programming language used to make web pages interactive.
        </dd>
        <dt id="api-term">API</dt>
        <dd>
            Application Programming Interface, a set of rules that allows software
            programs to communicate with each other.
        </dd>    </dl>
</body>   </html>
```

Output:

Web Development Glossary

HTML
: HyperText Markup Language, the standard language for creating web pages.

CSS
: Cascading Style Sheets, used for styling and layout of web pages.

JavaScript
: A programming language used to make web pages interactive.

API
: Application Programming Interface, a set of rules that allows software programs to communicate with each other.

Practical Use case:

1. **A company website lists its main navigation items like Home, About, Services, and Contact. Additionally, a product page lists key features using bullet points.**

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>HTML</title>
</head>   <body style="background-color: rgb(180, 233, 120);">
    <!-- Navigation Menu -->
    <h3>Company Navigation Menu</h3>
    <nav aria-label="Main Navigation">
        <ul>
            <li><a href="index.html">Home</a></li>
            <li><a href="about.html">About</a></li>
            <li><a href="services.html">Services</a></li>
            <li><a href="contact.html">Contact</a></li>
        </ul>   </nav>      <!-- Product Features -->
    <h3>Product Key Features</h3>
    <div class="features">
        <ul>
            <li>High-speed performance</li>
            <li>Durable design</li>
            <li>Affordable price</li>
            <li>24/7 customer support</li>
        </ul>    </div>    </body>   </html>
```

Output:


Company Navigation Menu
Home About Services Contact
Product Key Features
High-speed performance
Durable design
Affordable price
24/7 customer support

2. **A cooking website lists a recipe where steps must be followed in order, such as mixing ingredients, baking, and serving.**

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8" />
   <meta name="viewport" content="width=device-width, initial-scale=1.0" />
   <title>HTML</title>
</head>
<body>
   <h1>Welcome to My Cooking Page</h1>
   <p>This page contains a simple recipe for baking a cake.</p>

   <h3>Recipe: How to Bake a Cake</h3>
   <ol>
      <li>Preheat the oven to 180°C (350°F).</li>
      <li>Mix flour, sugar, eggs, and butter in a bowl.</li>
      <li>Pour the batter into a greased baking tray.</li>
      <li>Bake for 25–30 minutes or until golden brown.</li>
      <li>Remove from oven and let it cool.</li>
      <li>Serve and enjoy!</li>
   </ol>
</body>
</html>
```

Output:


Welcome to My Cooking Page
This page contains a simple recipe for baking a cake.
Recipe: How to Bake a Cake
1. Preheat the oven to 180°C (350°F).
2. Mix flour, sugar, eggs, and butter in a bowl.
3. Pour the batter into a greased baking tray.
4. Bake for 25–30 minutes or until golden brown.
5. Remove from oven and let it cool.
6. Serve and enjoy!

3. **A web development tutorial defines terms like HTML, CSS, and JavaScript.**

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>HTML</title>
</head>
<body style="background-color: rgb(180, 233, 120);">
    <h3>Web Development Glossary</h3>
    <dl>
        <dt>HTML</dt>
        <dd>HyperText Markup Language, used to structure content on the web.</dd>
        <dt>CSS</dt>
        <dd>Cascading Style Sheets, used to style and layout web pages.</dd>
        <dt>JavaScript</dt>
        <dd>A programming language that makes web pages interactive.</dd>
        <dt>API</dt>
        <dd>Application Programming Interface, allows communication between
software programs.</dd>
    </dl>
</body>
</html>
```

Output:

Web Development Glossary

HTML
: HyperText Markup Language, used to structure content on the web.

CSS
: Cascading Style Sheets, used to style and layout web pages.

JavaScript
: A programming language that makes web pages interactive.

API
: Application Programming Interface, allows communication between software programs.

4. **An e-commerce website has a main category “Products” with subcategories “Electronics” and “Clothing.” Electronics further includes “Mobiles” and “Laptops.**

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>HTML</title>
</head>
<body style="background-color: rgb(180, 233, 120);">
    <h3>Product Categories</h3>
    <ul>
        <li>Products
            <ul>
                <li>Electronics
                    <ul>
                        <li>Mobiles</li>
                        <li>Laptops</li>
                    </ul>
                </li>
                <li>Clothing</li>
            </ul>
        </li>
        <li>About Us</li>
        <li>Contact</li>
    </ul>
</body>
</html>
```

Output:


Product Categories
Products
Electronics
Mobiles
Laptops
Clothing
About Us
Contact

2.0 HTML Table:Creating and using Tables,Basic Table Structure,Table Headers and Captions

2.0.0 HTML Table

An HTML table is a way to organize data in rows and columns. We create it with the <table> element in HTML. Tables mainly show tabular data like schedules, product lists, or reports.

Basic HTML Table Structure

An HTML table is created using the **<table> tag**. Inside the table, we use:

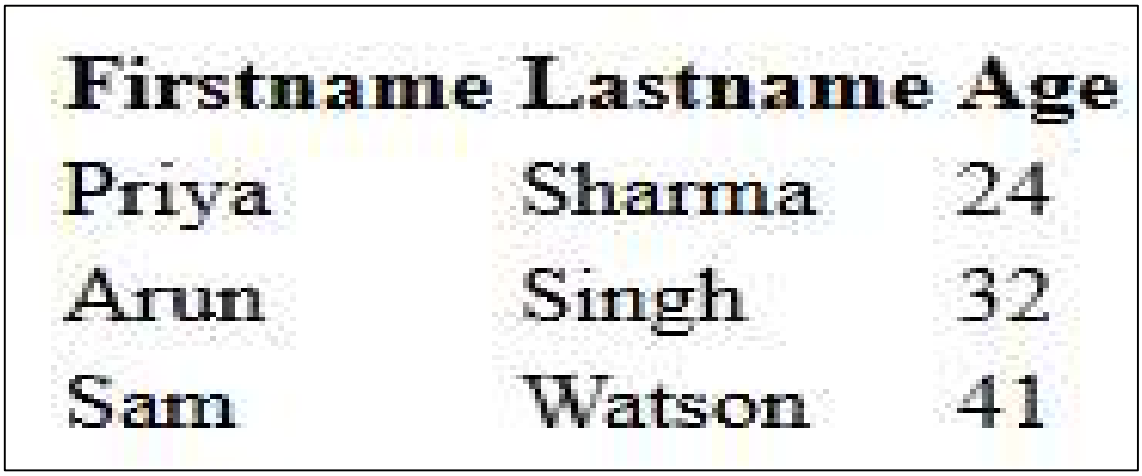
- **<tr>:** Table Row
- **<th>:** Table Header
- **<td>:** Table Data

Basic Example:

index.html > html

```
<!-- index.html -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport"
          content="width=device-width, initial-scale=1.0">
    <title>HTML</title>
</head>
<body>
    <table>
        <tr>
            <th>Firstname</th>
            <th>Lastname</th>
            <th>Age</th>
        </tr>
        <tr>
            <td>Priya</td>
            <td>Sharma</td>
            <td>24</td>
        </tr>
        <tr>
            <td>Arun</td>
            <td>Singh</td>
            <td>32</td>
        </tr>
        <tr>
            <td>Sam</td>
            <td>Watson</td>
            <td>41</td>
        </tr>
    </table>
</body>
</html>
```

Output:



Firstname	Lastname	Age
Priya	Sharma	24
Arun	Singh	32
Sam	Watson	41

Tag	Description
<table>	Defines the structure for organizing data in rows and columns on a web page.
<tr>	Represents a row in an HTML table that contains individual cells.
<th>	Shows a table header cell, which usually holds titles or headings.
<td>	Represents a standard data cell that holds content or data.
<caption>	Provides a title or description for the entire table.
<thead>	Defines the header section of a table, often containing column labels.
<tbody>	Represents the main content area of a table, separating it from the header or footer.
<tfoot>	Specifies the footer section of a table that typically holds summaries or totals.
<col>	Defines attributes for table columns that can apply to multiple columns at once.
<colgroup>	Groups together a set of columns in a table so you can apply formatting or properties to all of them at once.

2.0.1 Basic Table Structure in HTML

HTML tables are not just a collection of rows and columns; they can be **logically grouped into sections** for better organization, accessibility, and styling. The three main grouping tags are:

1) **<thead>:** Groups header rows.
2) **<tbody>:** Groups body rows.
3) **<tfoot>:** Groups footer rows.

Example:

```
index.html > html > body > table
<!-- index.html -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport"
          content="width=device-width, initial-scale=1.0">
    <title>HTML</title>
</head>
<body>
  <table>
  <thead>
    <tr><th>Header 1</th><th>Header 2</th></tr>
  </thead>
  <tbody>
    <tr><td>Data 1</td><td>Data 2</td></tr>
  </tbody>
  <tfoot>
    <tr><td colspan="2">Footer</td></tr>
  </tfoot>
</table>
</body>
</html>
```

Output

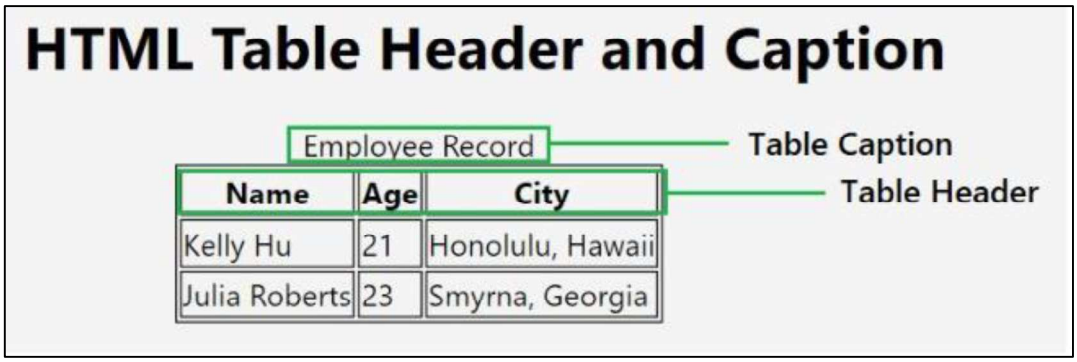
Header 1	Header 2
Data 1	Data 2
Footer	

2.0.2 Table Headers and Captions

Headers and captions help organize and present data in tables.

The table heading is important because it labels the columns. The **<th>** (table header) element defines these headings.

Captions give a title or explanation for the table. The **<caption>** element is placed right after the opening table tag.



Syntax to Create Table's Header and Caption:

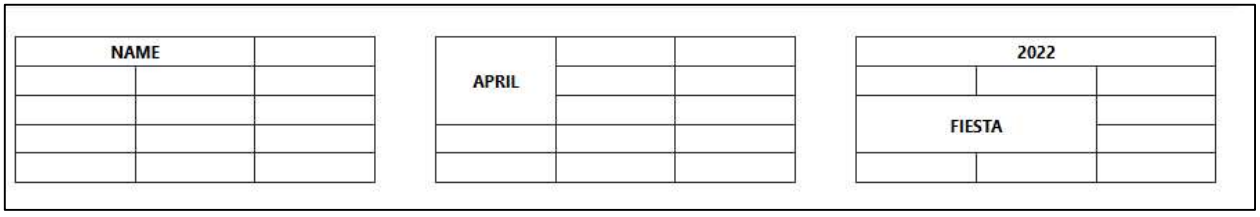
The following is the syntax to create a header and caption for an HTML table:

```
<table>
<caption>Description of table</caption>
<tr>
   <th>heading 1</th>
   <th>heading 2</th>
   <th>heading 3</th>
</tr>
</table>
```

2.1 HTML Table: Rowspan and Colspan, Table Attributes, Responsive Tables

2.1.0 HTML Table Colspan & Rowspan

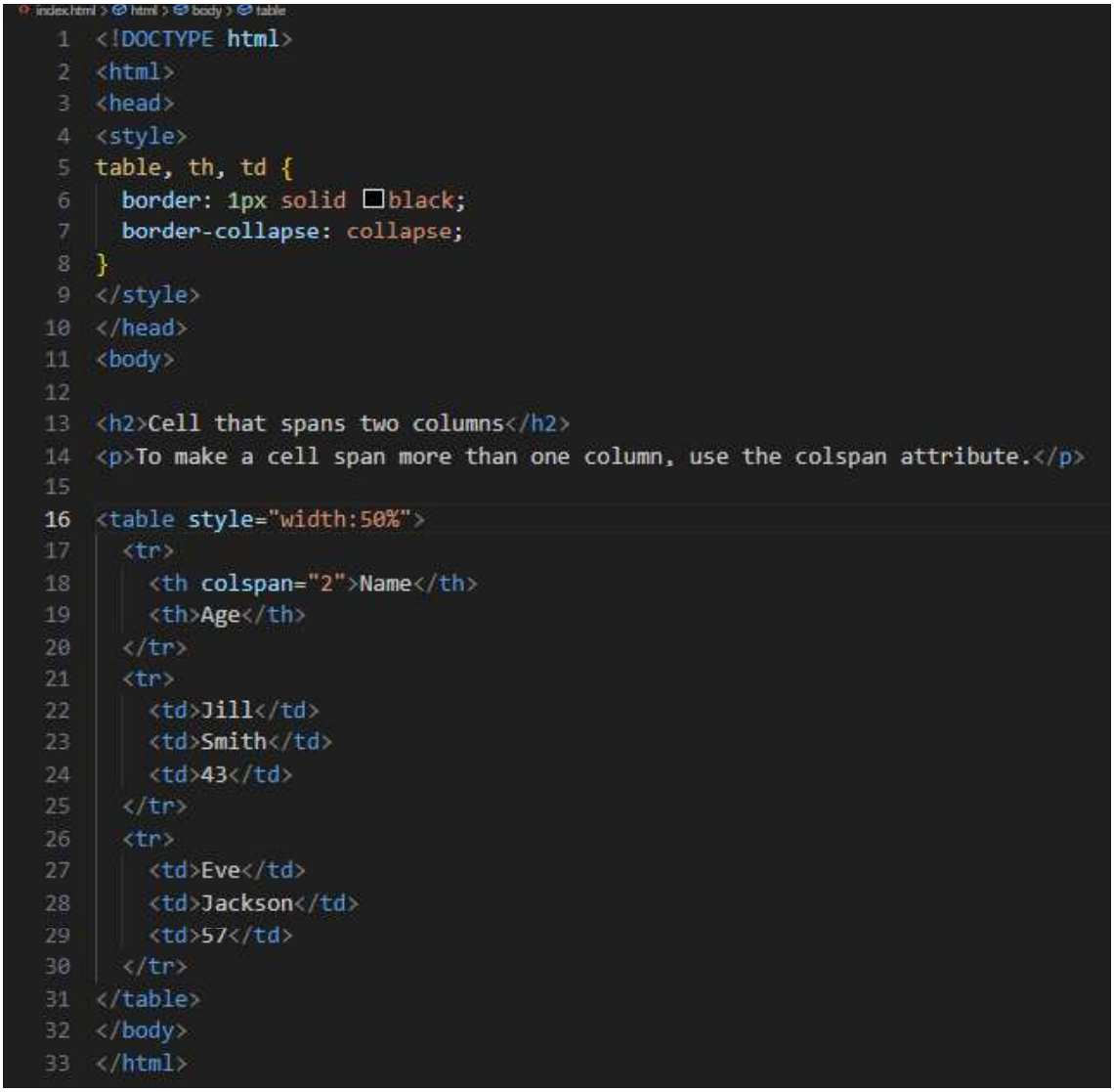
HTML tables can have cells that span over multiple rows and/or columns.



HTML Table - Colspan

To make a cell span over multiple columns, use the colspan attribute:

Example:



```
<!DOCTYPE html>
<html>
<head>
<style>
table, th, td {
  border: 1px solid black;
  border-collapse: collapse;
}
</style>
</head>
<body>

<h2>Cell that spans two columns</h2>
<p>To make a cell span more than one column, use the colspan attribute.</p>

<table style="width:50%">
  <tr>
    <th colspan="2">Name</th>
    <th>Age</th>
  </tr>
  <tr>
    <td>Jill</td>
    <td>Smith</td>
    <td>43</td>
  </tr>
  <tr>
    <td>Eve</td>
    <td>Jackson</td>
    <td>57</td>
  </tr>
</table>
</body>
</html>
```

Output:

Cell that spans two columns

To make a cell span more than one column, use the colspan attribute.

Name		Age
Jill	Smith	43
Eve	Jackson	57

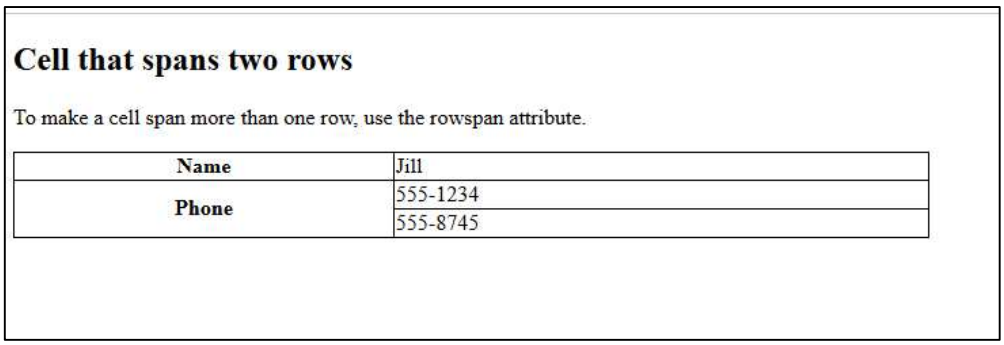
HTML Table - Rowspan

To make a cell span over multiple rows, use the rowspan attribute:

Example:

```
index.html > html > body > table
<!DOCTYPE html>
<html>
<head>
<style>
table, th, td {
  border: 1px solid black;
  border-collapse: collapse;
}
</style>
</head>
<body>

<h2>Cell that spans two rows</h2>
<p>To make a cell span more than one row, use the rowspan attribute.</p>

<table style="width:50%">
  <tr>
    <th>Name</th>
    <td>Jill</td>
  </tr>
  <tr>
    <th rowspan="2">Phone</th>
    <td>555-1234</td>
  </tr>
  <tr>
    <td>555-8745</td>
  </tr>
</table>
</body>
</html>

```

Output:



2.1.1 HTML Table Attributes

HTML tables can be customized using attributes to control appearance, spacing, and alignment. Many of these attributes are outdated in HTML5, but they still help in understanding older code. For modern projects, CSS is the preferred option.

Most common attributes are:

- **border:** The **HTML <table> border Attribute** is used to specify the border of a table. It sets the border around the table cells. This attribute defines the visual presentation of the table by setting the thickness of the borders. A higher value results in a thicker border. Alternatively, setting the border attribute to "0" removes the borders entirely.

  Syntax:
  <table border="value">
  The value represents the thickness of the border in pixels.
  border="1": This means the table will have a border of 1 pixel.
  border="0": This will remove the borders from the table, leaving it borderless.

```html
<html>
<head>
<style>
table, th, td {
  border: 1px solid black;
  border-collapse: collapse;
}
</style>
</head>
<body>
    <h1>This is Border Attribute</h1>
    <h2>HTML table border Attribute</h2>
    <table border="1">
        <caption>
            Author Details
        </caption>

        <tr>
            <th>NAME</th>
            <th>AGE</th>
            <th>BRANCH</th>
        </tr>
        <tr>
            <td>Shalini</td>
            <td>22</td>
            <td>CSE</td>
        </tr>
        <tr>
            <td>RAM</td>
            <td>21</td>
            <td>ECE</td>
        </tr>
    </table>
</body>
</html>
```

Output:

This is Border Attribute

HTML table border Attribute

Author Details

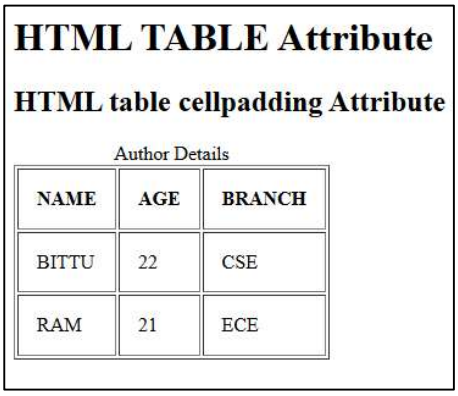
NAME	AGE	BRANCH
Shalini	22	CSE
RAM	21	ECE

- **Cellpadding:** The **HTML <table> cell padding Attribute** is used to specify the space between the cell content and cell wall. The cellpadding attribute is set in terms of pixels. By default, the padding is set to 0.
  Syntax: <table cellpadding="pixels">
  Note: The <table> cellpadding Attribute is not supported by HTML5.

```html
<html>
<head>
    <title>
        HTML table cellpadding Attribute
    </title>
</head>

<body>
    <h1>HTML TABLE Attribute</h1>

    <h2>
        HTML table cellpadding Attribute
    </h2>

    <table border="1" cellpadding="15">
        <caption>Author Details</caption>

        <tr>
            <th>NAME</th>
            <th>AGE</th>
            <th>BRANCH</th>
        </tr>
        <tr>
            <td>BITTU</td>
            <td>22</td>
            <td>CSE</td>
        </tr>
        <tr>
            <td>RAM</td>
            <td>21</td>
            <td>ECE</td>
        </tr>
    </table>
</body></html>
```

Output:

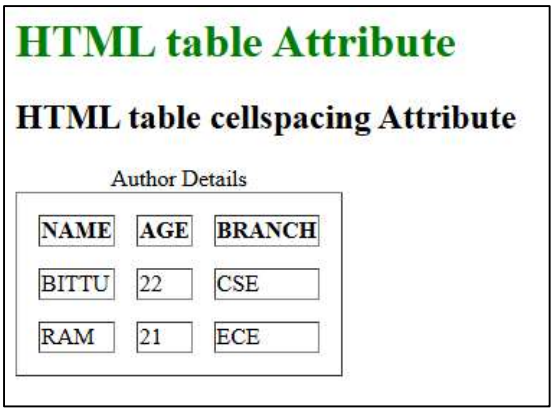
:



- **Cellspacing:** The cellspacing attribute in HTML is used to define the space between cells in a table. It controls the amount of space, in pixels, between the table cells. This spacing can make the table's content more readable by adding extra room between cells.
  Syntax: <table cellspacing="pixels">

```
index.html > html > head > title
<html>

<body>
    <h1 style="color: green;">
      HTML table Attribute
      </h1>

    <h2>HTML table cellspacing Attribute</h2>

    <table border="1" cellspacing="15">
        <caption>
          Author Details
          </caption>

        <tr>
            <th>NAME</th>
            <th>AGE</th>
            <th>BRANCH</th>
        </tr>
        <tr>
            <td>BITTU</td>
            <td>22</td>
            <td>CSE</td>
        </tr>
        <tr>
            <td>RAM</td>
            <td>21</td>
            <td>ECE</td>
        </tr>
    </table>
</body></html>

```

Output:



- **Align:** The **HTML <table> align Attribute** specify the table's alignment but CSS properties like margin and text-align are preferred for table alignment. To align content within table rows or cells, use the align attribute within <tr> and <td> tag or apply CSS styles
  Syntax:
  <table align="left | right | center">

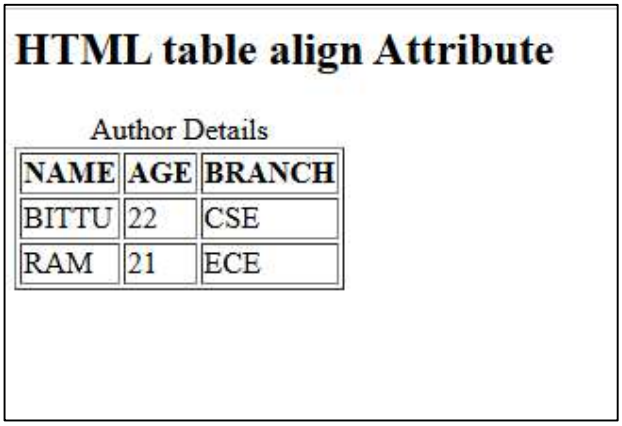
Table Alignment Attributes

Attribute	Description
left	Aligns the table to the left side of the page. This is the default setting.
right	Aligns the table to the right side of the page.
center	Aligns the table in the horizontal center of the page.

Example:

```html
<html><head>
    <title>
        HTML table align Attribute
    </title>
</head>

<body>

    <h2>HTML table align Attribute</h2>
    <table align="left" border="1">
        <caption>
            Author Details
        </caption>
        <tr>
            <th>NAME</th>
            <th>AGE</th>
            <th>BRANCH</th>
        </tr>
        <tr>
            <td>BITTU</td>
            <td>22</td>
            <td>CSE</td>
        </tr>
        <tr>
            <td>RAM</td>
            <td>21</td>
            <td>ECE</td>
        </tr>
    </table>
</body>
</html>

```

Output:



2.2 HTML Table: Nested Tables, Accessibility in Tables, Practical

2.2.0 Nested Tables

In HTML, tables are created using the <table> tag. To nest a table, put one table inside another. The outer table is your main table, while the inner table is the one you are nesting inside.

To create nested Table:

- First, you create the outer table using the <table> tag.
- Then, within one of the <td> tags (which are used to define individual cells in the table), you insert another <table> tag to create the inner table.

Example:

```
<html><head>
</head>
<body>
    <h2>About Table</h2>
    <p><b>Nested tables</b></p>

    <table border="2">
        <tr>
            <td>Main Table row 1 column 1</td>
            <td>Main Table column 2
                <table border="2">
                    <tr>
                        <td>Inner Table row 1 column 1</td>
                        <td>Inner Table row 1 column 2</td>
                    </tr>
                    <tr>
                        <td>Inner Table row 2 column 1</td>
                        <td>Inner Table row 2 column 2</td>
                    </tr>
                    <tr>
                        <td>Inner Table row 3 column 1</td>
                        <td>Inner Table row 3 column 2</td>
                    </tr>
                </table>
            </td>
        </tr>
        <tr>
            <td>Main Table row 2 column 1</td>
            <td>Main Table row 2 column 2</td>
        </tr>
    </table>
</body>
</html>
```

Output:

Nested tables

Main Table row 1 column 1	Main Table column 2	
	Inner Table row 1 column 1	Inner Table row 1 column 2
	Inner Table row 2 column 1	Inner Table row 2 column 2
	Inner Table row 3 column 1	Inner Table row 3 column 2
Main Table row 2 column 1	Main Table row 2 column 2	

2.2.1 HTML Table Accessibility

Tables help organize data visually, but **screen readers require additional information** to interpret them correctly. Without proper markup, users with visual impairments cannot understand the relationship between rows and columns as easily as sighted users. This is why **semantic HTML elements** are crucial.

Main Accessibility Features:

1. **Use Semantic Tags (<thead>, <tbody>, <tfoot>)**
   These tags provide structure and meaning to your table:

   a) **<thead>**: Groups the header rows of the table.
   b) **<tbody>**: Contains the main body rows (data rows).
   c) **<tfoot>**: Groups the footer rows, often used for totals or summaries.

   Using these tags helps assistive technologies and search engines understand the role of each section. It also improves styling and layout control with CSS.

2. **Add scope Attribute for Headers**
   The scope attribute clarifies whether a header cell applies to a row, a column, or a group:
   a) **scope="col"** → Column header
   b) **scope="row"** → Row header
   c) **scope="colgroup"** → Multiple columns
   d) **scope="rowgroup"** → Multiple rows

   This ensures that screen readers can associate the correct headers with data cells, improving accessibility for visually impaired users.

3. **Use <caption> for Description**
   The <caption> tag provides a brief description of the table's purpose and content.

- Placed immediately after the <table> tag.
- Helps users (including those using screen readers) understand the context without reading all the data.

Example:

```
<table>
  <caption>Student Attendance Report</caption>
</table>
```

4. **Add aria-describedby for Screen Readers**
   The aria-describedby attribute can link the table to additional descriptive text (like a summary or explanation) outside the table.

Example:

```
<div id="table-info">This table shows student attendance for the month of July.</div>
<table aria-describedby="table-info">
  ...
</table>
```

5. **Avoid Using Tables for Layout Purposes**
   Tables should **only** be used for displaying tabular data—not for page layout.

- Using tables for layout creates confusion for screen readers and breaks semantic structure.
- For layouts, use **CSS grid or flexbox** instead.

2.2.2 Practical Questions

Creating and Using Tables

1. Create a simple HTML table to display the following student data: **Name, Roll No, Subject, Marks** for at least **5 students**.
2. Add a **caption** to the table describing the data.
3. Apply basic table structure with <thead>, <tbody>, and <tfoot> tags.

```html
<html><head>
    <title>
        HTML table
    </title>
</head>
<body>
    <table border="">
        <caption>Student Marks Data</caption>
        <thead>
            <tr>
                <th scope="col">Name</th>
                <th scope="col">Roll No</th>
                <th scope="col">Subject</th>
                <th scope="col">Marks</th>
            </tr>
        </thead>
        <tbody>
            <tr><td>Amit</td><td>101</td><td>Math</td><td>90</td></tr>
            <tr><td>Riya</td><td>102</td><td>Science</td><td>88</td></tr>
            <tr><td>Karan</td><td>103</td><td>English</td><td>75</td></tr>
            <tr><td>Pooja</td><td>104</td><td>Math</td><td>82</td></tr>
            <tr><td>Rahul</td><td>105</td><td>Science</td><td>95</td></tr>
        </tbody>
        <tfoot>
            <tr>
                <td colspan="4">Total Students: 5</td>
            </tr>
        </tfoot>
    </table>

</body>
</html>

```

Output:

Student Marks Data

Name	Roll No	Subject	Marks
Amit	101	Math	90
Riya	102	Science	88
Karan	103	English	75
Pooja	104	Math	82
Rahul	105	Science	95
Total Students: 5			

Rowspan and Colspan

Create a timetable for your class using HTML tables:

1. Use rowspan for subjects that cover multiple time slots.
2. Use colspan for breaks or combined sessions.

```html
<html>
<head>
</head>
<body>
    <table border="1" align="center">
        <caption>Class Timetable</caption>
        <thead>
            <tr>
                <th>Day</th>
                <th>8am-9am</th>
                <th>9am-10am</th>
                <th>10am-11am</th>
                <th>12pm-1pm</th>
            </tr>
        </thead>
        <tbody>
            <tr>
                <td>Mon</td>
                <td rowspan="2">Math</td>
                <td>Science</td>
                <td colspan="2">Lab</td>
            </tr>
            <tr>
                <td>Tue</td>
                <td>English</td>
                <td>Break</td>
                <td>History</td>
            </tr>
        </tbody>
    </table>
</body>
</html>
```

Output:

Class Timetable

Day	8am-9am	9am-10am	10am-11am	12pm-1pm
Mon	Math	Science	Lab	
Tue		English	Break	History

Nested Tables

Create a table where one of the cells contains **another table** (nested table) showing additional details.

```html
<!DOCTYPE html>
<html>
<head>
    <title>Order List</title>

</head>
<body>
    <table border="1" align="center">
        <caption>Order Details</caption>
        <tr>
            <th>Order ID</th>
            <th>Customer</th>
            <th>Items</th>
        </tr>
        <tr>
            <td>101</td>
            <td>Amit</td>
            <td>
                <table>
                    <tr><th>Item</th><th>Qty</th></tr>
                    <tr><td>Laptop</td><td>1</td></tr>
                    <tr><td>Mouse</td><td>2</td></tr>
                </table>
            </td>
        </tr>
    </table>
</body>
</html>
```

Output:

Order Details

Order ID	Customer	Items
101	Amit	Item Qty Laptop 1 Mouse 2

2.3 HTML Form: The <form> Element, Input Elements, Label and Field set

2.3.0 HTML Form

Forms add the ability to web pages to not only provide the person viewing the document with dynamic information but also to obtain information from the person viewing it, and process the information.

Objectives: Upon completing this section, you should be able to

- Create a FORM.
- Add elements to a FORM.
- Define CGI (Common Gateway Interface).
- Describe the purpose of a CGI Application.
- Specify an action for the FORM.
- Forms work in all browsers.
- Forms are Platform Independent.

To insert a form we use the <FORM></FORM> tags. The rest of the form elements must be inserted in between the form tags.

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>HTML Form</title>
</head>
<body>
  <!--form start-->
   <form name="" method="" action="" target="">
   </form>
   <!--form end-->
</body>
</html>
```

2.3.1 Form Element Attributes

- **ACTION**: It is the URL of the CGI (Common Gateway Interface) program that is going to accept the data from the form, process it, and send a response back to the browser.
- **METHOD**: GET (default) or POST specifies which HTTP method will be used to send the form's contents to the web server. The CGI application should be written to accept the data from either method.
- **NAME**: It is a form name used by VBScript or JavaScript.
- **TARGET**: It is the target frame where the response page will show up.

2.3.2 Form Input Elements

HTML form input elements are interactive controls used within an HTML <form> element to collect user input.

- Form elements have properties: Text boxes, Password boxes, Checkboxes, Option (Radio) buttons, Submit, Reset, File, Hidden and Image.
-

The properties are specified in the TYPE Attribute of the HTML element <INPUT></INPUT>.

```
<!DOCTYPE html>
<html lang="en">
 <head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>HTML Form</title>
 </head>
 <body>
 <!--form start-->
  <form name="" method="" action="" target="">
   <input
    type="" name="" value="" checked
    size="" maxlength="" minlength="" required  />
  </form> <!--form end-->
 </body></html>
```

<INPUT> Element's Properties	
TYPE	Type of INPUT entry field
NAME	Variable name passed to CGI application
VALUE	The data associated with the variable name to be passed to the CGI application
CHECKED	Button/box checked
SIZE	Number of visible characters in text field
MAXLENGHT	Maximum number of characters accepted
MINLENGHT	Minimum number of characters accepted

2.33 Form Input Elements, Label and Fieldset

Input Type	Description
<label>	It defines labels for <form> elements.
<input>	It is used to get input data from various types such as text, password, email, etc by changing its type.
<input type="text">	Defines a one-line text input field
<input type="password">	Defines a password field
<input type="submit">	Defines a submit button
<input type="reset">	Defines a reset button
<input type="radio">	Defines a radio button
<input type="email">	Validates that the input is a valid email address.

Input Type	Description
<input type="number">	Allows the user to enter a number. You can specify min, max, and step attributes for range.
<input type="checkbox">	Used for checkboxes where the user can select multiple options.
<input type="date">	Allows the user to select a date from a calendar.
<input type="time">	Allows the user to select a time.
<input type="file">	Allows the user to select a file to upload.
<button>	It defines a clickable button to control other elements or execute a functionality.
<select>	It is used to create a drop-down list.
<textarea>	It is used to get input long text content.
<fieldset>	It is used to draw a box around other form elements and group the related data.
<legend>	It defines a caption for fieldset elements
<datalist>	It is used to specify pre-defined list options for input controls.
<output>	It displays the output of performed calculations.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>HTML Form</title>
</head>
<body>
  <form>
    <fieldset>
      <legend>User Personal Information</legend>
      <label for="name">Enter your full name:</label>
      <input type="text" id="name" name="name" required /><br/>
      <label for="email">Enter your email:</label>
      <input type="email" id="email" name="email" required /><br/>
      <label for="password">Enter your password:</label>
      <input type="password" id="password" name="pass" required /><br/>
      <label for="confirmPassword">Confirm your password:</label>
      <input type="password" id="confirmPassword" name="confirmPass" required /> <br/>
<select>
  <!-- option tag starts -->
  <option>Choose an option</option>
  <option value="html">HTML</option>
  <option value="java">JAVA</option>
  <option value="C++">C++</option>
  <option value="php">PHP</option>
  <!-- option tag ends -->
 </select>
```

```
<label>Enter your gender:</label>
        <div class="gender-group">
          <input type="radio" name="gender" value="male" id="male" required />
          <label for="male">Male</label>
          <input type="radio" name="gender" value="female" id="female" />
          <label for="female">Female</label>
          <input type="radio" name="gender" value="others" id="others" />
          <label for="others">Others</label>
        </div>
<br/>
        <label for="dob">Enter your Date of Birth:</label>
        <input type="date" id="dob" name="dob" required /><br/>
        <label for="address">Enter your Address:</label>
        <textarea id="address" name="address" required></textarea>
<br/>
        <input type="submit" value="Submit" />
      </fieldset>
  </form>
</body>
</html>
```

2.4 HTML Form: Select and Option Tags, TextArea, Button Element

Input Type	Description
<select>	It is used to create a drop-down list.
<option>	It is used to define options in a drop-down list.
<optgroup>	It is used to define group-related options in a drop-down list.
<textarea>	It is used to get input long text content.
<input type="submit">	Defines a submit button
<input type="reset">	Defines a reset button

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>HTML Form</title>
</head>
<body>
  <form>
      <h2>HTML optgroup Tag</h2>
  <select>
```

```
  <select>
     <!-- optgroup tag starts -->
     <optgroup label="Programming Languages">
        <option value="C">C</option>
        <option value="C++">C++</option>
        <option value="Java">Java</option>
     </optgroup>

    <optgroup label="Scripting Language">
        <option value="JavaScript">JavaScript</option>
        <option value="PHP">PHP</option>
        <option value="Shell">Shell</option>
     </optgroup>
     <!-- optgroup tag ends     -->
  </select>
  </form>
</body>
</html>
```

See above example under 2.4 for textarea and button.

2.5 HTML Form: Form Attributes, Input Validation, Accessibility and Usability

2.5.0 Form Attribute See under 2.4 titled Form Attribute.

Attributes that are unique to particular input types—or attributes which are common to all input types but have special behaviors when used on a given input type—are instead documented on those types' pages.

Attributes for the <input> element include the global HTML attributes and additionally:

Attribute	Type(s)	Description
accept	file	Hint for expected file type in file upload controls
alt	image	alt attribute for the image type. Required for accessibility
autocapitalize	all except url, email, and password	Controls automatic capitalization in inputted text.
autocomplete	all except checkbox, radio, and buttons	Hint for form autofill feature
capture	file	Media capture input method in file upload controls
checked	checkbox, radio	Whether the command or control is checked
dirname	hidden, text, search, url, tel, email	Name of form field to use for sending the element's directionality in form submission
disabled	all	Whether the form control is disabled
form	all	Associates the control with a form element
formaction	image, submit	URL to use for form submission
formenctype	image, submit	Form data set encoding type to use for form submission
formmethod	image, submit	HTTP method to use for form submission
formnovalidate	image, submit	Bypass form control validation for form submission
formtarget	image, submit	Browsing context for form submission
height	image	Same as height attribute for <img>; vertical dimension
list	all except hidden, password, checkbox, radio, and buttons	Value of the id attribute of the <datalist> of autocomplete options

max	date, month, week, time, datetime-local, number, range	Maximum value
maxlength	text, search, url, tel, email, password	Maximum length (number of characters) of value
min	date, month, week, time, datetime-local, number, range	Minimum value
minlength	text, search, url, tel, email, password	Minimum length (number of characters) of value
multiple	email, file	Boolean. Whether to allow multiple values
name	all	Name of the form control. Submitted with the form as part of a name/value pair
pattern	text, search, url, tel, email, password	Pattern the value must match to be valid
placeholder	text, search, url, tel, email, password, number	Text that appears in the form control when it has no value set
popovertarget	button	Designates an <input type="button"> as a control for a popover element
popovertargetaction	button	Specifies the action that a popover control should perform

readonly	all except hidden, range, color, checkbox, radio, and buttons	Boolean. The value is not editable
required	all except hidden, range, color, and buttons	Boolean. A value is required or must be checked for the form to be submittable
size	text, search, url, tel, email, password	Size of the control
src	image	Same as src attribute for <img>; address of image resource
step	date, month, week, time, datetime-local, number, range	Incremental values that are valid
type	all	Type of form control
value	all except image	The value of the control. When specified in the HTML, corresponds to the initial value
width	image	Same as width attribute for <img>

```
<!DOCTYPE html>
<html lang="en">
 <head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>HTML Form</title>
 </head>
 <body>
  <form action="page.html" method="post">
   <label>
    Fruit:
    <input type="text" name="fruit" dirname="fruit-dir" value="cherry" />
   </label>
   <input type="submit" />
  </form>
 </body>
</html>
```

2.5.1 Input Validation

Some input types automatically validate format:

- type="email" → must contain @ and .
- type="url" → must start with http:// or https://
- type="number" → must be numeric
- type="date" → must be a valid date

List of Validation Attributes:

- required → field must be filled

- min / max → numeric/date range
- minlength / maxlength → character length limits
- pattern → custom regex rule
- step → numeric intervals
- type → built-in format validation
- novalidate → disable validation

```
<!DOCTYPE html>
<html lang="en">
 <head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>HTML Form</title>
 </head>
 <body>
  <form action="page.html" method="post">
    <label>Email:</label>
<input type="email" name="email" required><br/>
    <label>Age (18-60):</label>
<input type="number" name="age" min="18" max="60" required><br/>
 <label>Username (5-10 chars):</label>
<input type="text" name="username" minlength="5" maxlength="10" required><br/>
 <label>Only Letters:</label>
<input type="text" name="letters" pattern="[A-Za-z]+" required><br/>
  <label>Even Number:</label>
<input type="number" name="even" step="2" required><br/>
 <label>Website:</label><input type="url" name="website" required><br/>
    <input type="submit" />
  </form>
  <!--  <form novalidate>
 <input type="email" name="email" required>
 <input type="submit"></form> -->
 </body>
</html>
```

2.5.2 Accessibility and Usability

When including inputs, it is an accessibility requirement to add labels alongside. This is needed so those who use assistive technologies can tell what the input is for. Also, clicking or touching a label gives focus to the label's associated form control. This improves the accessibility and usability for sighted users, increases the area a user can click or touch to activate the form control. This is especially useful (and even needed) for radio buttons and checkboxes, which are tiny.

See above example under 2.4 and 2.5 how to associate the **<label>** with an **<input>** element.

2.6 What is Git and why use it?, Git setup and config (git config), Creating a local repository: git init

2.6.0 What is Git and why do we use it?

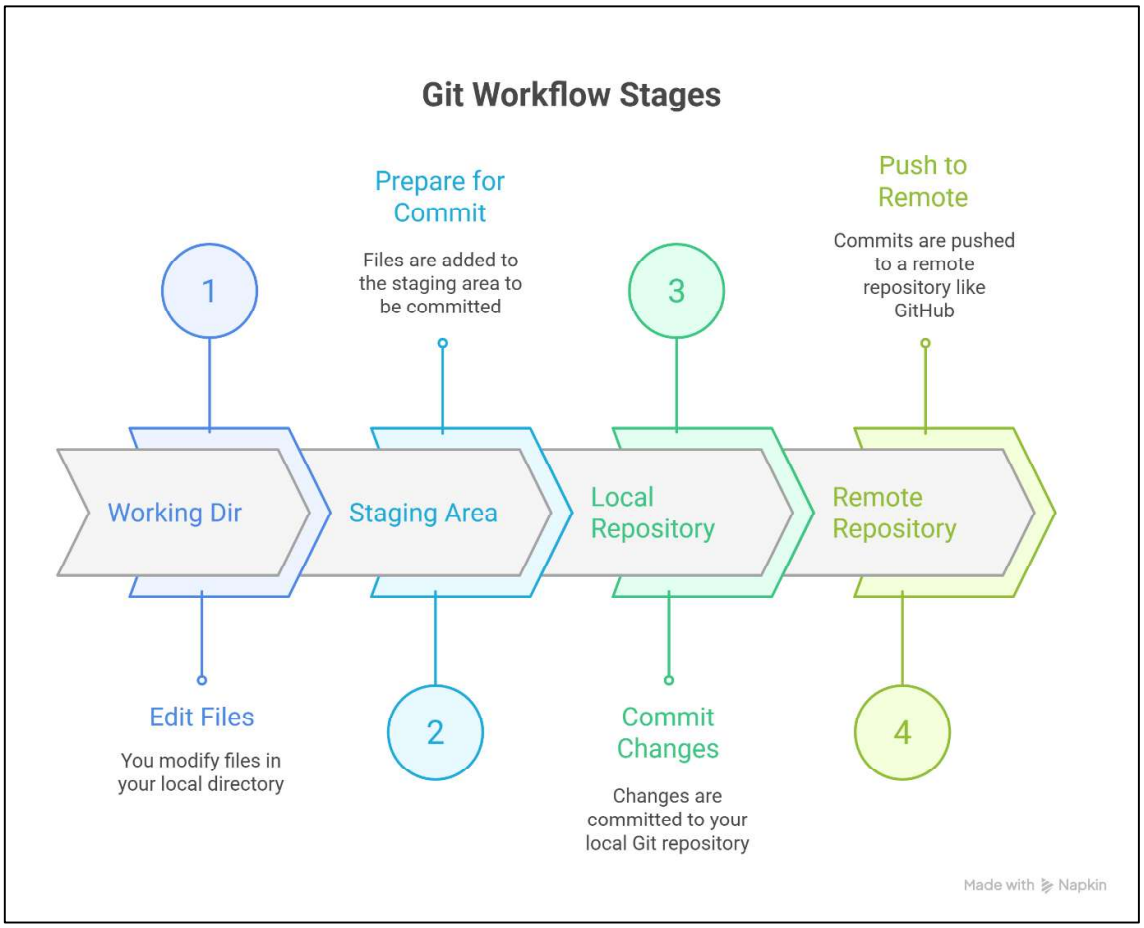
- Git is a **Version Control System (VCS)**, means Every developer has a full copy of repo.
- It helps keep track of every change you make to files in a project.
- If something breaks, you can go back to an earlier version.
- Multiple people can work on the same project without disturbing each other's work.
- Git is widely used in the software industry to manage projects and teamwork.

Key points:

- Git runs on your local computer.
- GitHub is a website (cloud) where Git projects can be stored and shared.
- Together, Git + GitHub = teamwork + backup + tracking changes.

Git has four important areas:

1. **Working Directory** – Where you actually edit files (your project folder).
2. **Staging Area (Index)** – A temporary area where you put the changes you want to commit.
3. **Local Repository** – The permanent history of commits stored on your computer.
4. **Remote Repository (GitHub/GitLab/Bitbucket)** – The copy of your repository stored on the server for sharing and collaboration.



Quick check: In VS Code, open the Terminal and run:

git –version

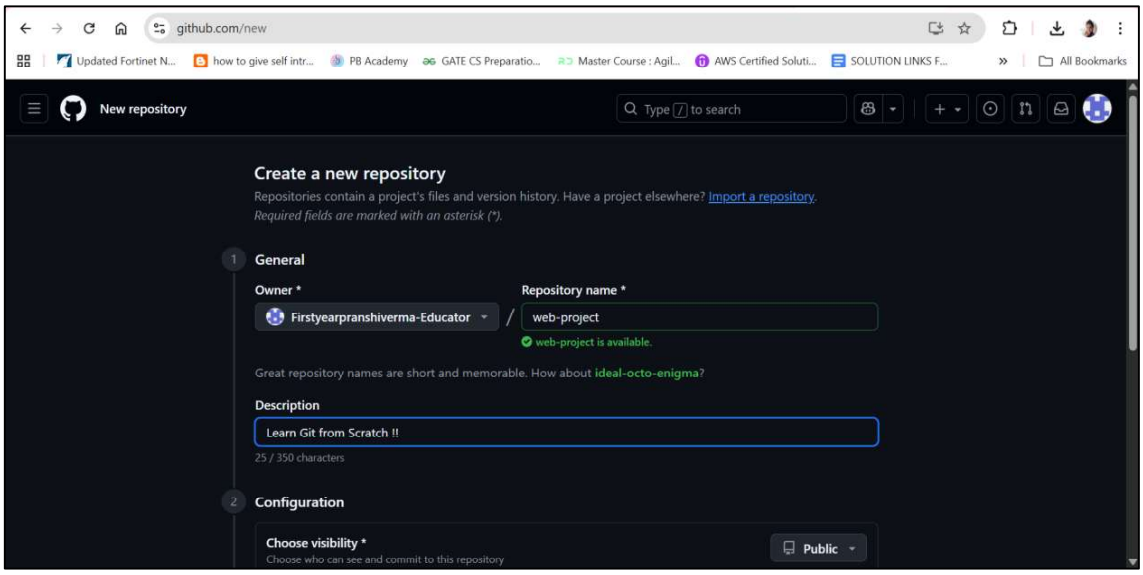
If you see a version (e.g., git version 2.45.x), you're set.

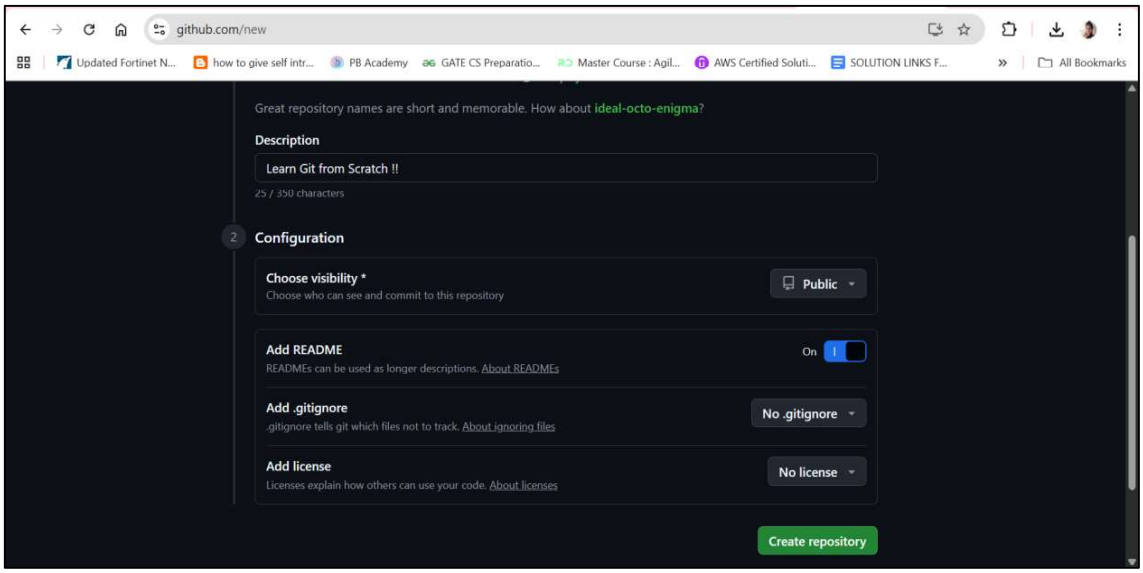
2.6.1 First-time Git setup in VS Code (one-time)

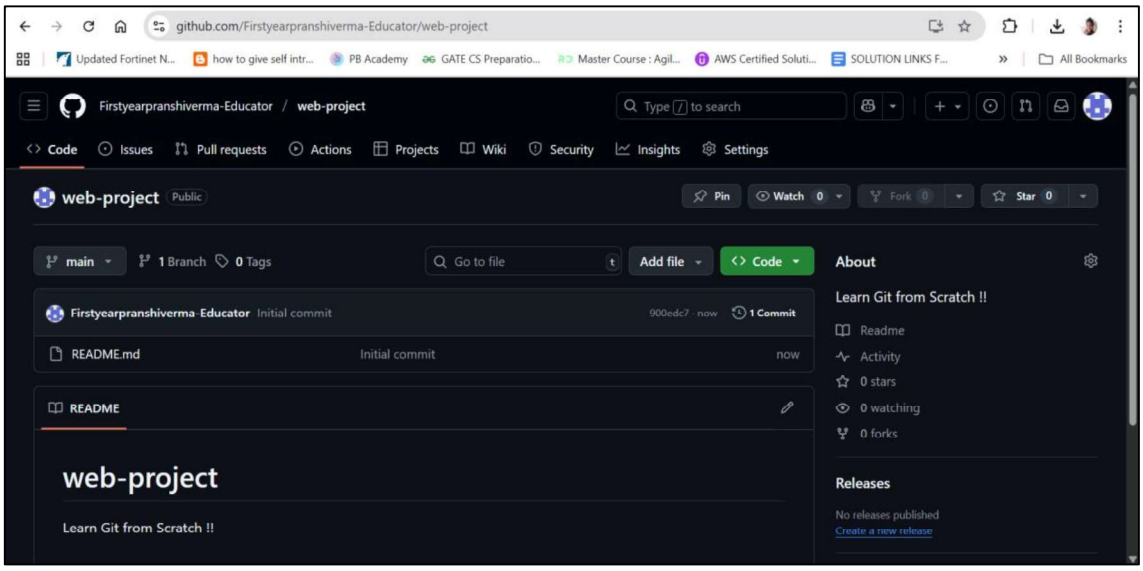
1. Open VS Code → **Terminal** → **New Terminal**.
2. Set your identity (appears in commit history):

Creating a GitHub Account

1. Go to https://github.com/
2. Sign up with email → verify → choose username.
3. Create a new repository:
   - Repository Name: web-project
   - Description: " Learn Git from Scratch !! "
   - Public → Add README.md → Create Repository.


github.com/new
New repository
Type / to search
Create a new repository
Repositories contain a project's files and version history. Have a project elsewhere? Import a repository.
Required fields are marked with an asterisk (*).
1 General
Owner *
Firstyearpranshiverma-Educator
Repository name *
web-project
web-project is available.
Great repository names are short and memorable. How about ideal-octo-enigma?
Description
Learn Git from Scratch !!
25 / 350 characters
2 Configuration
Choose visibility *
Choose who can see and commit to this repository
Public


github.com/new
Great repository names are short and memorable. How about ideal-octo-enigma?
Description
Learn Git from Scratch !!
25 / 350 characters
2 Configuration
Choose visibility *
Choose who can see and commit to this repository
Public
Add README
READMEs can be used as longer descriptions. About READMEs
On
Add .gitignore
.gitignore tells git which files not to track. About ignoring files
No .gitignore
Add license
Licenses explain how others can use your code. About licenses
No license
Create repository



We will use this GitHub Repo to push our project code remotely after few steps.

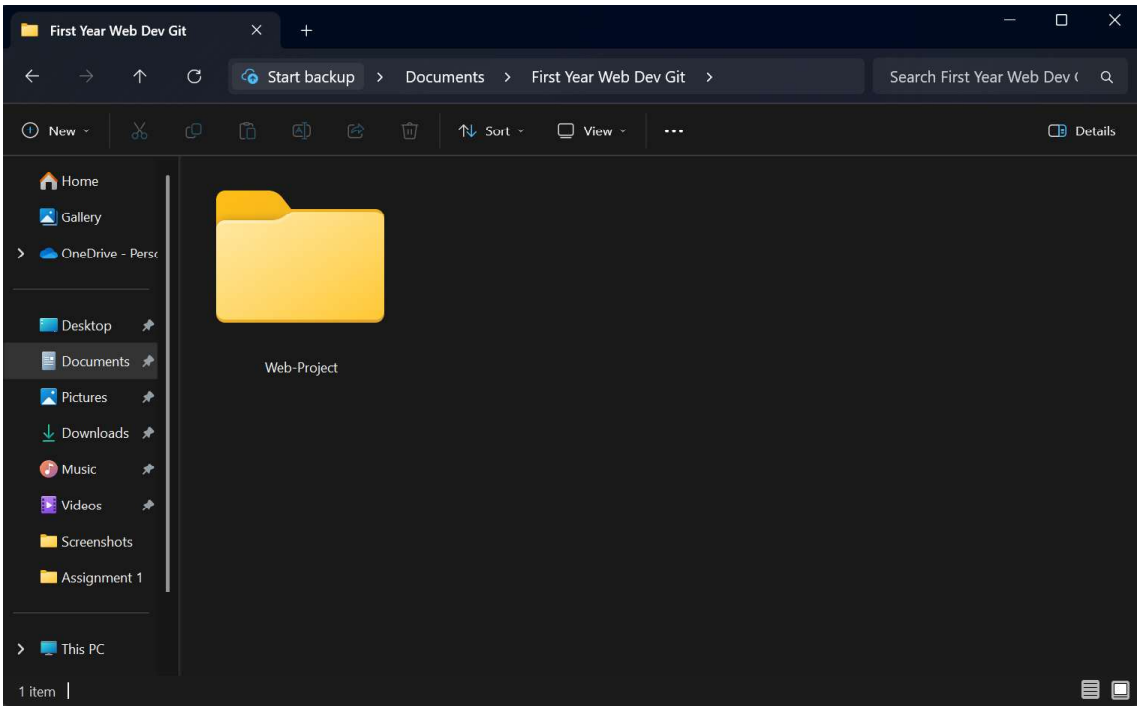
2.6.2 Creating a Local Repository

A **local repository** is a project folder on your computer that Git will track. This is the **first step** before pushing anything to GitHub.

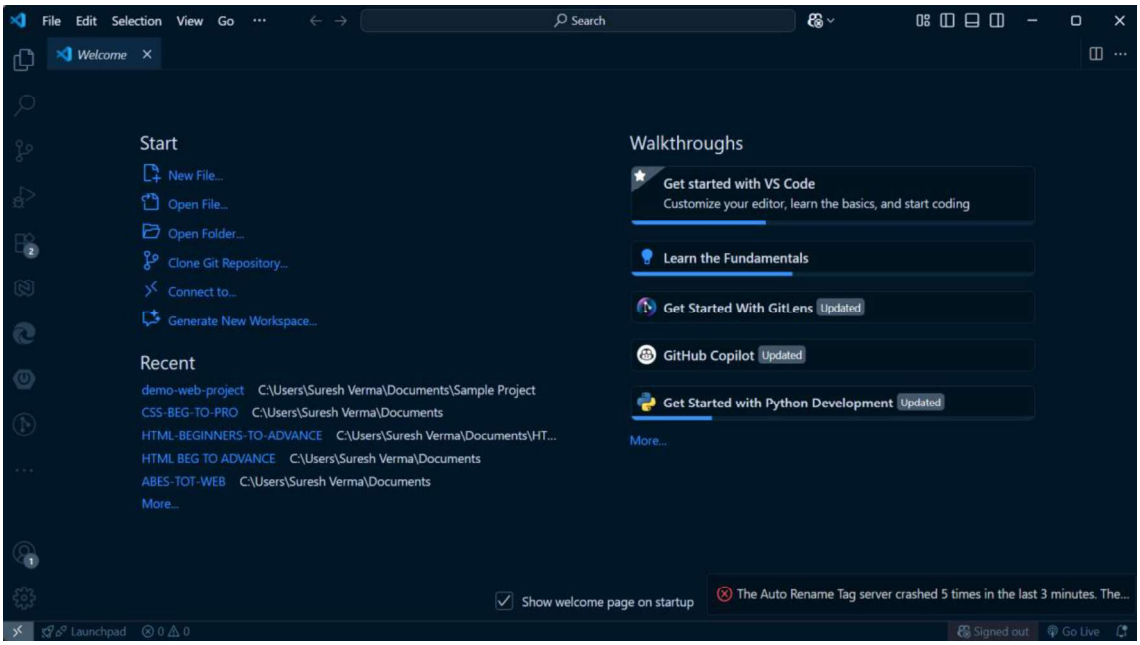
Using VS Code (Beginner-Friendly UI)

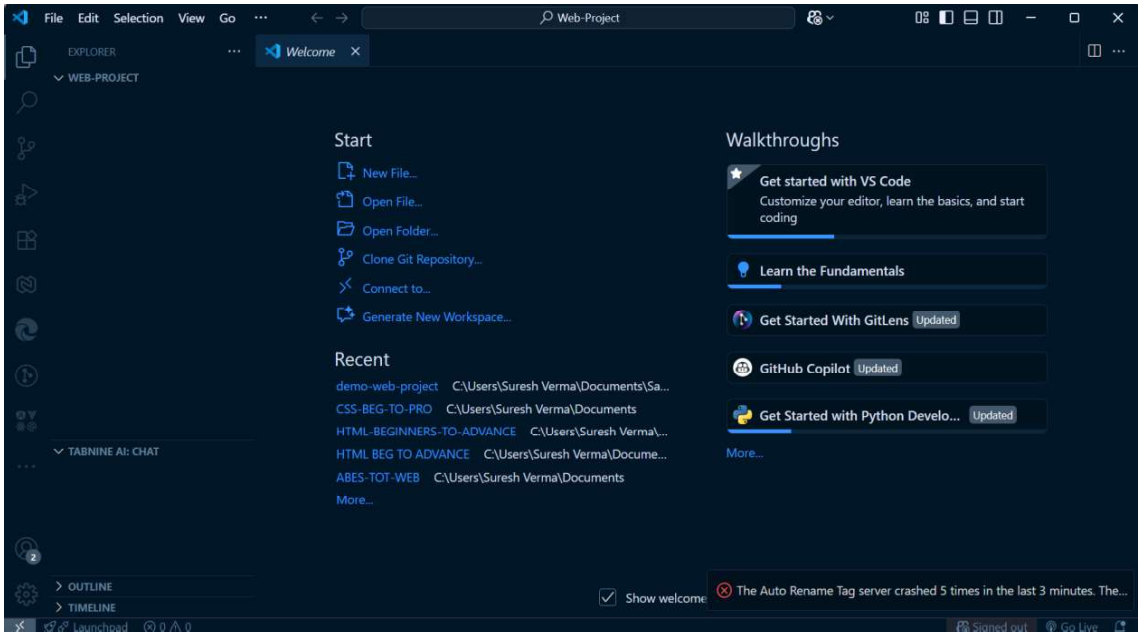
Create a project folder

Example: web-project



- Open it in VS Code (**File → Open Folder**).



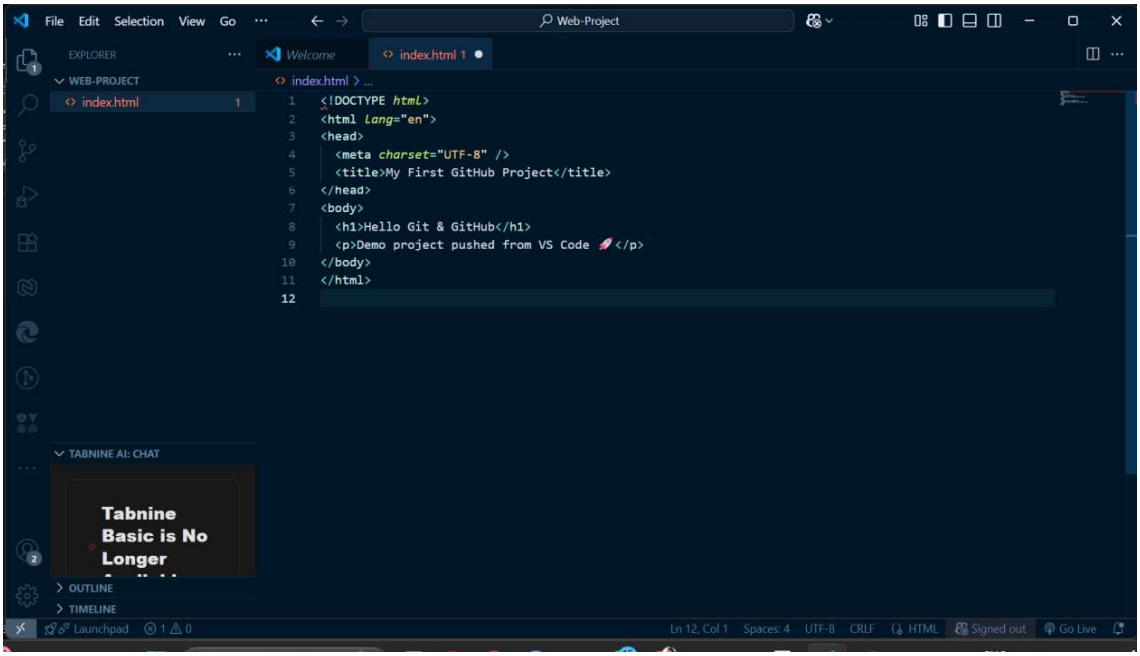


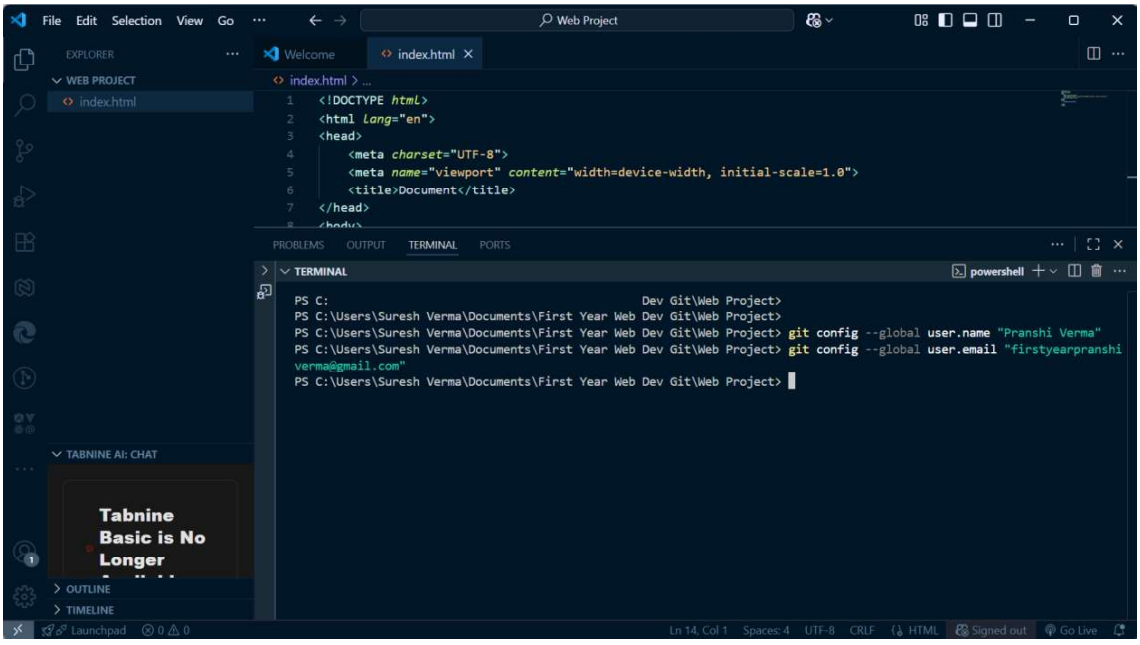
Create project files

Example: index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>My First GitHub Project</title>
</head>
<body>
  <h1>Hello Git & GitHub</h1>
```

```
  <p>Demo project pushed from VS Code </p>
</body>
</html>
```

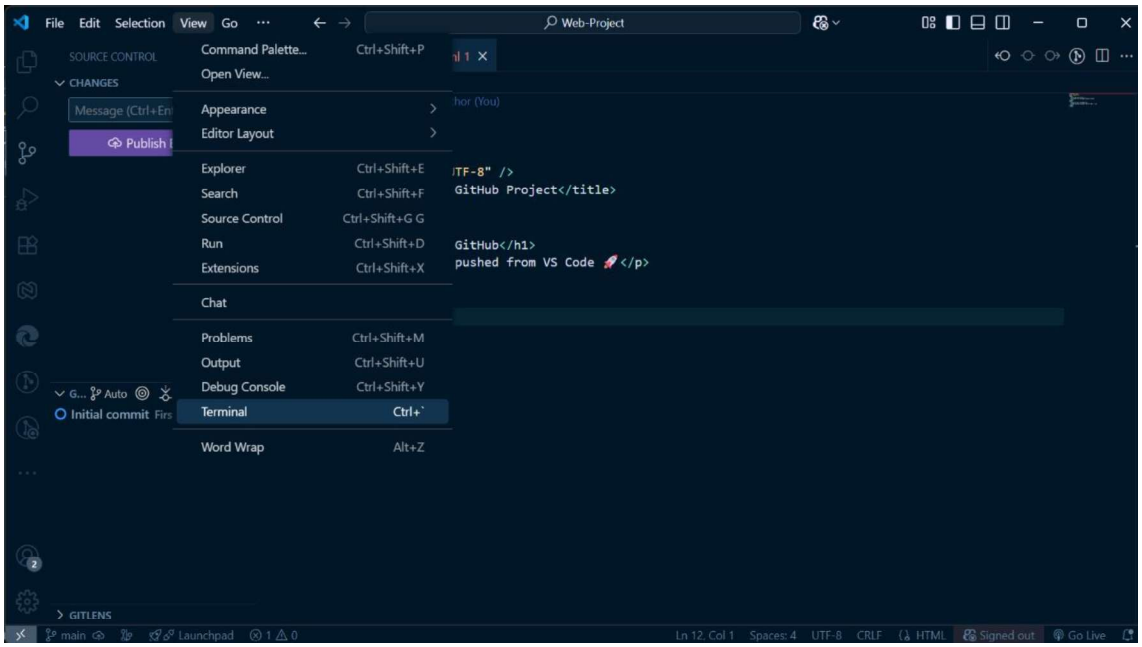




Initialize Git (create local repository)

- **Where to Perform This Step?**

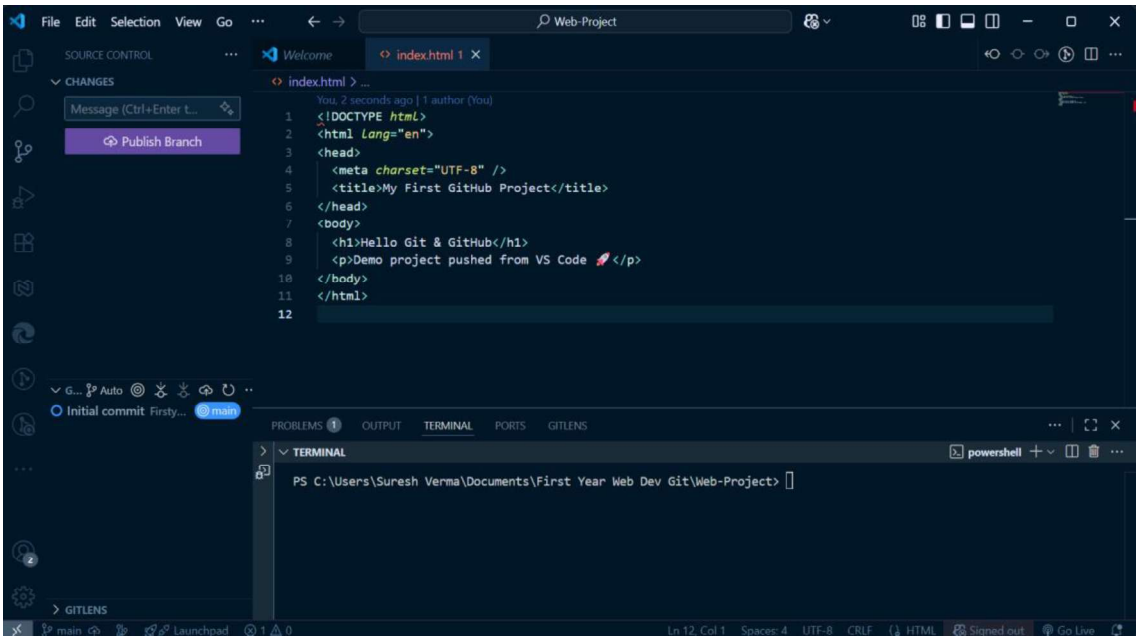
- This step is performed **in your terminal/command prompt (VS Code terminal or Git Bash)** — not on GitHub's website.
- **In terminal:**

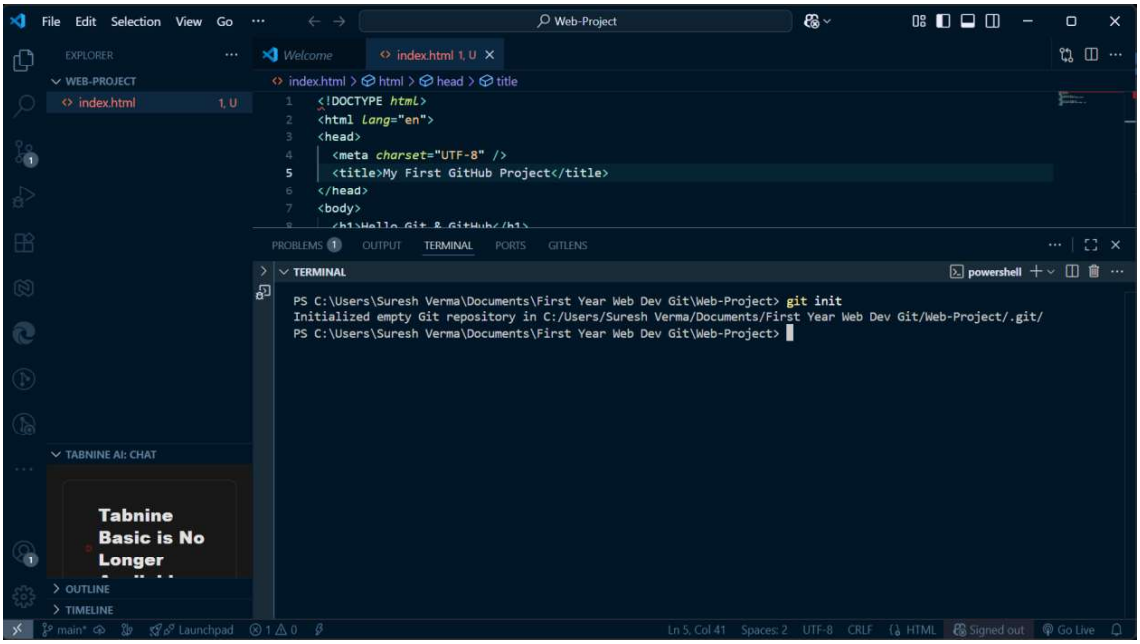


Use the commands:

Initialize the repository:

Using the command git init





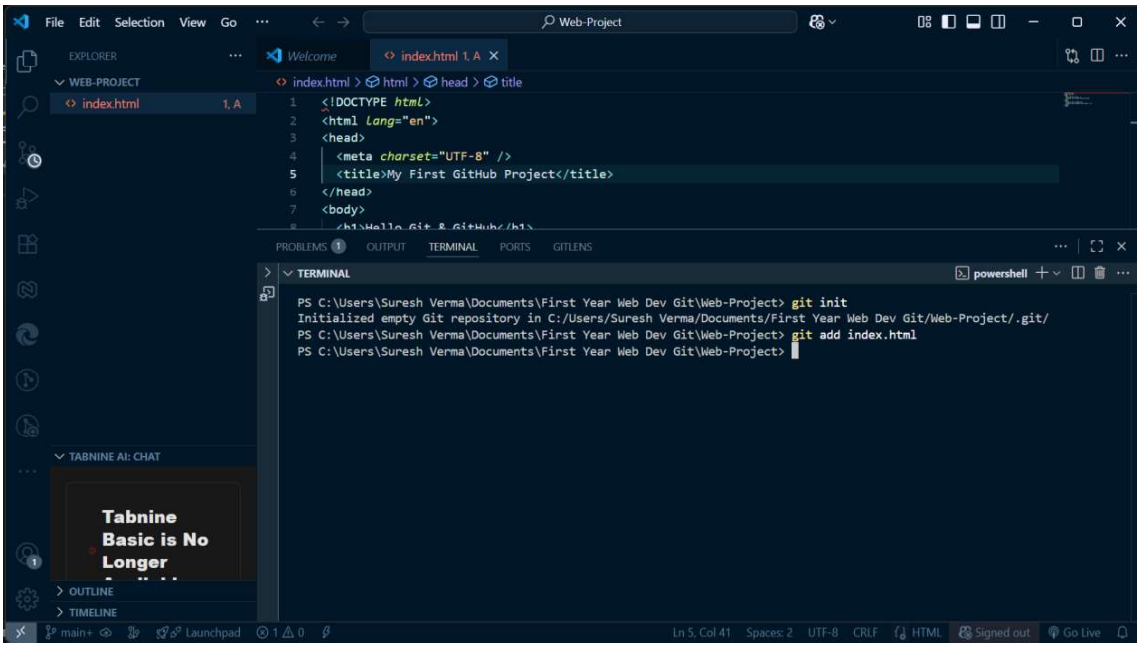
- This command creates a **hidden .git folder**, which means Git is now tracking your project.
- Add and commit files:

2.7 Tracking changes: add, commit, status, log,.gitignore basics,Practical

2.7.0 Add a Single File

If you only want to add **one file** to the staging area (for example, index.html):

git add index.html



Add Multiple Specific Files

You can also add more than one file at a time by naming them:

git add index.html style.css

Add All Files (Recommended when you want to add everything)

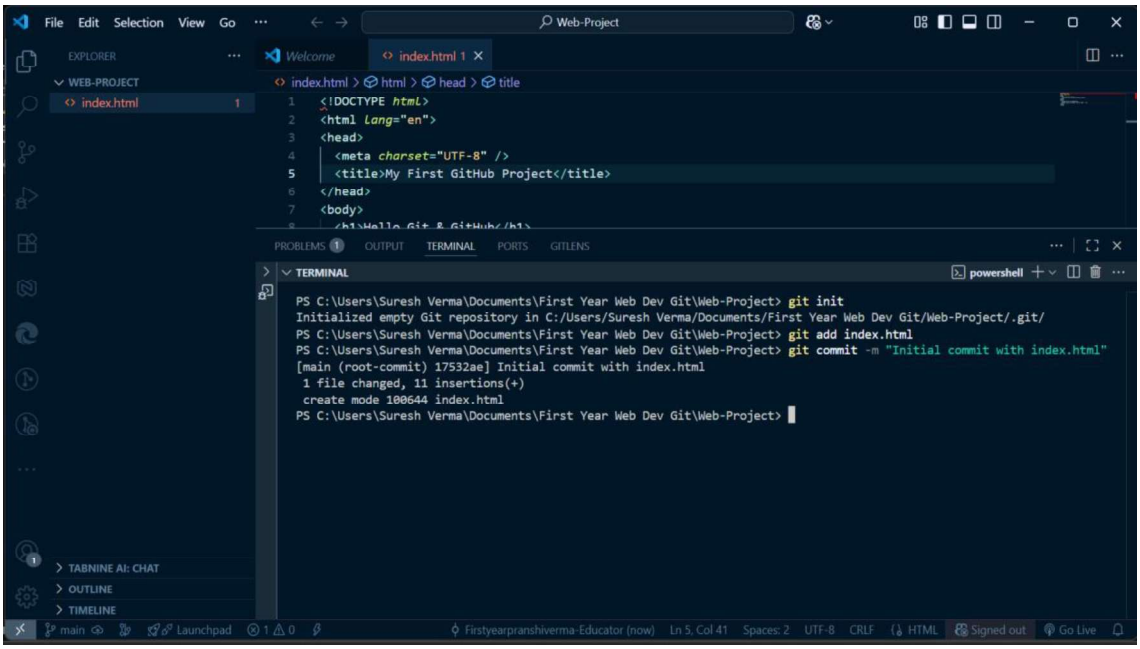
If you want to add **all files in the project folder**:

git add .

2.7.1 Commit the Changes

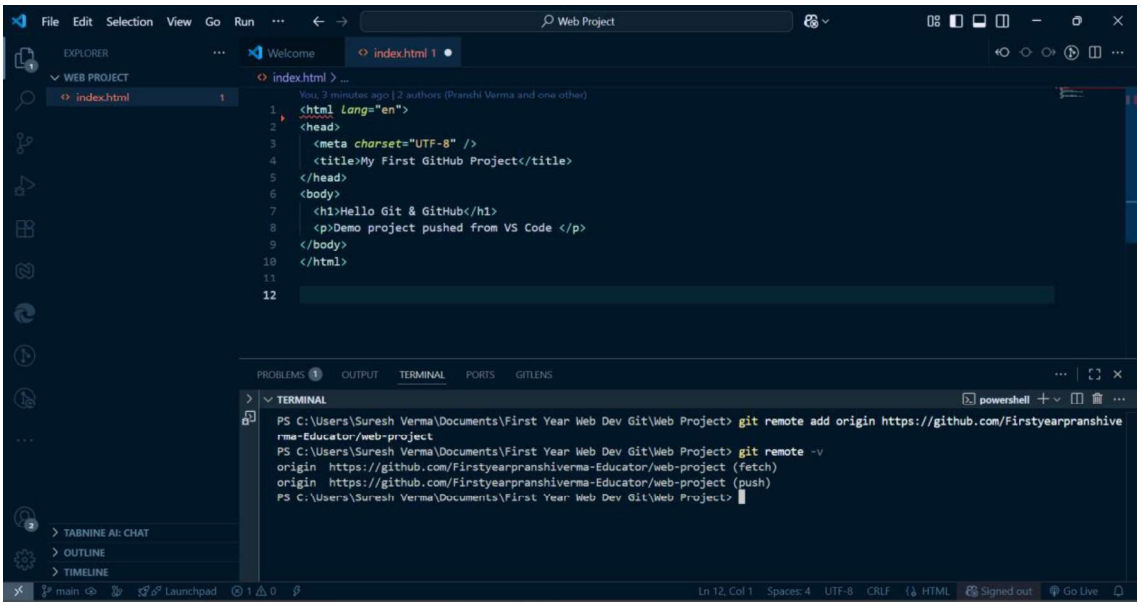
After adding files, you need to **commit** them with a message:

git commit -m "Initial commit with index.html"



A **commit** is like saving a snapshot of your project at that point in time.

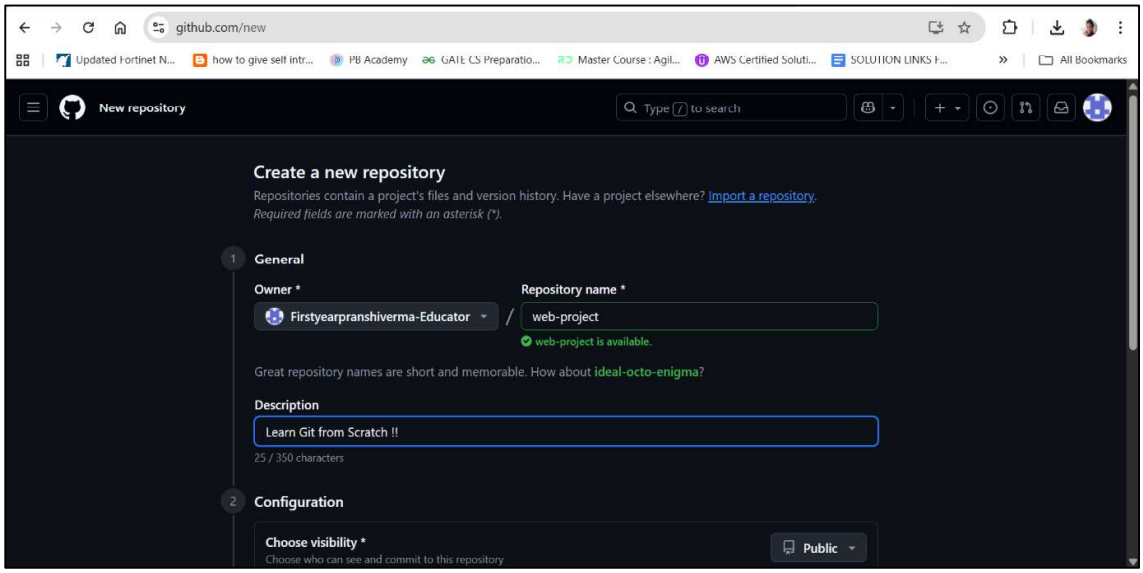
Now, project file is saved in local repository.

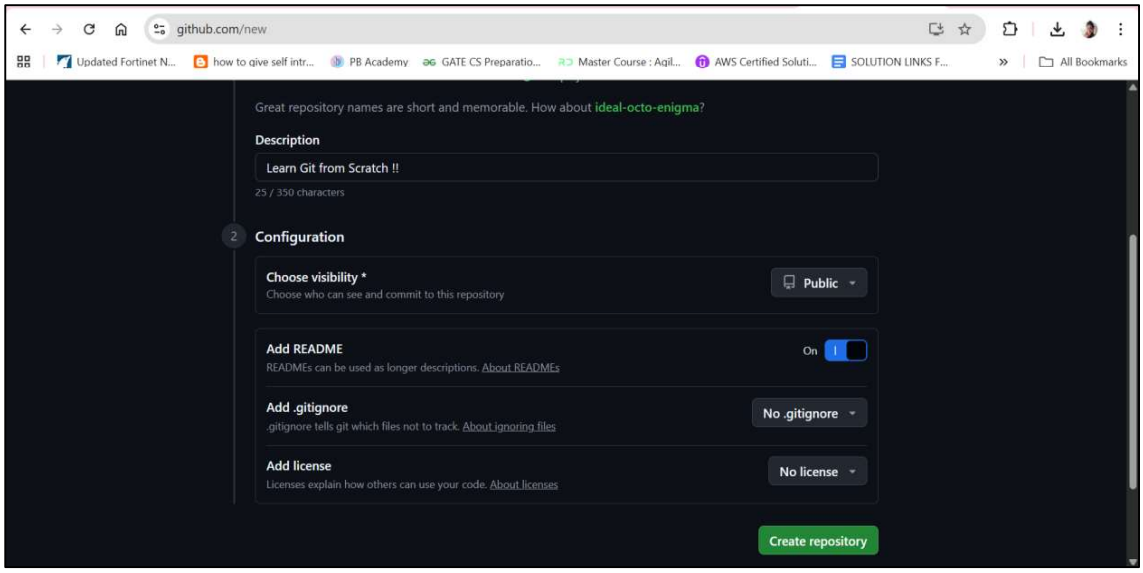


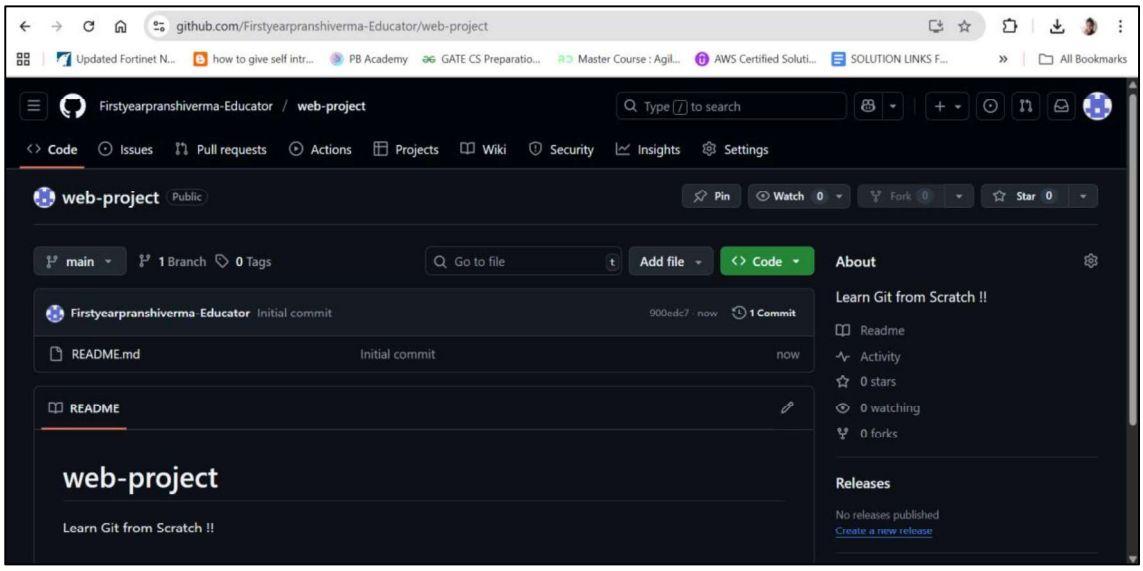
3.0 Creating a GitHub account, Connecting local repos to GitHub: remote add, push

3.0.0 Creating a GitHub Account

1. Go to https://github.com/
2. Sign up with email → verify → choose username.
3. Create a new repository:
    - Repository Name: web-project
    - Description: " Learn Git from Scratch !! "
    - Public → Add README.md → Create Repository.







We will use this GitHub Repo to push our project code remotely after few steps.

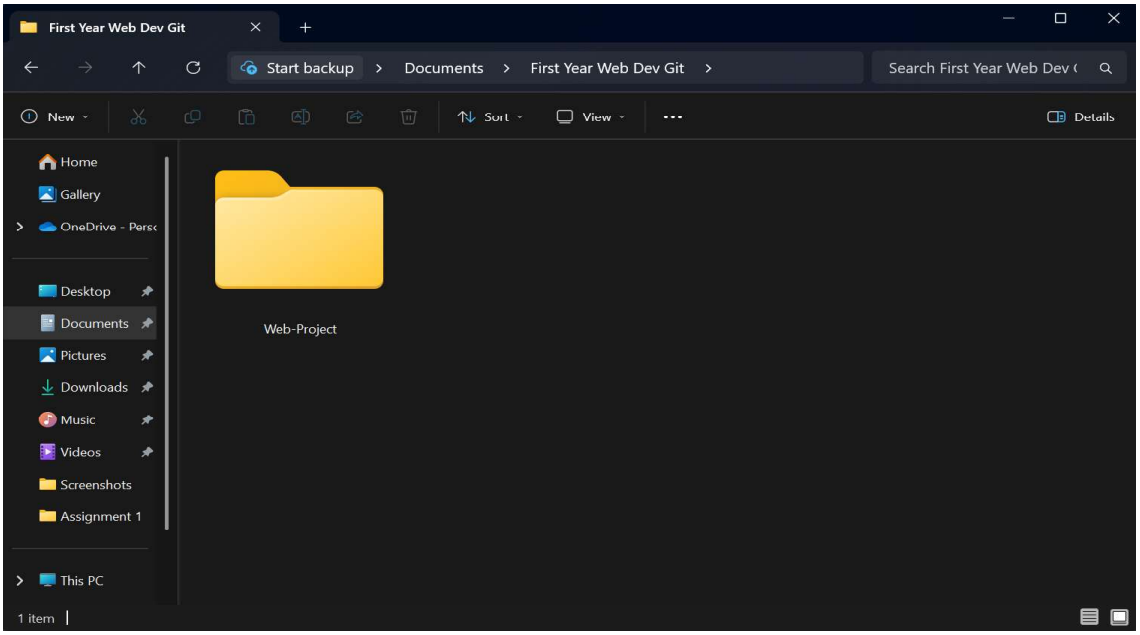
3.0.1 Creating a Local Repository

A **local repository** is a project folder on your computer that Git will track. This is the **first step** before pushing anything to GitHub.

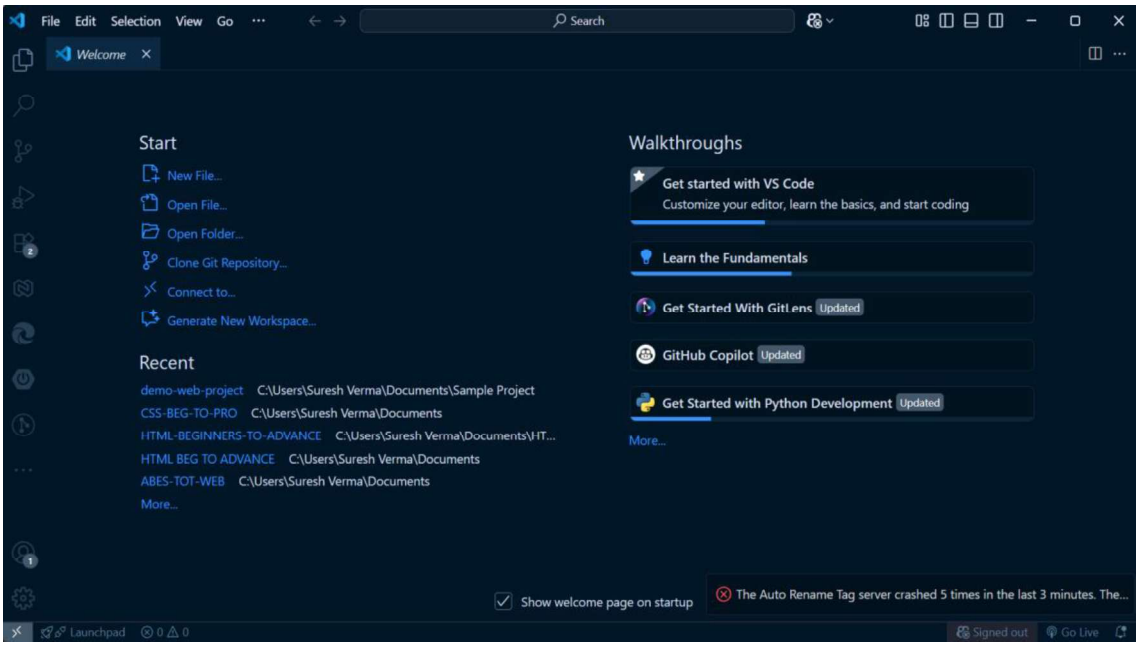
Using VS Code (Beginner-Friendly UI)

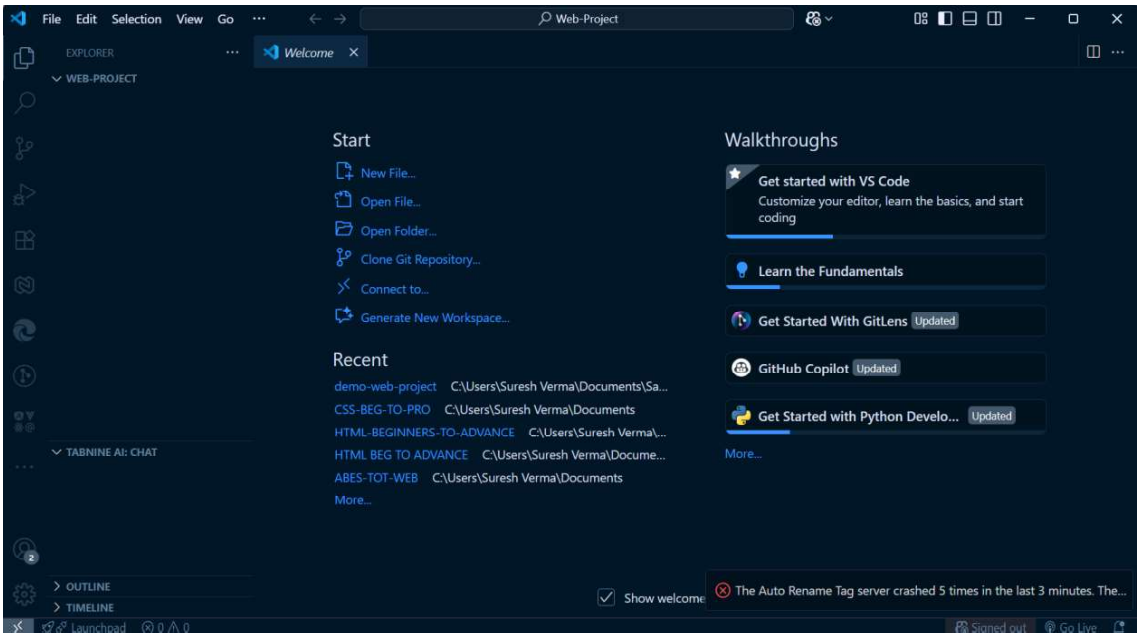
Create a project folder

Example: web-project



- Open it in VS Code (**File → Open Folder**).





Create project files

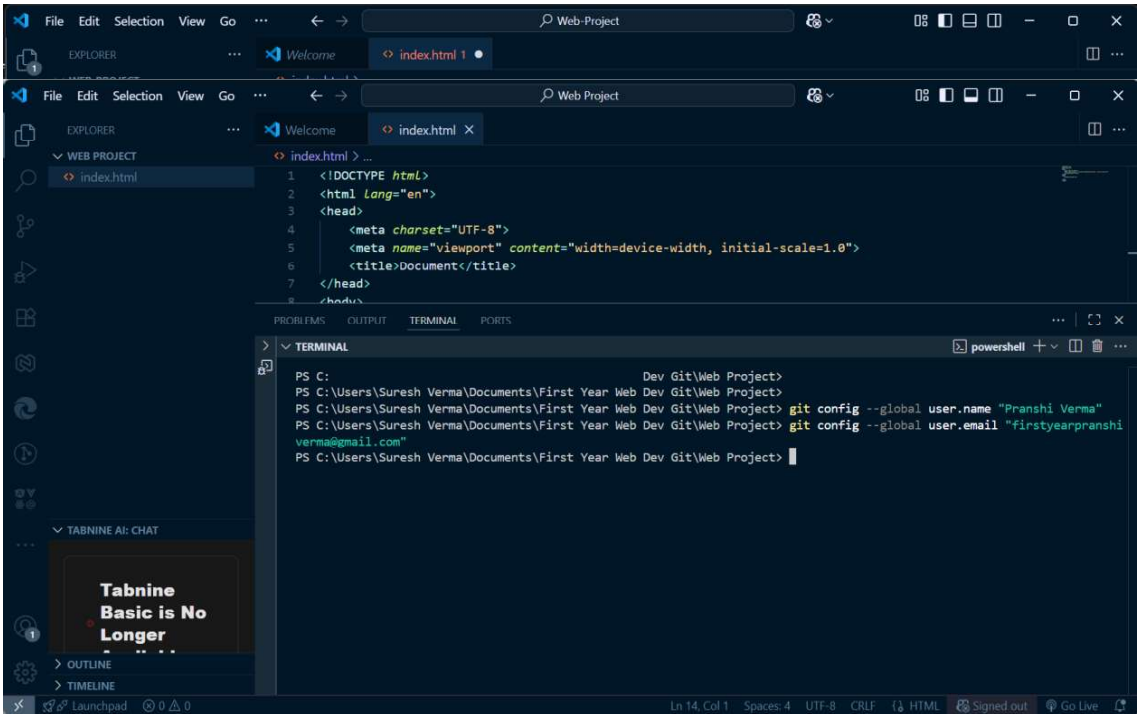
Example:

- index.html

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>My First GitHub Project</title>
</head>
<body>
  <h1>Hello Git & GitHub</h1>
```

```
  <p>Demo project pushed from VS Code </p>

</body>

</html>
```

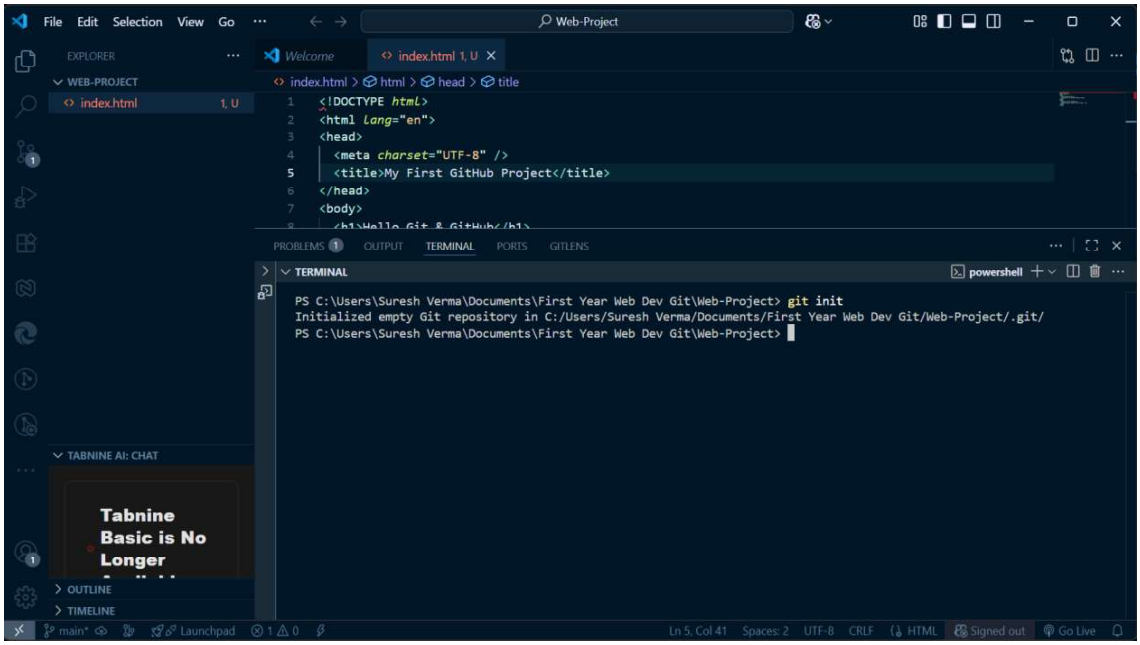


Initialize Git (create local repository)

- **Where to Perform This Step?**
- This step is performed **in your terminal/command prompt (VS Code terminal or Git Bash)** — not on GitHub's website.

In terminal:

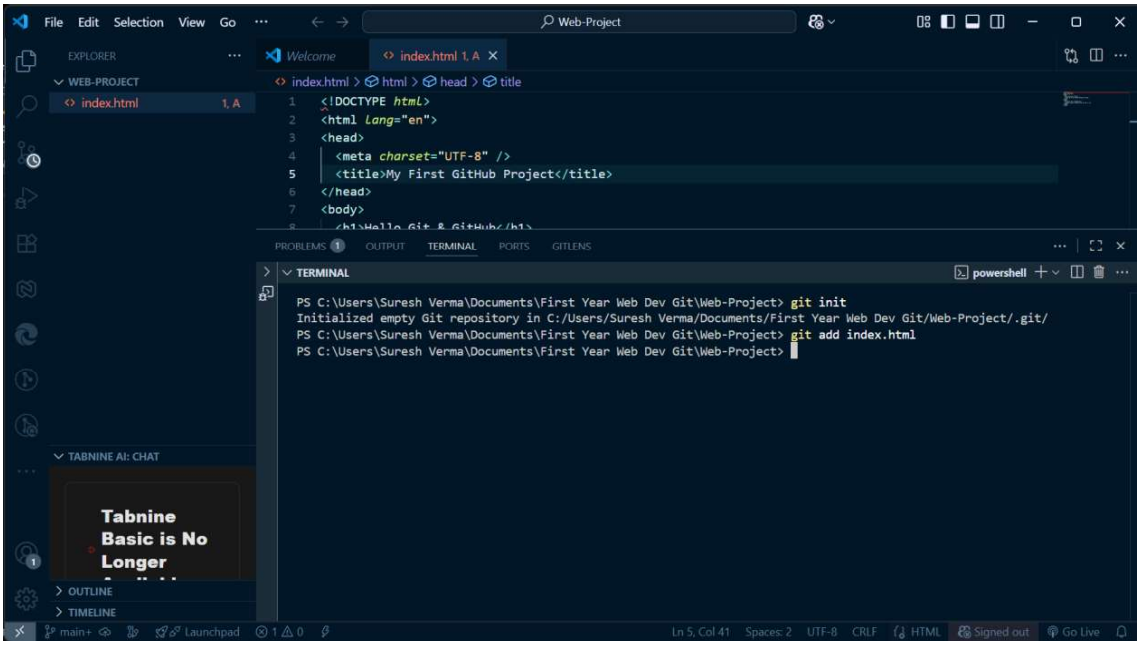
- Use the commands: Using the command git init
- This command creates a **hidden .git folder**, which means Git is now tracking your project.



Add a Single File

If you only want to add **one file** to the staging area (for example, index.html):

git add index.html



Add Multiple Specific Files

You can also add more than one file at a time by naming them:

git add index.html style.css

Add All Files (Recommended when you want to add everything)

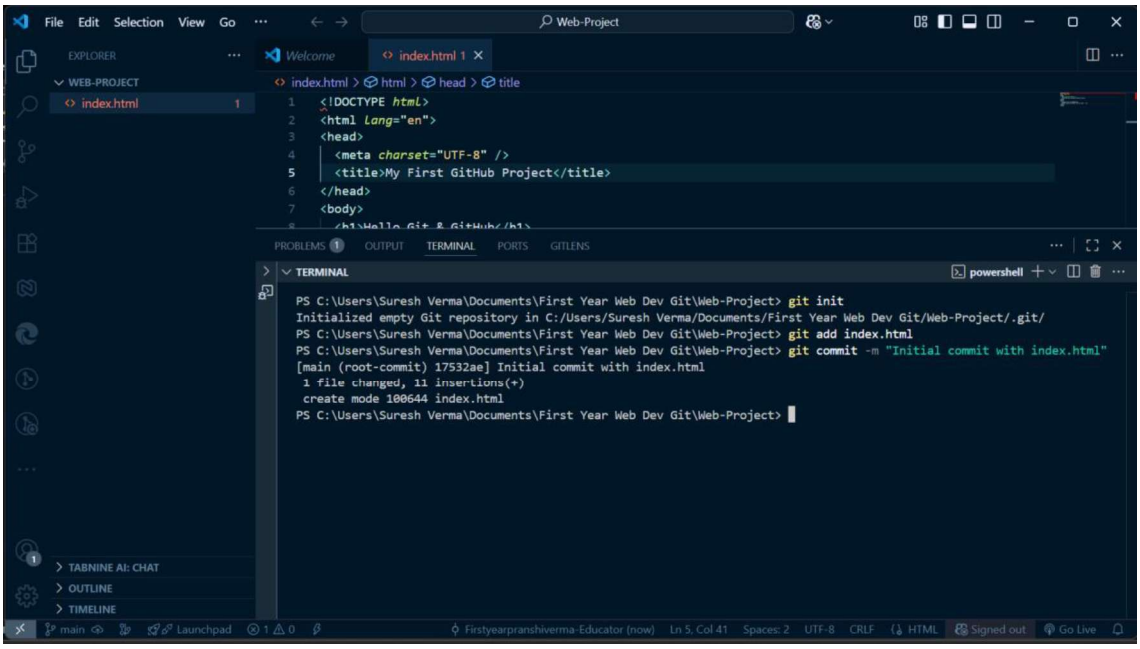
If you want to add **all files in the project folder**:

git add .

Commit the Changes

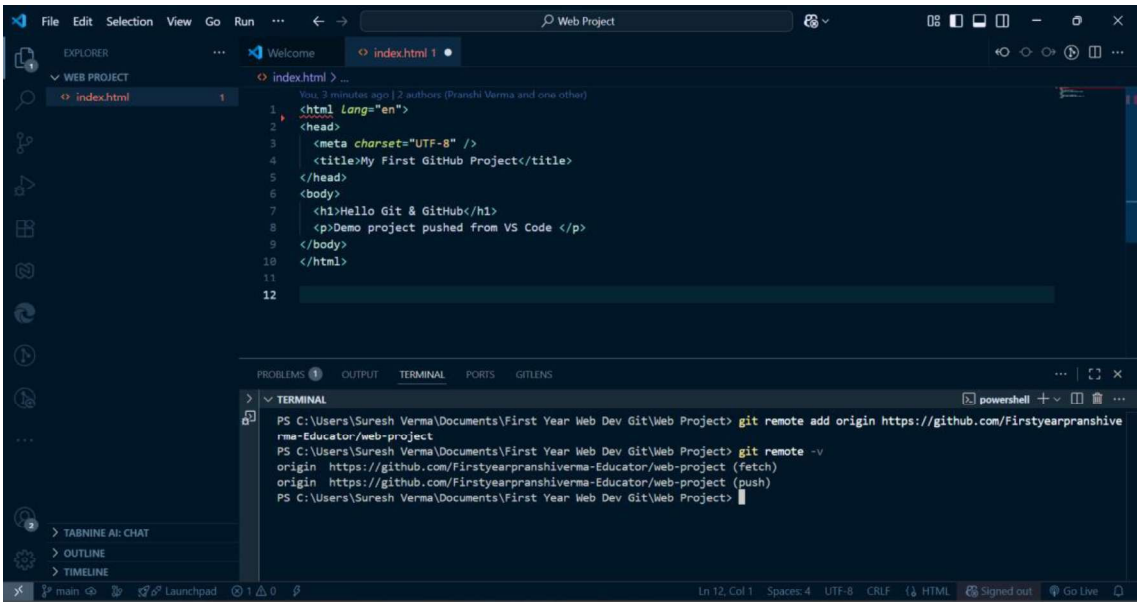
After adding files, you need to **commit** them with a message:

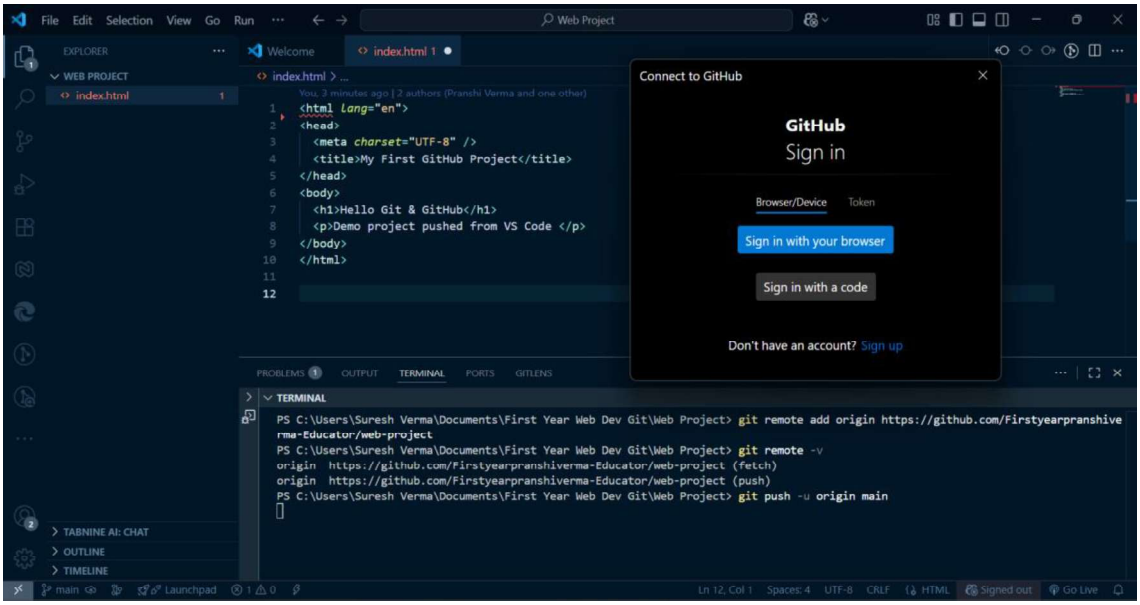
git commit -m "Initial commit with index.html"



A **commit** is like saving a snapshot of your project at that point in time.

Now, project file is saved in local repository.


File Edit Selection View Go Run
Web Project
EXPLORER
WEB PROJECT
index.html
Welcome
index.html 1
<html Lang="en">
<head>
<meta charset="UTF-8" />
<title>My First GitHub Project</title>
</head>
<body>
<h1>Hello Git & GitHub</h1>
Demo project pushed from VS Code
</body>
</html>
PROBLEMS 1 OUTPUT TERMINAL PORTS GITLENS
TERMINAL
powershell
PS C:\Users\Suresh Verma\Documents\First Year Web Dev Git\Web Project> git remote add origin https://github.com/Firstyearpranshiverma-Educator/web-project
PS C:\Users\Suresh Verma\Documents\First Year Web Dev Git\Web Project> git remote -v
origin https://github.com/Firstyearpranshiverma-Educator/web-project (fetch)
origin https://github.com/Firstyearpranshiverma-Educator/web-project (push)
PS C:\Users\Suresh Verma\Documents\First Year Web Dev Git\Web Project>
TABNINE AI: CHAT
OUTLINE
TIMELINE


File Edit Selection View Go Run
Web Project
Connect to GitHub
GitHub
Sign in
Browser/Device
Token
Sign in with your browser
Sign in with a code
Don't have an account? Sign up
PS C:\Users\Suresh Verma\Documents\First Year Web Dev Git\Web Project> git push -u origin main